



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA

Universidad de Huelva

Grado en Ingeniería Informática

Trabajo Fin de Grado

Desarrollo de un sistema IOT en lenguaje Ada
con fines didácticos

Autor: Jorge Enrique Márquez Rodríguez

Tutor: Manuel Sanchez Raya

10 de junio de 2026

Agradecimientos

*“Entre los pecados mayores que los hombres cometen,
yo digo que es el desagradecimiento.”*

— Miguel de Cervantes, *Don Quijote de la Mancha*

La elaboración de este trabajo y, por consiguiente, la finalización de esta fase de mi vida no habrían sido posibles sin el apoyo de las personas que han estado presentes a lo largo de este camino.

A mi familia, por darme los cimientos de lo que soy hoy; por conferirme los valores que tengo y enseñarme lo que quiero llegar a ser.

A mi familia elegida, por los ánimos en los momentos difíciles, por estar cuando es necesario y por hacer más llevadero cada paso de este proceso.

A todos vosotros, gracias.

Resumen

Los sistemas IoT (Internet de las Cosas) están formados por un conjunto de dispositivos capaces de recibir, interpretar, procesar y comunicar datos. Dichos sistemas se componen de elementos físicos —sensores, actuadores y microcontroladores— junto con aplicaciones de software que permiten su funcionamiento en tiempo real. Su desarrollo presenta desafíos significativos en términos de fiabilidad y de gestión eficiente de recursos.

Este trabajo se centra en la creación de un entorno de desarrollo que permita adentrarse en la construcción de estos sistemas de forma actualizada y accesible mediante el lenguaje de programación Ada. Se aborda la configuración del entorno, la integración con hardware moderno, en particular, el microcontrolador STM32F446, y el desarrollo de un marco de trabajo completo para demostrar la viabilidad del lenguaje Ada en este contexto.

Palabras clave: Internet de las cosas (IoT), Ada, tiempo real, sistemas empotrados, STM32F446, entorno de desarrollo, marco de trabajo.

Abstract

IoT (Internet of Things) systems are composed by a set of devices capable of receiving, interpreting, processing and communicating data. Such systems consist of physical elements - sensors, actuators and microcontrollers- with software applications, that enable real-time working. The development is conceived as a challenge in terms of reliability and efficient use of resources.

This project focus on the development of an development environment which allows the construction of this systems in an updated and accessible way, using Ada programming language. This addresses the configuration of the environment, the integration with modern hardware -in particular, the STM32F446 microcontroller- and the creation of a framework to demonstrate the viability of Ada programming language in this context.

Keywords:Internet of Things(IOT), Ada, Real-time, embedded systems, STM32F446, development environment, framework.

Índice general

Agradecimientos	0
Resumen	1
Abstract	2
1 Introducción	9
1.1 Microcontrolador STM32F446	9
1.2 El lenguaje Ada	10
1.3 Objetivos	10
1.4 Motivación	11
1.5 Competencias a desarrollar	12
1.6 Organización del documento	12
1.7 Recursos empleados	13
1.7.1 Hardware	13
1.7.2 Software y toolchain	13
Justificación de la elección de Ada frente a C/C++	14
1.8 Metodología de desarrollo	15
1.8.1 Desarrollo incremental del software	15
1.8.2 Validación por capas	16
1.8.3 Configuración del entorno hardware	16
1.8.4 Modularidad y principio de abstracción	16
2 Microcontrolador STM32F446	17
2.1 Arquitectura ARM Cortex-M4	17
2.2 El microcontrolador STM32F446	17
2.2.1 Características principales	18
2.2.2 ART Accelerator	18
2.2.3 NVIC y gestión de interrupciones	19
2.2.4 Eficiencia energética	19
2.3 Periféricos de mayor interés	19
2.4 Memoria y arquitectura interna	20
2.4.1 Tipos de memoria	20
2.4.2 Mapa de memoria simplificado	20
2.5 Justificación de la elección	21
2.5.1 Coste y disponibilidad	21
2.5.2 Ecosistema STM32	21
2.5.3 Adecuación a aplicaciones IoT	21

2.5.4	Depuración integrada con ST-LINK	22
2.5.5	Soporte del <i>runtime</i> Ada	22
3	Diseño hardware	23
3.1	Fuente de Alimentación	23
3.2	Acceso a Red vía SPI	24
3.3	Interfaz Serie a USB	24
3.4	Tarjeta microSD	25
3.5	Microcontrolador STM32F446	26
3.6	Pulsadores, LEDs, Encoder y ADC	26
3.7	Puerto USB Host	27
3.8	Expansor de E/S vía I2C	28
3.9	Pantalla OLED, GPS y Sensores Externos	28
3.10	Diseño de la PCB	29
4	Diseño software	30
4.1	Runtime Ada	30
4.1.1	El perfil embedded-stm32f4	30
4.1.2	Comparativa de runtimes Ada	30
4.1.3	Proceso de arranque (<i>startup</i>)	31
4.1.4	Linker script y mapa de memoria	32
4.1.5	Gestión de memoria en Bare-metal	32
4.1.6	Configuración del compilador	33
4.1.7	Flujo de compilación y programación	33
4.2	GPIO - Entradas/Salidas de Propósito General	34
4.2.1	Diseño del paquete	34
4.2.2	Acceso a registros hardware	35
4.2.3	Habilitación del reloj	35
4.2.4	Configuración de un pin	36
4.2.5	Control de salida: Set, Clear y Toggle	37
4.2.6	Control de entrada	37
4.2.7	Ejemplo de uso	37
4.3	ADC - Conversor Analógico-Digital	38
4.3.1	Diseño del paquete	38
4.3.2	Acceso a registros	39
4.3.3	Habilitación del reloj	39
4.3.4	Inicialización	40
4.3.5	Tiempo de muestreo	41
4.3.6	Lectura de disparo único	42
4.3.7	Lectura en modo continuo	42

4.3.8	Sensor de temperatura interno	43
4.3.9	Referencia de tensión interna	43
4.3.10	Ejemplo de uso	44
4.4	Timers	45
4.4.1	Funcionamiento del contador	45
4.4.2	Arquitectura del driver	45
4.5	Protocolos básicos de sistemas electrónicos	47
4.5.1	USART — Comunicación Serie Asíncrona	47
	Protocolo serie asíncrono	47
	Diseño genérico del paquete	48
	Registros mapeados en memoria	48
	Inicialización	49
	Transmisión	50
	Recepción	51
	Ejemplo de uso	51
4.5.2	I ² C — Bus de Dos Hilos	52
	Protocolo I ² C	52
	Diseño genérico y objeto protegido	53
	Registros hardware	53
	Inicialización	54
	Herramientas de diagnóstico	55
	Transmisión	55
	Recepción	55
	Herramientas de diagnóstico	55
	Espera con timeout	55
	Ejemplo de uso	56
4.5.3	SPI — Interfaz Serie Síncrona	57
	Protocolo SPI	57
	Diseño genérico del paquete	57
	Modos SPI y configuración CPOL/CPHA	58
	Función alternativa automática	58
	Inicialización	59
	Gestión dinámica de la velocidad	60
	Transferencia full-duplex	60
	Lectura y escritura simplificadas	61
	Verificación: test de loopback	61
4.6	Dispositivos externos	62
4.6.1	Pantalla OLED SSD1306	62
	Modelos soportados y constantes dependientes	63
	Organización de la memoria de vídeo	63

Protocolo de comunicación e inicialización	64
Operaciones de dibujo	64
4.6.2 Fuente de caracteres 5×7	66
4.6.3 Programa de pruebas	66
4.6.4 Módulo Ethernet W5500	66
Funcionamiento general y trama SPI	67
Sockets TCP y UDP	67
DNS	70
HTTP	71
Ping (ICMP)	71
mDNS	72
Integración y uso conjunto	72
4.7 Prueba del sistema completo	73
5 Conclusiones	74
5.1 Conclusiones del trabajo	74
5.2 Líneas de trabajo futuro	74
Bibliografía	75
Anexo I: Guía de instalación y configuración del entorno de desarrollo	A-1
Anexo II: Código fuente	B-1

Índice de figuras

1.1	Ejemplo de empleo de módulos genéricos en Ada.	15
2.1	Mapa de memoria simplificado del STM32F446. Elaboración propia a partir de [1].	21
3.1	Esquema fuente de alimentación	23
3.2	Esquema de acceso a red	24
3.3	Esquema de interfaz serie a USB	25
3.4	Esquema de interfaz microSD	25
3.5	Esquema del microcontrolador STM32F446	26
3.6	Esquema de pulsadores, LEDs, encoder y ADC	27
3.7	Esquema puerto USB Host	27
3.8	Expansor de E/S	28
3.9	Esquema de OLED, GPS y sensores externos	28
3.10	Modelo 3D PCB	29
3.11	Vista adicional	29
3.12	Diseño de capas PCB	29
4.1	Formato de una trama UART asíncrona de 8 bits (8N1).	48
4.2	Diagrama de temporización de una transacción I ² C: START, 7 bits de dirección, bit R/W, ACK del esclavo, byte de datos,ACK y STOP.	53
4.3	Transferencia SPI full-duplex	57
4.4	Organización de la memoria de vídeo del SSD1306 (128×64)	64
4.5	Rasterización de la línea (0,0) → (6,4) con Bresenham (DX = 6, DY = 4).	65
4.6	Estructura de la trama SPI del W5500. Elaboración propia a partir de [22].	67
4.7	Estructura de la cabecera TCP.	68
4.8	Estructura de la cabecera UDP.	69
4.9	Estructura del mensaje DHCP.	69
4.10	Estructura del mensaje DNS.	70
4.11	Transacción HTTP/1.1 básica implementada en el W5500.	71
4.12	Resultado prueba del sistema compleot.	73

Índice de tablas

2.1	Resumen de características del STM32F446 [1].	18
4.1	Comparativa de runtimes Ada para sistemas empujados.	31
4.2	Procedimientos públicos del driver de timers	47
4.3	Modos SPI definidos por CPOL y CPHA [1].	58
4.4	Trazas del algoritmo de Bresenham para la línea $(0, 0) \rightarrow (6, 4)$	65
4.5	Secuencia de pruebas del driver SSD1306.	66
4.6	Estados del socket TCP en el W5500.	68
4.7	Asignación de sockets del W5500 en el sistema.	72

1 Introducción

El avance de las tecnologías IoT requiere plataformas embebidas fiables, seguras y escalables. El microcontrolador STM32F446, basado en arquitectura ARM Cortex-M4, ofrece un balance entre capacidad de procesamiento, periféricos integrados y eficiencia energética, lo que lo convierte en una solución adecuada tanto para aplicaciones industriales como académicas.

Por otro lado, el lenguaje de programación Ada se distingue por su robustez, seguridad en tiempo de compilación y adecuación a sistemas críticos, cualidades especialmente relevantes en el ámbito IoT, donde la integridad de datos y la fiabilidad del sistema son esenciales.

1.1 Microcontrolador STM32F446

STMicroelectronics [2] desarrolla el microcontrolador *STM32F446*, perteneciente a la familia STM32 y basado en la arquitectura *ARM Cortex-M4*. La arquitectura ARM Cortex-M4 forma parte de la familia Cortex-M de procesadores de 32 bits diseñados por ARM Holdings. Se caracteriza por su pipeline eficiente, su conjunto de instrucciones Thumb-2, que combina instrucciones de 16 y 32 bits, mejorando el tamaño del código entre un 26% y un 30%, a la incorporación de una unidad de coma flotante (*Floating Point Unit*, FPU) integrada y de un conjunto de instrucciones especializadas para el procesamiento digital de señales (DSP), permitiendo alcanzar un elevado rendimiento con un consumo energético reducido.[1, 2, 7].

El STM32F446 puede alcanzar un rendimiento de 225 DMIPS a una frecuencia máxima de 180 MHz ejecutando código desde memoria Flash mediante el acelerador ART (*Adaptive Real-Time Accelerator*), permitiendo ejecución sin estados de espera [1, 2]. Asimismo, dispone de entre 256 KB y 512 KB de memoria Flash y hasta 128 KB de memoria SRAM, características que permiten implementar aplicaciones empotradas de complejidad elevada y sistemas IoT con múltiples tareas concurrentes [1].

El microcontrolador incorpora una amplia variedad de periféricos y buses de comunicación, destacando interfaces SPI, I²C y USART/UART, ampliamente utilizadas en sistemas de adquisición de datos y comunicación con sensores y dispositivos externos. Además, integra convertidores analógico-digitales (ADC), temporizadores avanzados, controladores DMA y soporte para interfaces USB OTG HS/FS [1].

Otro aspecto relevante es su eficiencia energética. Según la documentación oficial, el dispositivo presenta un consumo típico de aproximadamente 200 $\mu\text{A}/\text{MHz}$ en modo de ejecución desde memoria Flash a 180 MHz [2].

La elección del STM32F446 se basa en la elevada incorporación de periféricos, la amplia documentación proporcionada por STMicroelectronics y un extendido uso, tanto en entornos académicos como industriales, lo que lo convierte en una plataforma adecuada para el desarrollo de aplicaciones IoT y sistemas empujados didácticos.

1.2 El lenguaje Ada

El lenguaje de programación Ada fue diseñado con la intención de poder desarrollar sistemas fiables, seguros y mantenibles, especialmente en entornos en los que la robustez y la tolerancia a fallos constituyen una necesidad primaria. Su origen se localiza en el ámbito militar e industrial, donde la verificación del comportamiento y la detección temprana de errores son esenciales [3, 6].

Ada incorpora mecanismos avanzados de comprobación de errores en tiempo de compilación, un tipado fuerte y herramientas orientadas a la reducción de errores comunes presentes en otros lenguajes, como el desbordamiento de *buffers* o los errores de tipos. Estas características permiten que su seguridad se vea incrementada y que el código sea más mantenible, facilitando el desarrollo de aplicaciones con una vida útil extendida [3, 4].

Otro aspecto a destacar es su soporte nativo para concurrencia y sistemas de tiempo real mediante el uso de tareas, objetos protegidos y mecanismos de sincronización definidos por el propio estándar del lenguaje [3]. Esto dota al programador de utilidades especialmente adecuadas para aplicaciones IoT, donde es habitual la ejecución simultánea de tareas.

Adicionalmente, Ada dispone de un ecosistema de desarrollo moderno que permite su utilización sobre arquitecturas ARM Cortex-M mediante compiladores cruzados y herramientas como GNAT, Alire y GPRBuild [4, 5]. Esta posibilidad, junto a una sintaxis de estilo similar al lenguaje Pascal, facilita su adopción como lenguaje seguro para el desarrollo IoT y, en particular, para este proyecto.

1.3 Objetivos

El objetivo principal del presente trabajo es implementar una plataforma IoT didáctica basada en el microcontrolador STM32F446 mediante el lenguaje de programación Ada, de este modo, proporcionando un entorno de desarrollo para aplicaciones empujadas y comunicaciones en red.

Para alcanzar dicho objetivo, se plantean los siguientes objetivos, organizados en distintas áreas:

- **Hardware:** Se diseñará una plataforma basada en el microcontrolador STM32F446 orientada al desarrollo de aplicaciones IoT, integrando los módulos y periféricos de comunicación necesarios para la interacción entre sensores y redes de datos. Del mismo modo, se diseñará una PCB funcional para la ejecución del sistema desarrollado.
- **Software:** Se configurará el entorno de desarrollo para Ada sobre arquitectura ARM Cortex-M4, compilando y adaptando el runtime necesario para la ejecución bare-metal del sistema, junto con el desarrollo de una capa básica de inicialización y control de hardware.
- **Drivers y periféricos:** Se implementarán librerías para periféricos básicos como GPIO, temporizadores, ADC y USART, así como soporte para periféricos de mayor complejidad, incluyendo I2C y SPI.
- **IoT:** Se desarrollarán librerías y servicios de red, empleando el módulo Ethernet W5500, abarcando DHCP, TCP, UDP, ECHO, PING, DNS, mDNS y HTTP, integrando los protocolos más habituales.
- **Validación:** El sistema ha sido verificado mediante pruebas funcionales y aplicaciones demostrativas, evaluando la viabilidad del lenguaje como alternativa real para el desarrollo de estos sistemas.

1.4 Motivación

El punto de partida de este trabajo surge de la aparición de dos inquietudes relacionadas. Por un lado, el creciente protagonismo de los sistemas IoT en entornos industriales y académicos. Y, por otro, la escasa presencia del lenguaje Ada en dicho ámbito, pese a las garantías que ofrece en términos de fiabilidad y seguridad.

El mundo IoT se encuentra actualmente dominado por lenguajes como C y C++, cuya flexibilidad tiene como desventaja una mayor exposición a errores difíciles de detectar en tiempo de compilación, tales como desbordamientos de *buffer*, condiciones de carrera o accesos indebidos a memoria. En entornos donde la integridad del sistema es crítica, estas limitaciones representan un gran riesgo. Ada, con su tipado fuerte, sus comprobaciones estáticas y su soporte nativo para concurrencia, ofrece una alternativa sólida que, sin embargo, carece de una gran presencia en el sector.

Esta brecha constituyó una motivación directa para el desarrollo del presente proyecto. La existencia de una documentación actualizada y entornos de trabajo completos para Ada sobre microcontroladores ARM Cortex-M4 pone de manifiesto permite la contribución en esta dirección, tanto desde un punto de vista técnico como didáctico para demostrar la validez del lenguaje Ada para estos sistemas.

1.5 Competencias a desarrollar

La competencia principal es la capacidad para analizar, evaluar, seleccionar y configurar plataformas hardware para el desarrollo y ejecución de aplicaciones y servicios informáticos. Esta se trabaja a lo largo de todo el proyecto, ya que uno de sus ejes principales es la elección y puesta en marcha de una plataforma hardware adecuada para ejecutar aplicaciones desarrolladas en Ada en un entorno IoT. Para ello, se analizarán las características técnicas del microcontrolador STM32F446, estudiando su arquitectura ARM Cortex-M4, su capacidad de procesamiento, su memoria y sus periféricos, con el fin de determinar su idoneidad para los objetivos planteados. Esta plataforma se evaluará también frente a otras alternativas del mercado, teniendo en cuenta criterios como el rendimiento, el coste, la disponibilidad de herramientas y la compatibilidad con Ada. Finalmente, se llevará a cabo la configuración completa del entorno de trabajo, incluyendo la cadena de compilación cruzada, la carga de programas en el dispositivo y la integración de los entornos de trabajo necesarios para el desarrollo de aplicaciones reales.

1.6 Organización del documento

La memoria está organizada de la siguiente manera:

- **Capítulo 1:** Constituye la introducción del trabajo. En él se presentan el microcontrolador STM32F446 y el lenguaje Ada, se exponen los objetivos y la motivación del proyecto, se detallan las competencias a desarrollar, como ya han sido mostrados con anterioridad. Posteriormente, los recursos empleados y se describe la metodología de desarrollo seguida.
- **Capítulo 2:** Presenta en profundidad el microcontrolador STM32F446, describiendo su arquitectura ARM Cortex-M4, sus principales características, periféricos y organización de memoria, así como los criterios que justifican su selección como plataforma hardware para el desarrollo del proyecto.
- **Capítulo 3:** Aborda el diseño hardware de la placa desarrollada, detallando cada uno de los subsistemas integrados —alimentación, acceso a red, interfaz serie, almacenamiento, entrada/salida y dispositivos externos— así como las decisiones de diseño adoptadas durante la elaboración de la PCB.
- **Capítulo 4:** Constituye el núcleo técnico del trabajo y se centra en el diseño software. En él se describe el entorno de ejecución Ada empleado, la implementación de los controladores de periféricos y protocolos de comunicación —USART, I²C y SPI—, así como el desarrollo de los módulos destinados a la interacción con dispositivos externos y a las funcionalidades IoT.
- **Capítulo 5:** Recoge las conclusiones derivadas de la realización del proyecto, evaluando

el grado de consecución de los objetivos planteados e identificando posibles líneas de trabajo futuro que permitan ampliar las capacidades de la plataforma desarrollada.

- Por último, se incluyen dos **anexos** que estarán disponibles como documentos adicionales: el **Anexo I**, que contiene el código fuente desarrollado, y el **Anexo II**, que detalla el proceso de configuración del entorno de desarrollo.

1.7 Recursos empleados

1.7.1 Hardware

El desarrollo hardware del proyecto se va a llevar a cabo mediante **KiCad**, herramienta de código abierto utilizada tanto para el diseño esquemático como para el trazado de la placa de circuitos impresos. Para el desarrollo del sistema se empleará la placa de desarrollo **STM32 Nucleo-F446RE**, basada en el microcontrolador **STM32F446RE**, con el objetivo de facilitar la validación del sistema y el desarrollo de librerías genéricas reutilizables posteriormente con la placa anteriormente diseñada.

La plataforma hardware inicial se compondría de la placa de desarrollo **STM32 Nucleo-F446RE** junto a distintos módulos orientados a aplicaciones IoT, entre los que destacan una pantalla OLED basada en el controlador **SSD1306**, un controlador Ethernet **W5500** y módulos compatibles con el estándar **Arduino Shield V2** para almacenamiento mediante tarjeta SD y comunicaciones Ethernet. Para la carga y depuración del firmware sobre la arquitectura **ARM Cortex-M4**, se empleará un programador y depurador **ST-LINK**.

1.7.2 Software y toolchain

El desarrollo software se va a realizar en el lenguaje de programación **Ada** [13], compilado mediante compilación cruzada para la arquitectura **ARM Cortex-M4**.

Durante las primeras fases del proyecto se utilizó **GPRbuild** como sistema principal. No obstante, debido a limitaciones en la gestión de dependencias y problemas de compatibilidad con las herramientas, el entorno fue migrado a **Alire** [14], gestor de paquetes y entornos de desarrollo para Ada, que permitió una gestión más eficiente de las dependencias del proyecto.

Adicionalmente, se empleó **GitHub** como plataforma de control de versiones y alojamiento del repositorio del proyecto, facilitando el trabajo con distintas versiones del software y el seguimiento de los cambios realizados durante el desarrollo.

El proceso de compilación y programación fue automatizado mediante un script de shell que encadena las distintas etapas: Limpieza y compilación del proyecto con Alire; generación

del binario plano mediante `arm-none-eabi-objcopy` y programación del microcontrolador a través de OpenOCD, incluyendo borrado de la memoria flash, escritura en la dirección base `0x08000000`, verificación de la imagen y reinicio del sistema.

Justificación de la elección de Ada frente a C/C++

La elección del lenguaje de programación Ada para el desarrollo del firmware responde a criterios técnicos y de ingeniería de software fundamentados.

Los lenguajes C y C++ constituyen la opción predominante en el desarrollo de sistemas empotrados debido a su rendimiento, su amplia disponibilidad de herramientas y la extensión de su ecosistema [20]. Sin embargo, ambos presentan debilidades estructurales relevantes en términos de seguridad y fiabilidad: la ausencia de comprobación estricta de tipos en tiempo de compilación; la gestión manual de memoria propensa a errores y la facilidad con la que se pueden introducir comportamientos indefinidos hacen que su uso en sistemas de alta fiabilidad requiera una atención adicional y herramientas de análisis externas.

Ada, por el contrario, fue diseñado desde sus orígenes con la fiabilidad y la verificabilidad como principios fundamentales [13]. Entre sus características más relevantes para este proyecto destacan:

- **Tipado fuerte y estricto:** Ada impide implícitamente las conversiones de tipo no seguras, eliminando en tiempo de compilación una familia entera de errores comunes en C [13].
- **Comprobación de rangos:** Es posible definir subtipos acotados cuyo rango se verifica automáticamente, evitando desbordamientos [13].
- **Concurrencia integrada en el lenguaje:** Ada incorpora el modelo de tareas (*tasking*) como parte del propio lenguaje. [13].
- **Legibilidad y mantenibilidad:** la sintaxis explícita de Ada favorece la escritura de código más legible, reduciendo la documentación necesaria para el código.
- **Soporte para sistemas empotrados de bajo nivel:** El subconjunto **SPARK Ada** [19] y el perfil **Ravenscar** [18] permiten restringir el lenguaje a un subconjunto adecuado para sistemas de tiempo real y recursos limitados.

En este proyecto, la elección de Ada permitió detectar en tiempo de compilación errores que en C habrían pasado desapercibidos hasta la fase de ejecución, reduciendo el tiempo de depuración y aumentando la confianza en la corrección del sistema. Adicionalmente, su sintaxis familiar y el hecho de ser un lenguaje concurrente facilitan la implementación del sistema sin depender de librerías auxiliares.

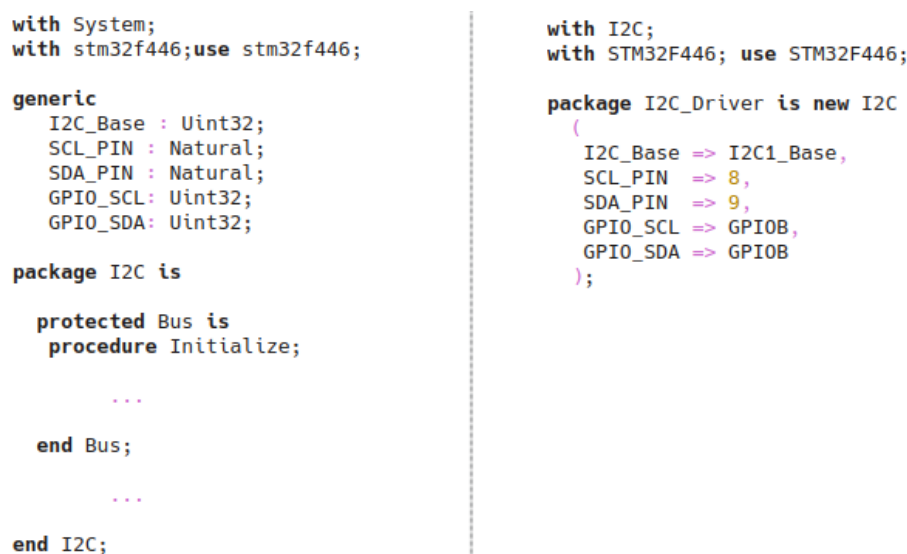
1.8 Metodología de desarrollo

La metodología aplicada combina el desarrollo incremental *bottom-up* [24], partiendo de las unidades de menor abstracción hacia las de mayor complejidad, con principios de programación defensiva mediante la definición de interfaces estables entre módulos [24]. El proceso de validación siguió un enfoque de prueba de regresión continua, acumulando casos de prueba a lo largo del desarrollo sin suprimir los anteriores [24].

1.8.1 Desarrollo incremental del software

La implementación del software se abordó partiendo de las unidades funcionales más simples, verificando su correcto funcionamiento antes de proceder a la construcción de las capas superiores que las empleaban. De este modo, cada subprograma quedaba validado antes de ser integrado como componente de una función de mayor complejidad, facilitando las modificaciones y la depuración de errores.

Una vez que el conjunto de funciones resultaba funcional, se procedió a una fase de optimización para mejorar la eficiencia y la limpieza del código. Finalmente, las unidades que lo requerían fueron generalizadas mediante el mecanismo de genericidad propio del lenguaje Ada, dotándolas de mayor reutilizabilidad y abstracción. En la figura 1.1 se muestra un ejemplo del sistema de genericidad del lenguaje Ada y su uso: la parte izquierda de la imagen corresponde a la declaración y la parte derecha a su instanciación; finalmente, solo sería necesario importar el paquete declarado.



```
with System;
with stm32f446; use stm32f446;

generic
  I2C_Base : Uint32;
  SCL_PIN  : Natural;
  SDA_PIN  : Natural;
  GPIO_SCL : Uint32;
  GPIO_SDA : Uint32;

package I2C is

  protected Bus is
    procedure Initialize;

    ...

  end Bus;

  ...

end I2C;
```

```
with I2C;
with STM32F446; use STM32F446;

package I2C_Driver is new I2C
(
  I2C_Base => I2C1_Base,
  SCL_PIN  => 8,
  SDA_PIN  => 9,
  GPIO_SCL => GPIOB,
  GPIO_SDA => GPIOB
);
```

Figura 1.1: Ejemplo de empleo de módulos genéricos en Ada.

1.8.2 Validación por capas

El proceso de prueba siguió el mismo principio incremental que el desarrollo. El programa de pruebas de cada librería se construyó de forma acumulativa: al incorporar una nueva funcionalidad, se añadían los casos de prueba correspondientes sin eliminar los anteriores. Esto garantizaba que, a medida que el sistema crecía en complejidad, las unidades básicas continuaban siendo verificadas en cada ejecución, asegurando que las modificaciones introducidas en las capas superiores no comprometían el comportamiento ya validado en las inferiores, facilitando la corrección de errores. Este enfoque constituye una forma de prueba de regresión continua a lo largo de todo el ciclo de desarrollo.

1.8.3 Configuración del entorno hardware

La integración con el hardware se abordó de manera diferente. En este ámbito, se realizó un análisis previo de los requisitos y las restricciones del sistema antes de iniciar cualquier configuración, adoptando una que minimizara la necesidad de cambios posteriores. Los cambios que se produjeron en el entorno de desarrollo a lo largo del proyecto respondieron principalmente a dos causas: La resolución de problemas de compatibilidad detectados durante la integración y la incorporación de nuevas herramientas o capacidades necesarias para ampliar las funcionalidades del sistema.

1.8.4 Modularidad y principio de abstracción

Tanto la estructura del software como la organización del proceso de desarrollo estuvieron guiadas por el principio de modularidad y el principio de abstracción. La separación clara entre unidades independientes, siendo coherente con el modelo de paquetes de Ada, permitió desarrollar, probar y optimizar cada componente de forma independiente.

Especialmente relevante fue la aplicación del principio de abstracción: las interfaces entre unidades se diseñaron de forma que los detalles internos de implementación de una función quedaran fuera de las vistas del resto del sistema. Gracias a ello, las modificaciones realizadas sobre una unidad, ya fuera por la corrección de algún error o la optimización, no propagaban efectos no deseados sobre las unidades que la utilizaban. Este enfoque facilitó tanto el mantenimiento del código como la localización de errores a lo largo de todo el proyecto.

2 Microcontrolador STM32F446

2.1 Arquitectura ARM Cortex-M4

ARM Holdings es una empresa de diseño de semiconductores que desarrolla arquitecturas de procesadores ampliamente adoptadas en la industria de los sistemas empujados [9]. A diferencia de otros fabricantes, ARM no produce chips físicos, sino que licencia sus diseños a terceros, entre ellos STMicroelectronics.

El núcleo **ARM Cortex-M4** pertenece a la familia Cortex-M, orientada exclusivamente a sistemas empujados y sistemas en tiempo real. Se trata de un procesador RISC (*Reduced Instruction Set Computer*) de 32 bits que implementa el conjunto de instrucciones ARMv7E-M [10]. Presenta una anchura de bus de 32 bits que permite manejar datos de mayor precisión y direccionar hasta 4 GiB de espacio de memoria de forma uniforme.

Dos características distinguen al Cortex-M4 dentro de la familia Cortex-M [11]:

- **Unidad de punto flotante (FPU):** Implementación hardware de operaciones en coma flotante de precisión simple (IEEE 754) [12], lo que evita la emulación software, ya que es muy costosa y resulta indispensable en aplicaciones de control con cálculos matemáticos frecuentes.
- **Extensiones DSP:** Instrucciones tipo SIMD (*Single Instruction, Multiple Data*) y operaciones de multiplicación con acumulación en un solo ciclo [10], habituales en procesamiento de señal y algoritmos de filtrado.

El Cortex-M4 incorpora además el **NVIC** (*Nested Vectored Interrupt Controller*), un controlador de interrupciones anidadas que garantiza tiempos de respuesta determinados [11], requisito fundamental en sistemas de tiempo real

2.2 El microcontrolador STM32F446

El **STM32F446** es un microcontrolador de 32 bits fabricado por **STMicroelectronics**, empresa multinacional con sede en Ginebra y uno de los principales proveedores mundiales de semiconductores para aplicaciones industriales y de consumo [2]. Pertenece a la familia **STM32F4**, un grupo de dispositivos de alto rendimiento basados en el núcleo ARM Cortex-M4 [1]. La familia se caracteriza por ofrecer una combinación de frecuencia de operación elevada, conjunto de periféricos extenso y consumo energético ligero, lo que la hace especialmente adecuada para aplicaciones IoT (*Internet of Things*), automatización industrial y sistemas empujados.

2.2.1 Características principales

Tabla 2.1: Resumen de características del STM32F446 [1].

Parámetro	Valor
Núcleo	ARM Cortex-M4 con FPU
Frecuencia máxima	180 MHz
Memoria Flash	512 KiB
SRAM	128 KiB (SRAM1) + 16 KiB (SRAM2)
Tensión de alimentación	1,7 V – 3,6 V
Rango de temperatura	–40 °C a +85 °C (Industrial)

Frecuencia de operación. El STM32F446 puede operar a una frecuencia de hasta **180 MHz**, lo que lo sitúa en la parte superior de la familia STM32F4 [1]. Esta frecuencia se obtiene mediante el PLL (*Phase-Locked Loop*) interno a partir de una fuente de reloj externa o interna configurable [1].

Organización de memoria. El dispositivo dispone de **512 KiB de memoria Flash** para el almacenamiento del firmware y **128 KiB de SRAM principal** para datos en tiempo de ejecución, además de 16 KiB de SRAM adicional [1]. Esta distribución es suficiente para aplicaciones de complejidad media con pilas de protocolos y sistemas operativos de tiempo real ligeros.

2.2.2 ART Accelerator

Un cuello de botella habitual en los microcontroladores Flash es el tiempo de acceso a la memoria de programa: la Flash interna del STM32F446 opera a un máximo de **30 MHz**, mientras que el núcleo puede alcanzar los 180 MHz [1]. Para resolver esta diferencia, STMicroelectronics incorpora el **ART Accelerator** (*Adaptive Real-Time Memory Accelerator*), compuesto por [1]:

- Una *caché de instrucciones* de 64 líneas de 128 bits.
- Una *caché de datos* de 8 líneas de 128 bits.
- Un *prefetcher* de instrucciones que anticipa la línea siguiente durante la ejecución de la actual.

Gracias al ART Accelerator, el rendimiento del sistema se aproxima al de ejecución desde SRAM, permitiendo alcanzar las **225 DMIPS** especificadas en la hoja de características [1]. En la práctica, esto significa que el código puede ejecutarse a plena velocidad (0 *wait states* efectivos) sin necesidad de copiar el firmware a SRAM, simplificando el proceso de enlazado y reduciendo el consumo.

2.2.3 NVIC y gestión de interrupciones

El **NVIC** del STM32F446 soporta hasta **90 líneas de interrupción** con 16 niveles de prioridad configurables. Esta arquitectura permite anidar interrupciones de mayor prioridad sobre otras, garantizando la respuesta a eventos críticos en tiempos determinados, requisito esencial en cualquier sistema de tiempo real.

2.2.4 Eficiencia energética

El STM32F446 implementa varios modos de bajo consumo (*Sleep*, *Stop* y *Standby*) que permiten reducir drásticamente la corriente consumida cuando no es necesario la actividad completa. En modo *Stop*, el consumo puede descender por debajo de 100 μ A manteniendo el contenido de SRAM, lo que resulta relevante en nodos IoT alimentados por batería.

2.3 Periféricos de mayor interés

El STM32F446 integra un extenso conjunto de periféricos [1]. En esta sección se describen los principales y de mayor interés para el ámbito IoT, evitando reproducir la enumeración exhaustiva del *datasheet*.

GPIO — Entradas/Salidas de Propósito General: Los pines **GPIO** (*General Purpose Input/Output*) del STM32F446 están organizados en puertos de hasta 16 líneas cada uno (GPIOA–GPIOH) [1]. Cada pin puede configurarse de forma independiente como entrada digital, salida digital (modo *push-pull* o *open drain*), función alternativa (AF) asociada a un periférico o entrada analógica.

UART/USART — Comunicación Serie Asíncrona: El STM32F446 incorpora cuatro **USART** (*Universal Synchronous Asynchronous Receiver Transmitter*) y dos **UART**, con velocidades de hasta 10,5 Mbit/s en modo síncrono [1]. En este proyecto, la USART2 se utiliza para la comunicación con el terminal de depuración a través de la interfaz ST-LINK de la placa Nucleo-F446RE, lo que permite realizar diagnóstico sin hardware adicional.

SPI — Interfaz Serie Síncrona: El protocolo **SPI** (*Serial Peripheral Interface*) es una interfaz síncrona full-duplex ampliamente utilizada para comunicar microcontroladores con periféricos de alta velocidad. El STM32F446 dispone de hasta cuatro controladores SPI capaces de operar a frecuencias de reloj de hasta **45 MHz** [1].

I²C — Bus de Dos Hilos: El **I²C** (*Inter-Integrated Circuit*) es un bus serie síncrono de dos hilos (SCL y SDA) que permite conectar múltiples dispositivos esclavos a un mismo maestro. El STM32F446 dispone de hasta tres controladores I²C compatibles con los modos estándar (100 kbit/s) y rápido (400 kbit/s) [1].

Timers — Temporizadores: El STM32F446 incluye temporizadores de propósito general, avanzados y básicos [1]. Los de propósito general (TIM2–TIM5) son de 32 bits y pueden funcionar hasta a 90 MHz, mientras que los avanzados (TIM1, TIM8) añaden capacidades de control PWM complementario para aplicaciones de accionamiento de motor.

DMA — Acceso Directo a Memoria: El **DMA** (*Direct Memory Access*) permite la transferencia autónoma de datos entre periféricos y memoria sin intervención del núcleo, liberándolo para la ejecución de tareas de mayor nivel. El STM32F446 dispone de dos controladores DMA con 8 canales cada uno [1].

2.4 Memoria y arquitectura interna

El STM32F446 adopta la arquitectura Harvard modificada del núcleo Cortex-M4, que dispone de buses de instrucciones y datos separados para mejorar el rendimiento, aunque comparten un espacio de direccionamiento lineal de 32 bits (4 GiB) definido por las especificaciones del conjunto ARMv7-M [10, 11].

2.4.1 Tipos de memoria

Memoria Flash: Los 512 KiB de memoria Flash no volátil almacenan el código del programa y los datos de configuración persistentes [1]. La Flash se organiza en sectores de tamaño variable (16 KiB, 64 KiB y 128 KiB), lo que permite actualizar regiones del firmware de forma independiente. La escritura requiere un ciclo de borrado previo por sector [1].

SRAM: La SRAM principal (128 KiB) se mapea en la región 0x20000000 del espacio de direcciones y es accesible desde el núcleo a velocidad plena. La SRAM2 adicional (16 KiB) se ubica inmediatamente a continuación. Ambas retienen su contenido en los modos de bajo consumo *Sleep* y *Stop* [1].

2.4.2 Mapa de memoria simplificado

La Figura 2.1 muestra de forma esquemática las regiones principales del espacio de direccionamiento del STM32F446 [1]. La zona de Flash se sitúa en la parte baja del mapa (a partir de 0x08000000), la SRAM en la región de datos (0x20000000) y los registros de periférico en el área AHB/APB (0x40000000–0x60000000).

Gracias al ART Accelerator (Descrito en la Sección 2.2.2), la ejecución del código directamente desde Flash, a 180 MHz, no supone una penalización de rendimiento apreciable. No es necesario, por tanto, copiar el código a SRAM (*Load-to-RAM*), lo que simplifica el *linker script* y preserva la SRAM para datos de aplicación.

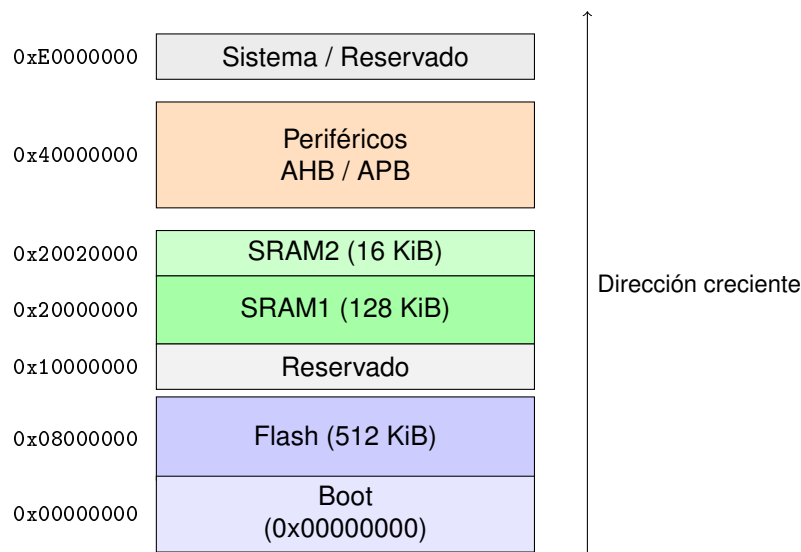


Figura 2.1: Mapa de memoria simplificado del STM32F446. Elaboración propia a partir de [1].

2.5 Justificación de la elección

La selección del STM32F446 para este proyecto no responde únicamente a sus características técnicas, sino a un conjunto de factores que lo hacen especialmente adecuado.

2.5.1 Coste y disponibilidad

La placa de desarrollo **Nucleo-F446RE** tiene un precio inferior a 15 euros en los principales distribuidores [2], lo que la sitúa entre las plataformas de desarrollo más accesibles con un microcontrolador de 180 MHz. Esta disponibilidad facilita la replicabilidad del sistema y reduce el riesgo de desabastecimiento durante el desarrollo del proyecto.

2.5.2 Ecosistema STM32

La familia STM32 cuenta con uno de los ecosistemas de desarrollo más consolidados del mercado de microcontroladores ARM [2], lo que se traduce en una amplia documentación oficial —*datasheet* [1], manual de referencia y notas de aplicación detalladas—, una comunidad activa con abundantes ejemplos, foros y recursos de aprendizaje, así como compatibilidad de pines y periféricos dentro de la familia, lo que facilita la migración a dispositivos con mayor capacidad si el proyecto lo requiriese.

2.5.3 Adecuación a aplicaciones IoT

El STM32F446 ofrece un equilibrio entre rendimiento y consumo para un dispositivo IoT que debe procesar datos de sensores, gestionar comunicaciones de red y ejecutar tareas periódicas [1]. Los modos de bajo consumo permiten prolongar la vida útil de la batería.

2.5.4 Depuración integrada con ST-LINK

La placa Nucleo-F446RE incorpora el programador y depurador **ST-LINK/V2-1** en la propia placa, sin necesidad de hardware externo [2]. Esto permite la programación del microcontrolador y la depuración en tiempo real a través de un único cable USB, agilizando el desarrollo. La integración con **OpenOCD** permite además el uso de herramientas de depuración de código libre [16].

2.5.5 Soporte del *runtime* Ada

Un requisito particular de este proyecto es la utilización del lenguaje **Ada** para el desarrollo del firmware [3]. El compilador **GNAT** de AdaCore dispone de soporte estable para la arquitectura ARM Cortex-M4 [15] y existe un *runtime* Ada específico para la familia STM32F4 (`bb-runtimes` [25]), lo que garantiza la compatibilidad entre el entorno de programación elegido y el hardware seleccionado.

3.2 Acceso a Red vía SPI

Se han integrado dos interfaces de red sobre bus SPI. Para la conectividad inalámbrica se emplea un microcontrolador ESP8266 con firmware estándar AT, que gestiona conexiones TCP y UDP sobre Wi-Fi. Para la conectividad cableada se incorpora el adaptador Ethernet WizNet W5500, basado en un procesador de 16 bits que opera igualmente sobre SPI y soporta múltiples conexiones simultáneas. Ambos dispositivos son controlados mediante el driver SPI desarrollado para el microcontrolador principal, requiriendo además un driver específico para cada uno. El esquema de este bloque se muestra en la figura 3.2.

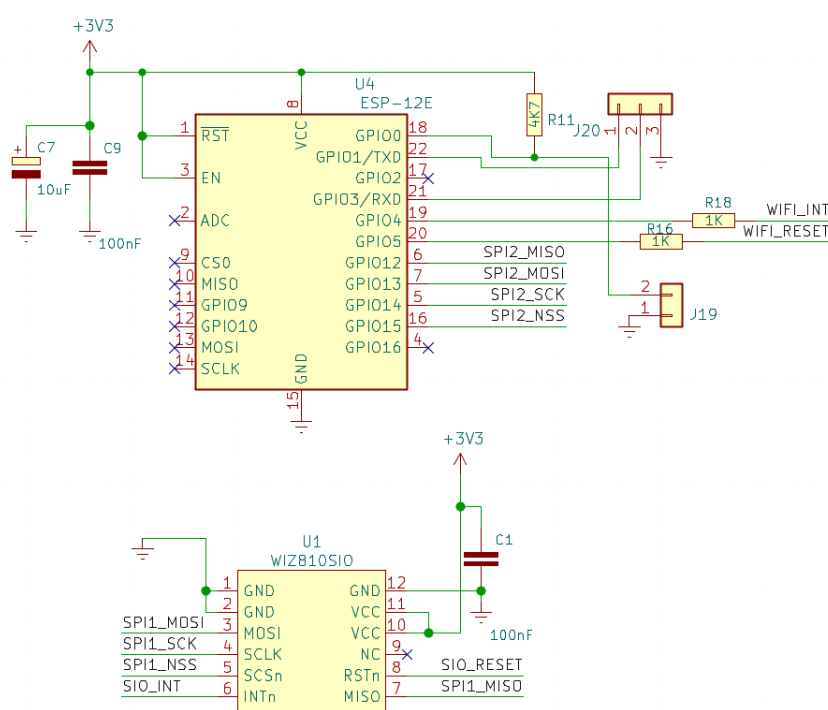


Figura 3.2: Esquema de acceso a red

3.3 Interfaz Serie a USB

Para las tareas de depuración se ha implementado una interfaz serie-USB mediante el conversor CH340G, conectado a uno de los puertos UART del STM32F446. La placa dispone de un conector USB tipo B que puede utilizarse igualmente como entrada de alimentación. El conversor expone también señales de control adicionales que permitirían el uso de un *bootloader*, aunque esta funcionalidad no ha sido implementada en el presente proyecto. En la figura 3.3 se muestra el esquema correspondiente.

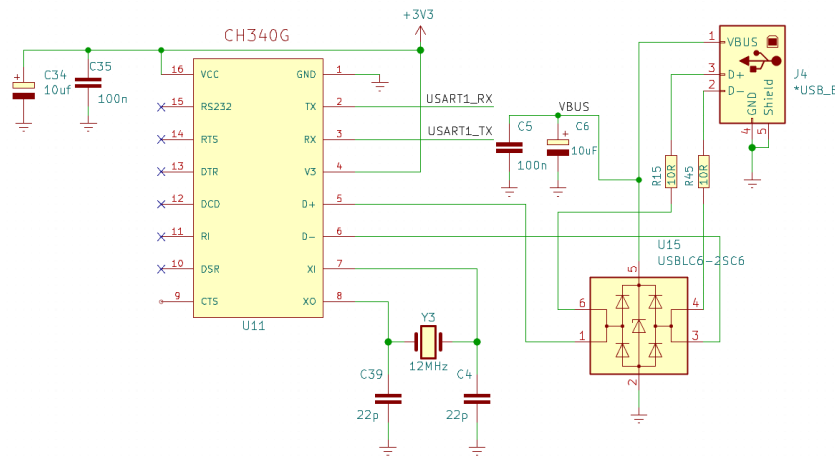


Figura 3.3: Esquema de interfaz serie a USB

3.4 Tarjeta microSD

La tarjeta microSD se conecta directamente al microcontrolador mediante el bus de datos completo, lo que permite su operación en modo quad-SPI, favoreciendo un mayor ancho de banda y compatibilidad con tarjetas de mayor capacidad. Aunque el driver FAT no ha sido implementado dada la complejidad que conlleva la gestión del sistema de ficheros, la tarjeta es accesible como un dispositivo SPI convencional. El esquema de este bloque puede consultarse en la figura 3.4.

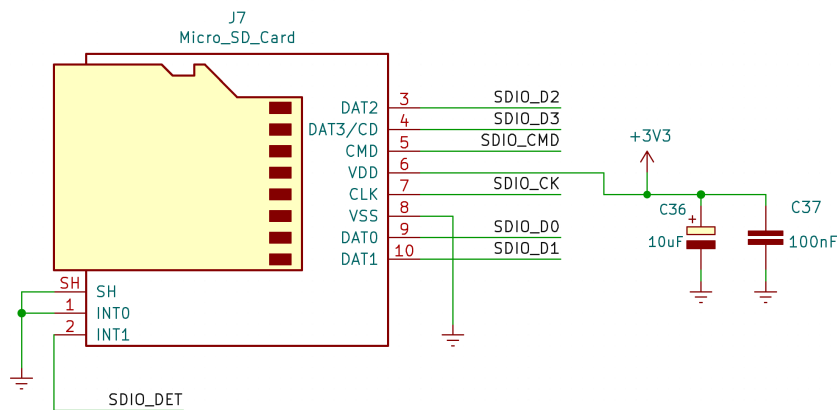


Figura 3.4: Esquema de interfaz microSD

3.5 Microcontrolador STM32F446

El microcontrolador STM32F446 en encapsulado LQFP64 constituye el núcleo de procesamiento de la placa. Como soporte se han incorporado dos osciladores de cristal: uno de 8 MHz, multiplicado internamente para obtener la frecuencia de operación principal, y uno de 32 kHz destinado al RTC interno. Para el mantenimiento del RTC sin alimentación principal se ha previsto un zócalo para pila de botón CR1220. El diseño incluye también los condensadores de desacoplo necesarios, el pulsador de reset y un par de *jumpers* para la selección del modo de arranque. Finalmente, se ha dispuesto un conector de programación con las señales SWDIO y SWCLK para la depuración y actualización del firmware. Este subsistema se puede ver en la figura siguiente 3.5 .

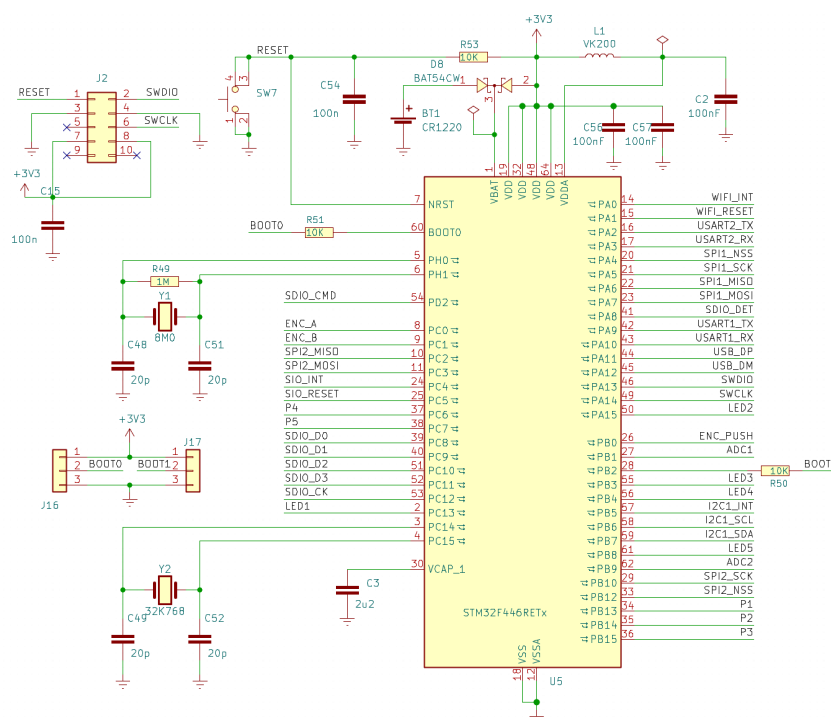


Figura 3.5: Esquema del microcontrolador STM32F446

3.6 Pulsadores, LEDs, Encoder y ADC

Con el fin de facilitar las primeras fases del aprendizaje, se han dispuesto cinco pulsadores con resistencias en serie, concebidos para ser utilizados junto con las resistencias de *pull-up* internas del microcontrolador. Para la visualización del estado lógico del sistema se incluyen cinco LEDs programables, así como LEDs indicadores de actividad en los puertos serie 1 (conectado al conversor USB) y serie 2 (reservado para un módulo GPS opcional). El diseño incorpora también un encoder incremental, útil para el estudio de temporizaciones a bajo nivel,

y un potenciómetro conectado a un canal ADC interno del microcontrolador, que permite emular el comportamiento de sensores analógicos. El esquema correspondiente se muestra en la figura 3.6.

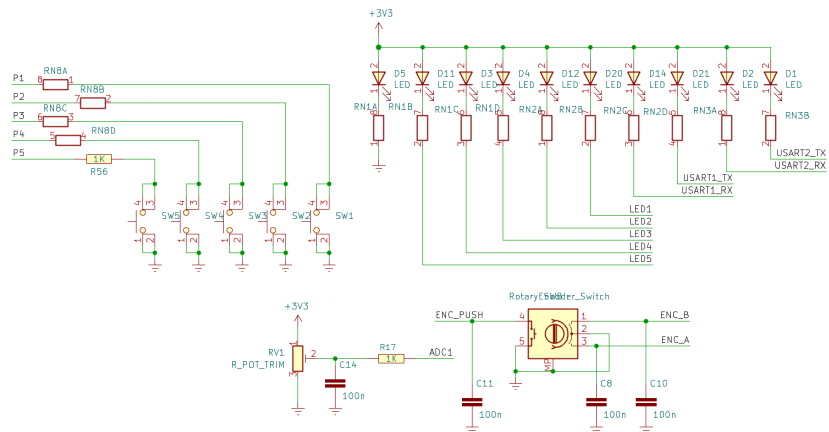


Figura 3.6: Esquema de pulsadores, LEDs, encoder y ADC

3.7 Puerto USB Host

Para la validación de la funcionalidad USB Host del microcontrolador se ha incorporado un conector USB tipo A. Aunque el driver y la pila de protocolo USB no han sido implementados en este trabajo dada su complejidad, esta funcionalidad queda disponible para su desarrollo en trabajos futuros. El esquema de este bloque está en la figura 3.7.

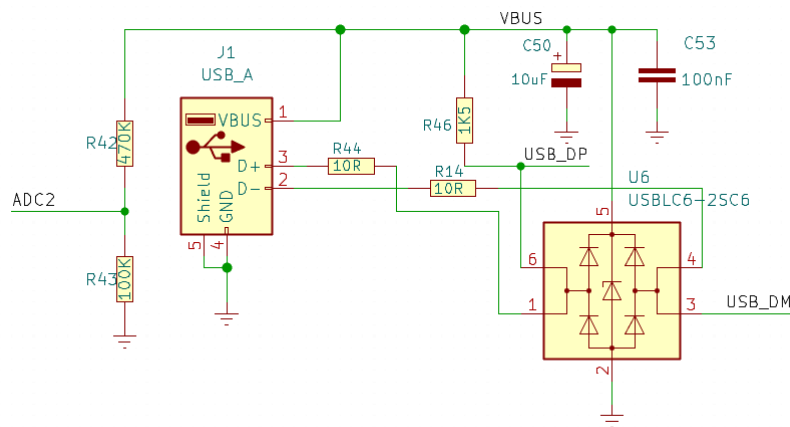


Figura 3.7: Esquema puerto USB Host

3.8 Expansor de E/S vía I2C

El expansor de E/S se basa en el integrado PCF8574, controlado mediante bus I2C y alimentado a 5 V. Dado el diferente nivel lógico entre el microcontrolador (3,3 V) y el expansor (5 V), se ha incorporado un circuito adaptador de niveles basado en transistores MOS. Las líneas de entrada/salida del expansor se encuentran conectadas a LEDs que reflejan visualmente el estado lógico de cada pin. El esquema correspondiente se muestra en la figura 3.8.

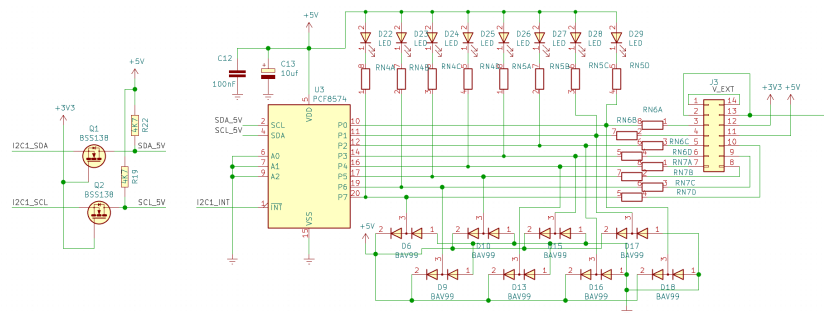


Figura 3.8: Expansor de E/S

3.9 Pantalla OLED, GPS y Sensores Externos

Este bloque está compuesto por conectores destinados a periféricos externos. La pantalla OLED de 128×64 píxeles se conecta mediante el bus I2C con alimentación a 3,3 V. Los sensores externos emplean igualmente este esquema de conexión. El módulo GPS se conecta a través del puerto serie 2 (señales RX/TX) con alimentación a 3,3 V. El esquema se muestra en la siguiente figura 3.9:

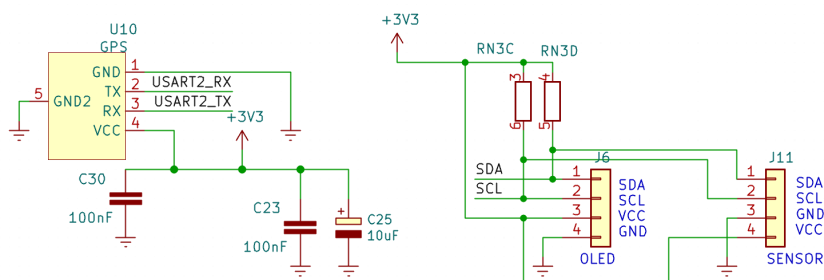


Figura 3.9: Esquema de OLED, GPS y sensores externos

3.10 Diseño de la PCB

El diseño de la PCB se ha realizado con la herramienta KiCad en su última versión, priorizando la compactación del diseño con el fin de reducir los costes de fabricación. Se ha hecho uso extensivo de componentes SMD de bajo coste y de redes de resistencias para optimizar el espacio en la placa. La fabricación está orientada a servicios como JLCPCB o PCBWay, que ofrecen costes reducidos con calidad aceptable para un prototipo de estas características. Del mismo modo, se ha optado por el uso de módulos enchufables para simplificar el montaje y dotar al diseño de mayor flexibilidad en función de los periféricos requeridos en cada proyecto. A continuación, se muestran los modelos 3D de la placa en las figuras 3.10 y 3.11, así como el diseño de capas en la figura 3.12.

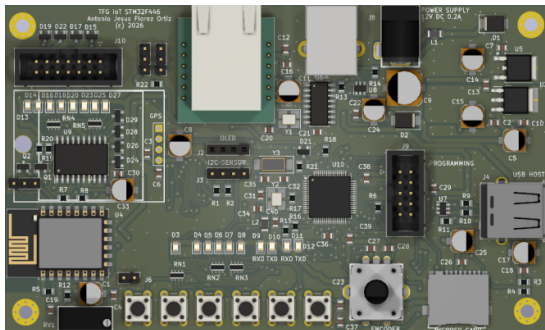


Figura 3.10: Modelo 3D PCB

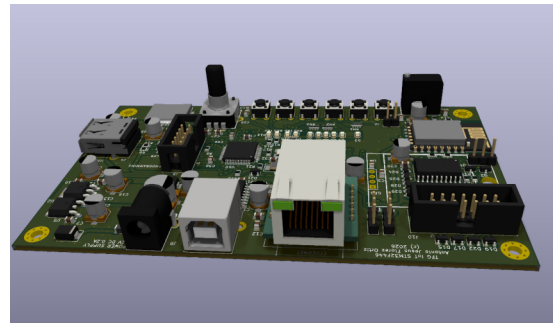


Figura 3.11: Vista adicional

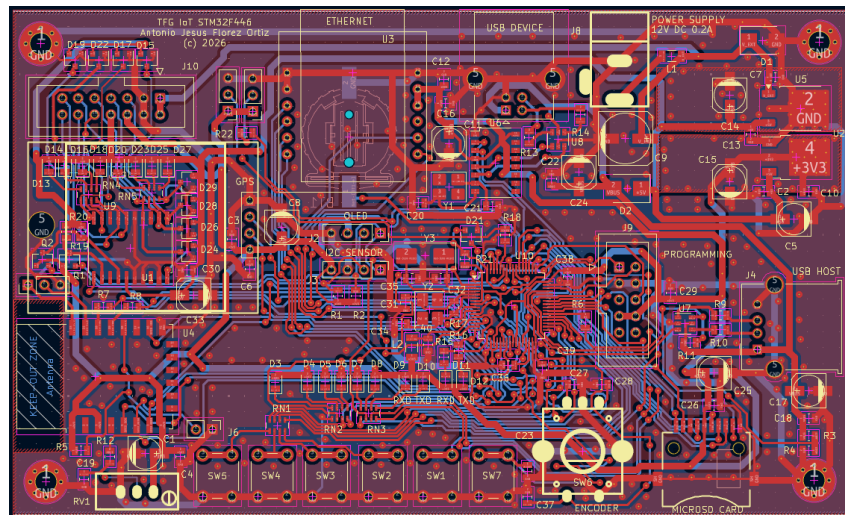


Figura 3.12: Diseño de capas PCB

4 Diseño software

Este capítulo describe las decisiones de implementación adoptadas en el desarrollo del firmware para los sistemas IoT, desde la configuración del entorno de ejecución Ada hasta el uso de cada periférico del STM32F446. A diferencia de los enfoques habituales basados en C o C++, el uso de Ada en un sistema bare-metal impone restricciones y ofrece garantías que condicionan toda la arquitectura software.

4.1 Runtime Ada

En un sistema con sistema operativo, el *runtime* de un lenguaje de programación puede apoyarse en servicios del núcleo del SO: gestión de memoria dinámica, manejo de señales, sincronización de hilos o entrada/salida. En un sistema *bare-metal* como el presente, ninguno de esos servicios existe. El *runtime* Ada debe ser autosuficiente y caber íntegramente en los recursos del microcontrolador [25].

4.1.1 El perfil embedded-stm32f4

El proyecto utiliza el runtime `embedded-stm32f4`, declarado en el fichero de proyecto GNAT (`.gpr`) de la siguiente forma [15, 25]:

```
for Target use "arm-eabi";
for Runtime ("Ada") use "embedded-stm32f4";
```

Listing 4.1: Selección de target y runtime en `nucleo_test.gpr`.

Este perfil pertenece a la familia de runtimes **Ravenscar**, concretamente a la variante *Small FootPrint* (`ravenscar-sfp`), adaptada por AdaCore para la familia STM32F4 [25]. Ravenscar es un subconjunto del modelo de concurrencia de Ada definido originalmente para sistemas de tiempo real de alta integridad [18], que elimina las construcciones más costosas del lenguaje manteniendo las garantías de tiempo real.

4.1.2 Comparativa de runtimes Ada

AdaCore proporciona varios perfiles de runtime para sistemas empujados, con distintos niveles de funcionalidad y coste en recursos. La Tabla 4.1 resume qué puede y qué no puede hacerse en función del runtime elegido [15, 25].

El runtime `embedded-stm32f4` empleado en este proyecto equivale funcionalmente a **Ravenscar-SFP**. Permite el uso de tareas, objetos protegidos y el paquete `Ada.Real_Time` para temporización precisa, sin incluir memoria dinámica ni las construcciones de concurrencia más complejas del lenguaje completo. Esta restricción no supone una limitación práctica para el sistema

desarrollado, puesto que las estructuras complejas son replicables mediante las simples y dada la naturaleza del proyecto otras funciones no serán usadas.

Tabla 4.1: Comparativa de runtimes Ada para sistemas empujados.

Característica	ZFP	Ravenscar-SFP	Ravenscar-Full
Subprogramas y paquetes básicos	✓	✓	✓
Tipos, records, arrays	✓	✓	✓
Excepciones	✓	✓	✓
Punto flotante (FPU)	✓	✓	✓
Ada 2022	✓	✓	✓
Tareas (task)	✗	✓	✓
Objetos protegidos (protected)	✗	✓	✓
Tiempo real (Ada.Real_Time)	✗	✓	✓
Memoria dinámica (new)	✗	✗	✓
Ada.Text_IO completo	✗	✗	✓
Entradas con guardas (when)	✗	✗	✓
Tipos de tarea anónimos	✗	✗	✓
Rendezvous (cita Ada)	✗	✗	✓
Coste Flash aproximado	<10 KiB	~30 KiB	>100 KiB
Coste RAM aproximado	<1 KiB	~4 KiB	>20 KiB
✓ disponible ✗ no disponible			

4.1.3 Proceso de arranque (*startup*)

Antes de que se ejecute la primera línea del programa Ada, el microcontrolador realiza una secuencia de inicialización gestionada por el runtime [25]:

1. **Vector de reset:** Al alimentar el STM32F446, el núcleo Cortex-M4 lee la dirección del manejador de reset desde la tabla de vectores situada en 0x08000000 y salta a él.
2. **Inicialización de la pila:** Se carga el puntero de pila (SP) con el valor del símbolo `_stack_end` (Dirección más alta de la pila), definido en el *linker script* del runtime.
3. **Copia de .data:** Los datos globales con valor inicial residen en Flash y el arranque los copia a su dirección de ejecución en SRAM.
4. **Puesta a cero de .bss:** Las variables globales sin valor inicial explícito se inicializan a

cero en SRAM.

5. **Inicialización del runtime Ada:** Se llama a `adainit`, que configura las estructuras internas del runtime. Por ejemplo, tablas de excepciones y la elaboración de paquetes.
6. **Llamada al programa principal:** Finalmente se invoca el procedimiento `Main` declarado en `main.adb`.

Todo este proceso es transparente al programador y está encapsulado en el runtime `embedded-stm32f4`, sin necesidad de escribir código de startup en ensamblador.

4.1.4 Linker script y mapa de memoria

El *linker script* describe al enlazador cómo distribuir las secciones del binario en la memoria física del microcontrolador. En este proyecto, los tamaños de memoria se pasan como símbolos desde el fichero de proyecto GNAT [15]:

```
package Linker is
  for Default_Switches ("Ada") use
    ("-Wl,--gc-sections",
     "-Wl,--print-memory-usage",
     "-Wl,--defsym=_rom_size=512K",
     "-Wl,--defsym=_ram_size=128K");
end Linker;
```

Listing 4.2: Parámetros del enlazador en `nucleo_test.gpr`.

El significado de cada opción es el siguiente:

- `-gc-sections`: Elimina secciones de código y datos no referenciados, reduciendo el tamaño del binario final.
- `-print-memory-usage`: Imprime en la salida del enlazado el uso de Flash y SRAM.
- `_rom_size=512K` y `_ram_size=128K`: Informan al linker script del runtime de los tamaños reales de Flash y SRAM del STM32F446 [1], permitiéndole calcular las direcciones de inicio y fin de cada región.

4.1.5 Gestión de memoria en Bare-metal

El runtime `embedded-stm32f4` no incluye asignador de memoria dinámica (*heap*). Esto significa que el operador `new` de Ada no está disponible, lo que constituye una restricción deliberada y beneficiosa en un sistema embebido de tiempo real [18]:

- Se elimina la posibilidad de fragmentación de memoria, fuente habitual de fallos no reproducibles en sistemas de larga ejecución.
- El consumo de SRAM es completamente predecible en tiempo de compilación.
- No se requiere un recolector de basura ni un asignador, reduciendo el tamaño del runtime y la latencia de las operaciones de memoria.

Toda la memoria del sistema se asigna, por tanto, de forma estática: Variables globales en `.bss` o `.data`, y variables locales en la pila del núcleo o de las tareas Ada.

4.1.6 Configuración del compilador

El fichero de proyecto activa las siguientes opciones del compilador GNAT [15]:

```
package Compiler is
  for Default_Switches ("Ada") use
    Nucleo_Test_Config.Ada_Compiler_Switches &
    ("-gnatws",    -- Suprimir todos los warnings
     "-gnat2022",  -- Usar Ada 2022
     "-gnatj70");  -- Formato legible de mensajes
end Compiler;
```

Listing 4.3: Opciones del compilador en `nucleo_test.gpr`.

El uso de `-gnat2022` habilita las construcciones del estándar Ada 2022 [3], la revisión más reciente del lenguaje.

4.1.7 Flujo de compilación y programación

El flujo completo desde el código fuente hasta el firmware programado en el microcontrolador consta de tres pasos, automatizados mediante el script `Programador.sh`:

1. **Compilación con Alire:** la herramienta `alr` [14] gestiona las dependencias del proyecto y invoca GNAT para compilar y enlazar, produciendo el ejecutable ELF en `bin/main`.
2. **Conversión a binario plano:** `arm-none-eabi-objcopy` extrae las secciones cargables del ELF y genera el archivo `firmware.bin`, que contiene únicamente los bytes a escribir en Flash a partir de `0x08000000`.
3. **Programación con OpenOCD:** OpenOCD [16] se conecta al ST-LINK de la placa Nucleo, detiene el núcleo, borra y escribe la Flash, verifica la imagen y libera la ejecución.

```

alr clean && alr build

arm-none-eabi-objcopy -O binary bin/main firmware.bin

openocd -f interface/stlink.cfg \
        -f target/stm32f4x.cfg \
        -c "init" \
        -c "reset halt" \
        -c "flash write_image erase firmware.bin 0x08000000" \
        -c "verify_image firmware.bin 0x08000000" \
        -c "reset run" \
        -c "exit"

```

Listing 4.4: Script de compilación y programación Programador.sh.

La verificación automática (`verify_image`) garantiza que los bytes escritos en Flash coinciden con el binario generado, detectando posibles errores de programación antes de liberar la ejecución del firmware.

4.2 GPIO - Entradas/Salidas de Propósito General

El acceso a los pines GPIO del STM32F446 desde Ada se hace mediante la manipulación directa de los registros mapeados en memoria, ya que el runtime usado no incluye una HAL(Hardware Abstraction Layer) para GPIO. Para estructurar este acceso se ha desarrollado el paquete GPIO, compuesto por la especificación `gpio.ads` y el cuerpo `gpio.adb`.

4.2.1 Diseño del paquete

El paquete se apoya en dos tipos fundamentales definidos en la especificación:

```

type GPIO_Port is
  (PORT_A, PORT_B, PORT_C, PORT_D,
   PORT_E, PORT_F, PORT_G, PORT_H);

subtype Pin_Number is Integer range 0 .. 15;

type GPIO_Point is record
  Port : GPIO_Port;
  Pin  : Pin_Number;
end record;

type GPIO_Pull is (No_Pull, Pull_Up, Pull_Down);

```

Listing 4.5: Tipos principales del paquete GPIO (`gpio.ads`).

El tipo enumerado `GPIO_Port` representa los ocho puertos del STM32F446 [1]. El subtipo `Pin_Number` restringe el rango de pines al intervalo válido, de modo que cualquier valor fuera de rango es detectado por el verificador de restricciones de Ada. El registro `GPIO_Point` finalmente agrupa puerto y pin en un único objeto.

Las operaciones públicas son las siguientes:

```

procedure Config_Output (GPIO_P : GPIO_Point);
procedure Config_Input  (GPIO_P : GPIO_Point;
                        Pull  : GPIO_Pull := No_Pull);
procedure Config_Analog (GPIO_P : GPIO_Point;
                        Pull  : GPIO_Pull := No_Pull);
procedure Set          (GPIO_P : GPIO_Point);
procedure Clear        (GPIO_P : GPIO_Point);
procedure Toggle       (GPIO_P : GPIO_Point);
function  Read         (GPIO_P : GPIO_Point) return Boolean;

```

Listing 4.6: Interfaz pública del paquete GPIO.

4.2.2 Acceso a registros hardware

El cuerpo del paquete accede a los registros del periférico GPIO mediante punteros calculados a partir de las direcciones base de cada puerto, definidas en el paquete `stm32f446`, donde se agrupan todas las constantes referentes a pines y posiciones de memorias. La función auxiliar `Reg` construye un puntero a `Uint32` a partir de una dirección base y un desplazamiento:

```

function Reg (Base    : Storage_Offset;
              Offset : Storage_Offset) return access Uint32 is
begin
    return Conv.To_Pointer (System.To_Address (Base + Offset));
end Reg;

```

Listing 4.7: Función auxiliar de acceso a registros mapeados en memoria.

Sobre esta función se construyen los accesos de cada registro del periférico GPIO: `MODER` (modo del pin), `PUPDR` (resistencias de pull), `ODR` (salida), `IDR` (entrada) y `BSRR` (set/reset atómico). Este enfoque evita el uso de numerosas estructuras de acceso a registros, sustituyéndolas por funciones que devuelven punteros de acceso, lo que es idiomático en Ada y aumenta la legibilidad del código.

4.2.3 Habilitación del reloj

El STM32F446 requiere habilitar el reloj de cada puerto GPIO antes de acceder a sus registros, activando el bit correspondiente en el registro `RCC_AHB1ENR` [1]. El procedimiento `Enable_Clock` realiza esta operación mediante una operación lógica OR sobre el registro, preservando el

estado del resto de bits:

```
procedure Enable_Clock (P : GPIO_Port) is
begin
    case P is
        when PORT_A => RCC_AHB1ENR := RCC_AHB1ENR or Bit (0);
        when PORT_B => RCC_AHB1ENR := RCC_AHB1ENR or Bit (1);
        when PORT_C => RCC_AHB1ENR := RCC_AHB1ENR or Bit (2);
        ...
    end case;
end Enable_Clock;
```

Listing 4.8: Habilitación del reloj de un puerto GPIO.

Todos los procedimientos de configuración (Config_Output, Config_Input, Config_Analog) invocan Enable_Clock como primera operación, garantizando que el reloj esté activo antes de cualquier acceso al periférico.

4.2.4 Configuración de un pin

La configuración de un pin GPIO se realiza modificando el campo de 2 bits correspondiente en los registros MODER y PUPDR. Para evitar alterar el resto de pines del puerto se emplea el patrón:

$$(R \text{ and not } Mask) \text{ or } Val$$

En esta operación, primero se limpia el campo de bits y posteriormente se escribe el nuevo valor.

La configuración como salida consiste en escribir 01b en MODER, mientras que la configuración como entrada escribe 00b y configura opcionalmente las resistencias *pull-up* o *pull-down* mediante PUPDR. Para ello se utilizan las funciones auxiliares Bit2_Mask y Bit2_Val, encargadas de generar la máscara y el valor correspondientes al pin seleccionado.

```
-- Salida digital
MODER (P).all :=
    (MODER (P).all and not Bit2_Mask (Pin))
    or Bit2_Val (Pin, 1);
-- Entrada digital
MODER (P).all :=
    MODER (P).all and not Bit2_Mask (Pin);
PUPDR (P).all :=
    (PUPDR (P).all and not Bit2_Mask (Pin))
    or Val;
```

Listing 4.9: Configuración de pines GPIO.

4.2.5 Control de salida: Set, Clear y Toggle

La escritura en un pin de salida se realiza a través del registro BSRR (*Bit Set/Reset Register*), que permite activar o desactivar un pin de forma atómica con una única escritura, sin necesidad de leer el estado previo [1]:

```
procedure Set (GPIO_P : GPIO_Point) is
begin
    BSRR (GPIO_P.Port).all := 2 ** GPIO_P.Pin;
end Set;

procedure Clear (GPIO_P : GPIO_Point) is
begin
    BSRR (GPIO_P.Port).all := 2 ** (GPIO_P.Pin + 16);
end Clear;
```

Listing 4.10: Operaciones atómicas de control de pin mediante el registro BSRR.

Los bits 0–15 del registro BSRR activan el pin correspondiente (*Set*); Los bits 16–31 lo desactivan (*Reset*). El procedimiento *Toggle* consulta el estado actual del pin a través del registro ODR (*Output Data Register*) y aplica *Set* o *Clear* según corresponda.

4.2.6 Control de entrada

La función *Read* devuelve el estado lógico de un pin configurado como entrada consultando el registro IDR (*Input Data Register*) y retornando un valor *Boolean*:

```
function Read (GPIO_P : GPIO_Point) return Boolean is
begin
    return (IDR (GPIO_P.Port).all and Bit (GPIO_P.Pin)) /= 0;
end Read;
```

Listing 4.11: Lectura del estado de un pin de entrada.

Devolver *Boolean* en lugar de un entero es una decisión que impide al programador operar aritméticamente y de forma directa con el resultado, lo que reduce la superficie de error.

4.2.7 Ejemplo de uso

El siguiente fragmento ilustra el uso típico del paquete para controlar el LED de usuario de la placa Nucleo-F446RE (D8) y leer el estado del pulsador de usuario (D7):

```
procedure Main is
    LED : constant GPIO_Point := (Port => PORT_A, Pin => 9);
    In_Pin : constant GPIO_Point := (Port => PORT_A, Pin => 8);
begin
```



```

    Config_Output (LED);
    Config_Input(In_Pin, Pull_Down);
loop
    if Read(In_Pin) then
        Set (LED);
        delay(1.0);
        Clear (LED);
    end if;
end loop;
end Main;

```

Listing 4.12: Uso del paquete GPIO para el LED y el pulsador de la Nucleo-F446RE.

En las entradas, es estrictamente necesario configurar las resistencias pull-up o pull-down, ya que, de lo contrario, el valor de la señal quedará indefinido(flotante).

4.3 ADC - Conversor Analógico-Digital

El STM32F446 integra tres conversores analógico-digitales (**ADC1**, **ADC2** y **ADC3**), con una resolución configurable de hasta 12 bits y hasta 19 canales de entrada, incluyendo canales internos para el sensor de temperatura y la tensión interna de referencia [1]. El paquete ADC desarrollado cubre la inicialización, la lectura en modo disparo único, la conversión continua y el acceso a los canales internos.

Los pines analógicos deben configurarse previamente en modo analógico mediante el paquete GPIO (Véase la sección 4.2), llamando a `Config_Analog` antes de inicializar el ADC. Esta separación de responsabilidades hace que el paquete ADC gestione exclusivamente el periférico conversor.

4.3.1 Diseño del paquete

El paquete define los tipos necesarios para registrar todas las opciones de configuración del periférico:

```

subtype ADC_Channel          is Natural range 0 .. 18;
subtype ADC_Channel_Regular is Natural range 0 .. 16;

type ADC_Resolution is (Res_12bit, Res_10bit, Res_8bit, Res_6bit);
type ADC_Alignment  is (Right, Left);
type ADC_Instance    is (ADC_1, ADC_2, ADC_3);
type ADC_Mode        is (Single, Continuous);
-- Canales internos
ADC_CH_Temp          : constant ADC_Channel := 18; -- sensor temperatura

```

```
ADC_CH_Vrefint : constant ADC_Channel := 17; -- referencia interna
```

Listing 4.13: Tipos del paquete ADC (adc.ads).

El uso de subtipos con rango restringido para ADC_Channel impide el acceso a canales fuera del rango válido del hardware [1]. El tipo enumerado ADC_Resolution sustituye la escritura directa de valores binarios en el registro CR1, haciendo el código de configuración más autoexplicativo.

4.3.2 Acceso a registros

El cuerpo del paquete sigue el mismo patrón de acceso a registros mapeados en memoria empleado en el paquete GPIO4.2: La función auxiliar Reg construye un puntero a Uint32 a partir de la dirección base del periférico y el desplazamiento del registro. La dirección base de cada instancia ADC se obtiene mediante ADC_Base:

```
function ADC_Base (A : ADC_Instance) return Storage_Offset is
begin
  case A is
    when ADC_1 => return Storage_Offset (ADC1_Base);
    when ADC_2 => return Storage_Offset (ADC2_Base);
    when ADC_3 => return Storage_Offset (ADC3_Base);
  end case;
end ADC_Base;
```

Listing 4.14: Resolución de la dirección base de cada instancia ADC.

Los registros accesibles desde el paquete son: SR (*Status Register*), CR1 y CR2 (*Control Registers*), SMPR1/SMPR2 (*Sample Time Registers*), SQR1/SQR3 (*Sequence Registers*) y DR (*Data Register*). Adicionalmente, el registro CCR del bloque común de los tres ADC se utiliza para habilitar el sensor de temperatura y la referencia interna.

4.3.3 Habilitación del reloj

El ADC está conectado al bus **APB2**, a diferencia de los GPIO que cuelgan del bus **AHB1** [1]. La habilitación del reloj se realiza activando el bit correspondiente en el registro RCC_APB2ENR:

```
procedure Enable_ADC_Clock (A : ADC_Instance) is
  RCC_APB2ENR : constant access Uint32 :=
    Reg (Storage_Offset (RCC), Storage_Offset (APB2ENR_A));
begin
  case A is
    when ADC_1 => RCC_APB2ENR.all := RCC_APB2ENR.all or Bit
      (ADC1EN);
    when ADC_2 => RCC_APB2ENR.all := RCC_APB2ENR.all or Bit
      (ADC2EN);
```

```

        when ADC_3 => RCC_APB2ENR.all := RCC_APB2ENR.all or Bit
            (ADC3EN);
    end case;
end Enable_ADC_Clock;

```

Listing 4.15: Habilitación del reloj del ADC en el bus APB2.

4.3.4 Inicialización

El procedimiento `Init` configura el ADC en cuatro pasos secuenciales [1]:

1. **Apagado previo:** Se desactiva el bit `ADON` del registro `CR2` antes de modificar la configuración.
2. **Resolución y modo escaneo:** Se escribe el valor de resolución en los bits `RES[1:0]` del registro `CR1` y se desactiva el modo escaneo (`SCAN=0`), ya que se opera con un único canal por conversión.
3. **Alineación y modo:** Se configura la alineación del resultado y el modo de operación (single o continuo) en el registro `CR2`.
4. **Encendido y estabilización:** Se activa `ADON` y se espera un breve periodo antes de permitir las lecturas, para estabilizar los valores.

```

procedure Init (ADC : ADC_Instance;
               Res   : ADC_Resolution := Res_12bit;
               Align : ADC_Alignment  := Right;
               Mode   : ADC_Mode       := Single) is
    Res_Val : Uint32;
begin
    Enable_ADC_Clock (ADC);
    CR2 (ADC).all := CR2 (ADC).all and not Bit (ADC_CR2_ADON);

    case Res is
        when Res_12bit => Res_Val := 0;
        when Res_10bit => Res_Val := 1;
        when Res_8bit  => Res_Val := 2;
        when Res_6bit  => Res_Val := 3;
    end case;

    CR1 (ADC).all :=
        (CR1 (ADC).all
         and not (Bit (ADC_CR1_RES) or Bit (ADC_CR1_RES + 1))
         and not Bit (ADC_CR1_SCAN))
        or (Res_Val * Bit (ADC_CR1_RES));

```

```

... Configuracion de CR2 ...

CR2 (ADC).all := CR2 (ADC).all or Bit (ADC_CR2_ADON);
for I in 1 .. 1000 loop null; end loop;
end Init;

```

Listing 4.16: Secuencia de inicialización del ADC.

4.3.5 Tiempo de muestreo

Antes de cada conversión es necesario configurar el tiempo de muestreo del canal, es decir, el número de ciclos de reloj durante los cuales el condensador interno del ADC se carga con la tensión de entrada. Un tiempo insuficiente produce lecturas erróneas en fuentes con impedancia alta.

El procedimiento `Set_Sample_Time` escribe el valor 111b (480 ciclos, el máximo) en el campo de 3 bits correspondiente al canal en los registros `SMPR1` o `SMPR2`, según si el número de canal es mayor o menor que 10 [1]:

```

procedure Set_Sample_Time (A : ADC_Instance; Ch : ADC_Channel) is
    Val    : constant Uint32 := 7;    -- 111b = 480 ciclos
    Shift  : Natural;
    Mask   : Uint32;
begin
    if Ch <= 9 then
        Shift := Ch * 3;
        Mask  := Bit (Shift) or Bit (Shift + 1) or Bit (Shift + 2);
        SMPR2 (A).all := (SMPR2 (A).all and not Mask)
                        or (Val * Bit (Shift));
    else
        Shift := (Ch - 10) * 3;
        Mask  := Bit (Shift) or Bit (Shift + 1) or Bit (Shift + 2);
        SMPR1 (A).all := (SMPR1 (A).all and not Mask)
                        or (Val * Bit (Shift));
    end if;
end Set_Sample_Time;

```

Listing 4.17: Configuración del tiempo de muestreo a 480 ciclos.

El uso de 480 ciclos es conservador y adecuado para fuentes con impedancia media-alta, como sensores resistivos o divisores de tensión.

4.3.6 Lectura de disparo único

La función Read realiza una conversión completa de un canal en modo disparo único: Selecciona el canal en el registro de secuencia SQR3; Limpia el flag EOC (*End Of Conversion*); Dispara la conversión mediante SWSTART y espera en bucle hasta que EOC se active, devolviendo los 12 bits del registro DR:

```
function Read (ADC : ADC_Instance;
              Ch  : ADC_Channel) return Uint32 is
begin
    Set_Channel (ADC, Ch);
    SR (ADC).all := SR (ADC).all and not Bit (ADC_SR_EOC);
    CR2 (ADC).all := CR2 (ADC).all or Bit (ADC_CR2_SWSTART);
    while (SR (ADC).all and Bit (ADC_SR_EOC)) = 0 loop
        null;
    end loop;
    return DR (ADC).all and 16#0FFF#;
end Read;
```

Listing 4.18: Lectura disparo único.

La función Read_mV convierte el resultado digital a milivoltios asumiendo una tensión de referencia de 3300 mV y resolución de 12 bits:

```
function Read_mV (ADC : ADC_Instance;
                 Ch  : ADC_Channel) return Natural is
    Raw : constant Uint32 := Read (ADC, Ch);
begin
    return Natural ((Raw * 3300) / 4095);
end Read_mV;
```

Listing 4.19: Conversión del resultado ADC a milivoltios.

La división entera introducida por esta fórmula implica un error máximo de ± 1 mV.

4.3.7 Lectura en modo continuo

El modo continuo es adecuado cuando se desea que el ADC realice conversiones de forma autónoma y repetitiva. El procedimiento Start_Continuous activa el bit CONT en CR2 y lanza el primer disparo; a partir de ese momento el hardware realiza la operación de forma automática [1]:

```
procedure Start_Continuous (ADC : ADC_Instance; Ch : ADC_Channel) is
begin
    CR2 (ADC).all := CR2 (ADC).all or Bit (ADC_CR2_CONT);
    Set_Channel (ADC, Ch);
    SR (ADC).all := SR (ADC).all and not Bit (ADC_SR_EOC);
    CR2 (ADC).all := CR2 (ADC).all or Bit (ADC_CR2_SWSTART);
```

```

end Start_Continuous;

function Read_Continuous (ADC : ADC_Instance) return Uint32 is
begin
    return DR (ADC).all and 16#0FFF#;
end Read_Continuous;

```

Listing 4.20: Inicio de la conversión continua y lectura del último resultado.

La función `Read_Continuous` devuelve el último valor disponible en el registro DR sin esperar a la finalización de la conversión actual, lo que la hace no bloqueante.

4.3.8 Sensor de temperatura interno

El STM32F446 incluye un sensor de temperatura integrado conectado al canal 18 del ADC1 [1]. Para utilizarlo es necesario activar el bit `TSVREFE` del registro `CCR` del bloque común, esperar el tiempo de arranque del sensor ($\approx 10 \mu\text{s}$) y realizar la conversión con un tiempo de muestreo elevado.

La función `Read_Temperature` implementa la fórmula de calibración del *datasheet* íntegramente en aritmética entera, evitando el uso de punto flotante y así reduciendo el coste computacional.

La fórmula aplicada es la especificada en el *datasheet* [1]:

$$T [C] = \frac{V_{25} - V_{\text{sense}}}{\text{Avg_slope}} + 25 \quad (4.1)$$

donde $V_{25} = 0,76 \text{ V}$ y $\text{Avg_slope} = 2,5 \text{ mV}/^\circ\text{C}$. El resultado se escala por 100 para expresarlo en centésimas de grado.

4.3.9 Referencia de tensión interna

El canal 17 del ADC1 está conectado a la tensión de referencia interna **Vrefint**, cuyo valor nominal es 1210 mV [1]. Su lectura permite estimar la tensión de alimentación real del sistema (V_{DDA}), lo que es útil para detectar caídas de tensión en posibles dispositivos alimentados por batería o fallos internos del hardware:

```

function Read_Vrefint_mV return Natural is
    Raw : constant Uint32 := Read (ADC_1, ADC_CH_Vrefint);
begin
    return Natural ((Raw * 3300) / 4095);

```

```
end Read_Vrefint_mV;
```

Listing 4.21: Lectura de la referencia de tensión interna.

4.3.10 Ejemplo de uso

El siguiente fragmento ilustra la secuencia completa de uso del paquete: configuración del pin como analógico mediante GPIO, inicialización del ADC y lecturas:

```
procedure Main is

    -- Pines ADC
    PA0 : constant GPIO_Point := (Port => PORT_A, Pin => 0);
    PA1 : constant GPIO_Point := (Port => PORT_A, Pin => 1);
    [. . .]
begin
    Initialize (115200);
    delay(0.1);
    Config_Analog (PA0);
    Config_Analog (PA1);
    Init (ADC_1, Res_12bit, Right, Single);
    Enable_Temp_Sensor;
    Enable_Vrefint;
    delay(0.1);
    loop
        Raw_CH0 := Read      (ADC_1, 0);
        mV_CH0  := Read_mV (ADC_1, 0);
        [. . .]
        Raw_CH1 := Read      (ADC_1, 1);
        mV_CH1  := Read_mV (ADC_1, 1);
        [. . .]
        Temp_Raw := Read_Temperature;
        Temp_Int := Temp_Raw / 100;
        Temp_Dec := abs (Temp_Raw mod 100);
        Send_Line ("Temp raw : " & To_String (Temp_Raw));
        Send_Line ("Temp : " & To_String (Temp_Int)
                    & "."
                    & To_String (Temp_Dec)
                    & " C");

        Vref_mV := Read_Vrefint_mV;
    end loop;
end Main;
```

Listing 4.22: Uso combinado de los paquetes GPIO y ADC para la lectura de una señal analógica.

4.4 Timers

El STM32F446 dispone de catorce timers hardware distribuidos en dos buses: TIM1 y TIM8 son timers avanzados conectados a APB2, mientras que TIM2–TIM7 y TIM12–TIM14 son timers de propósito general y básicos conectados a APB1. El reloj de cada bus puede duplicarse internamente mediante el prescaler de APB, alcanzando hasta 168 MHz en APB2 y 84 MHz en APB1 cuando el sistema opera a máxima frecuencia.

4.4.1 Funcionamiento del contador

El núcleo de cada timer es un contador de 16 o 32 bits (TIM2 y TIM5 son de 32 bits) que se incrementa o decrementa con cada pulso del reloj después de aplicar el prescaler PSC. Cuando el contador alcanza el valor de auto-recarga ARR se produce un evento de actualización (UEV), el contador se reinicia y, opcionalmente, se genera una interrupción o una solicitud DMA. El periodo resultante de la señal de actualización es:

$$T_{\text{timer}} = \frac{(PSC + 1)(ARR + 1)}{f_{\text{clk}}} \quad (4.2)$$

Los registros PSC y ARR disponen de registros **shadow**, el valor escrito no se transfiere al contador activo hasta el siguiente UEV, evitando glitches. Por este motivo `Config_Timer` fuerza un UEV software mediante el bit `UG` del registro `EGR` tras escribir la configuración, garantizando que los valores sean efectivos en el primer ciclo. En modo PWM el canal compara continuamente el contador con el registro `CCRx`. En PWM modo 1 la salida permanece activa mientras el contador es menor que `CCRx` y se desactiva al superarlo, generando un ciclo de trabajo proporcional a $CCRx/ARR$. El preload del `CCRx` permite actualizar el ciclo de trabajo en caliente sin artefactos.

4.4.2 Arquitectura del driver

El driver sigue el mismo patrón de acceso a registros que el paquete GPIO [4.2]. Todos los registros se acceden mediante punteros volátiles contruidos a partir de la dirección base de cada temporizador y el desplazamiento del registro:

Cada registro tiene su propio accesor tipado, por ejemplo:

```
function ARR (Tim : Timer_Id) return access Uint32 is
begin return Reg (Tim_Base (Tim), TIM_ARR_Offset); end ARR;
```

Listing 4.23: Accesor del registro ARR

La habilitación del reloj se realiza sobre `RCC_APB1ENR` o `RCC_APB2ENR` según el bus, usando las

constantes definidas en stm32f446.ads:

```
when TIM_2 => RCC_APB1ENR := RCC_APB1ENR or Bit (TIM2EN);
when TIM_1 => RCC_APB2ENR := RCC_APB2ENR or Bit (TIM1EN);
```

Listing 4.24: Habilitación de reloj en RCC

La configuración básica se concentra en Config_Timer:

```
procedure Config_Timer (Tim          : Timer_Id;
                        Prescaler    : Uint16;
                        Period        : Uint32;
                        Mode          : Count_Mode := Up) is
begin
    Enable_Clock (Tim);
    CR1 (Tim).all := 0;
    PSC (Tim).all := Uint32 (Prescaler);
    ARR (Tim).all := Period;
    CR1 (Tim).all := Cr1_Val or Bit (CR1_ARPE);
    Generate_Update (Tim);
    if Tim = TIM_1 or else Tim = TIM_8 then
        BDTR (Tim).all := BDTR (Tim).all or Bit (BDTR_MOE);
    end if;
end Config_Timer;
```

Listing 4.25: Procedimiento Config_Timer

La configuración PWM actúa sobre los registros CCMRx y CCER. El campo OCxM se sitúa en los bits [6:4] del byte correspondiente al canal dentro de CCMR1 o CCMR2, y el bit OCxPE habilita el preload del comparador:

```
Val  := (OCxM * Uint32 (2 ** 4)) or Bit (3);  -- OCxM + OCxPE
Mask := Uint32 (16#FF#) * Uint32 (2 ** Shift);
CCMR1 (Tim).all := (CCMR1 (Tim).all and not Mask) or Val;
CCER  (Tim).all := CCER (Tim).all or Bit (CCER_CC1E);
```

Listing 4.26: Configuración de canal PWM

Los timers avanzados TIM1 y TIM8 necesitan adicionalmente habilitar el bit MOE en el registro BDTR para que las salidas estén activas; Config_Timer lo gestiona de forma transparente al detectar el identificador del timer.

Los procedimientos públicos quedan recogidos en la Tabla 4.2.

Tabla 4.2: Procedimientos públicos del driver de timers

Subprograma	Descripción
Config_Timer	Configura prescaler, periodo y modo de cuenta
Start / Stop	Activa o detiene el contador (bit CEN)
Generate_Update	Fuerza carga inmediata de registros shadow
Get_Counter / Set_Counter	Lectura y escritura del registro CNT
Config_PWM	Configura un canal en modo PWM con preload
Set_Compare / Get_Compare	Acceso al registro CCR _x en caliente
Config_Capture	Configura un canal como captura de entrada
Get_Capture	Lee el valor capturado en CCR _x
Enable_Update_Interrupt	Activa el bit UIE en DIER
Disable_Update_Interrupt	Desactiva el bit UIE en DIER
Update_Flag / Clear_Update_Flag	Comprueba y borra el flag UIF en SR

4.5 Protocolos básicos de sistemas electrónicos

4.5.1 USART — Comunicación Serie Asíncrona

El STM32F446 integra cuatro **USART** (*Universal Synchronous Asynchronous Receiver Transmitter*) y dos **UART**, accesibles en distintos pines a través del sistema de funciones alternativas de los GPIO [1]. En este proyecto, la **USART2** se utiliza como canal de depuración, aprovechando que la placa Nucleo-F446RE la conecta internamente al ST-LINK/V2-1, que la expone como puerto serie virtual USB en el ordenador de desarrollo sin necesitar hardware adicional.

Protocolo serie asíncrono

La comunicación asíncrona no emplea una señal de reloj compartida entre transmisor y receptor. La sincronización se consigue empleando el mismo *baudrate* en ambos extremos. En el modo **8N1**, la configuración más extendida en el ámbito embebido, cada byte se transmite como una trama de 10 bits [1]:

- **Start bit:** Siempre a nivel lógico 0; Indica al receptor el inicio de la trama.
- **8 bits de datos:** Enviados del bit menos significativo (D0) al más significativo (D7).
- **Stop bit:** Siempre a nivel lógico 1; Marca el final del byte y devuelve la línea al estado de reposo.

Por ejemplo, el carácter 'A' (ASCII 0x41 = 01000001b) se transmite sobre la línea TX de la siguiente forma:

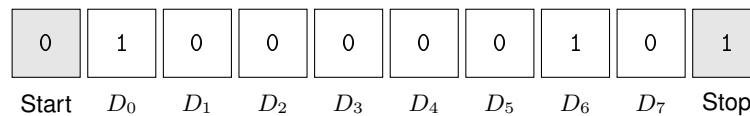


Figura 4.1: Formato de una trama UART asíncrona de 8 bits (8N1).

A 115200 baudios, cada bit ocupa 8,68 μ s, por lo que cada byte completo se transmite en 86,8 μ s.

Diseño genérico del paquete

En la implementación se ha utilizado un **paquete genérico** de Ada para el driver USART. Los parámetros genéricos permiten instanciar el mismo código para cualquier combinación de puerto GPIO, pines TX/RX, dirección base del periférico y función alternativa correspondiente, ya que un mismo pin puede soportar varias funciones alternativas asociadas al protocolo. De este modo, se evita la duplicación de código.

Adicionalmente, en la implementación se ha decidido descartar el modo síncrono, ya que no aporta ninguna ventaja frente al modo asíncrono en nuestro ámbito de aplicación y, además, requiere el uso de una línea adicional.

Registros mapeados en memoria

A diferencia del paquete GPIO, que construye punteros en tiempo de ejecución, el paquete USART declara los registros del periférico directamente como variables con el uso de Address de Ada, ya que la cantidad de direcciones no es tan elevada y reduce la carga de trabajo en tiempo de ejecución:

```
USART_SR : Uint32 with Volatile,
  Address => System'To_Address (USART_Base + USART_SR_Offset);
USART_DR : Uint32 with Volatile,
  Address => System'To_Address (USART_Base + USART_DR_Offset);
USART_BRR : Uint32 with Volatile,
  Address => System'To_Address (USART_Base + USART_BRR_Offset);
USART_CR1 : Uint32 with Volatile,
  Address => System'To_Address (USART_Base + USART_CR1_Offset);
USART_CR2 : Uint32 with Volatile,
  Address => System'To_Address (USART_Base + USART_CR2_Offset);
USART_CR3 : Uint32 with Volatile,
  Address => System'To_Address (USART_Base + USART_CR3_Offset);
```

Listing 4.27: Declaración de registros USART mediante el aspecto Address.

La característica `Volatile` es obligatoria para registros de periférico [3]: Indica al compilador

que el valor de la variable puede cambiar en cualquier momento por causas externas al flujo del programa (Modificación de los bits de estado), impidiendo optimizaciones que impedirían su correcto funcionamiento.

Los registros relevantes son [1]:

- **SR** (*Status Register*): Contiene los flags de estado. Los bits más relevantes son TXE (registro de datos de transmisión vacío), TC (*Transmission Complete*) y RXNE (*Read Data Register Not Empty*).
- **DR** (*Data Register*): Registro de datos compartido para transmisión y recepción; escribir en él lanza la transmisión, leerlo borra el byte recibido.
- **BRR** (*Baud Rate Register*): Fija la velocidad de comunicación a partir del reloj del periférico.
- **CR1**: Control principal; Contiene los bits de habilitación del periférico (UE), el transmisor (TE) y el receptor (RE).
- **CR2**: Configura los bits de stop y el modo síncrono.
- **CR3**: Control de funciones avanzadas (A 0 en modo 8N1 básico).

Inicialización

El procedimiento `Initialize` configura el periférico en seis pasos secuenciales [1]:

1. **Habilitación de relojes GPIO y USART.** Los GPIO se habilitan en el bus AHB1. La USART2 se encuentra en el bus APB1 (45 MHz), mientras que USART1 y USART6 están en APB2 (90 MHz); el procedimiento determina el bus correcto a partir del parámetro `USART_Base`.
2. **Configuración de pines como función alternativa.** Los bits del registro `MODER` correspondientes a TX y RX se escriben con el valor `10b`, modo función alternativa.
3. **Asignación de la función alternativa.** El campo de 4 bits del pin en `AFRL` (pines 0–7) o `AFRH` (pines 8–15) se escribe con el valor correspondiente a la función alternativa.
4. **Resistencias de pull-up.** Se activa pull-up en TX y RX para mantener las líneas en estado de reposo (alto) cuando el periférico no transmite.

5. **Cálculo y escritura del baudrate.** El divisor se obtiene como $BRR = PCLK / \text{baudrate}$. Para la USART2 con APB1 a 42 MHz y 115200 baudios: $BRR = 42\,000\,000 / 115\,200 \approx 364$.

6. **Habilitación del periférico.** Se activan simultáneamente los bits UE, TE y RE en CR1, y se ponen a cero CR2 y CR3 para el modo 8N1 asíncrono.

```
procedure Initialize (Baudrate : Uint32) is
    PCLK1 : Uint32 := 42_000_000;
    Div   : Uint32;
begin

    RCC_AHB1ENR := RCC_AHB1ENR or AHB1_RX_Bit or AHB1_TX_Bit;
    RCC_APB1ENR := RCC_APB1ENR or APB_USART_Bit;
    . . .
    Div := PCLK1 / Baudrate;
    USART_BRR := Div;
    USART_CR1 := (2**CR1_TE) or (2**CR1_RE) or (2**CR1_UE);
    USART_CR2 := 0;
    USART_CR3 := 0;
end Initialize;
```

Listing 4.28: Cálculo del baudrate e inicialización del periférico USART.

Transmisión

La transmisión de un byte se realiza con espera activa sobre dos flags del registro SR [1]:

```
procedure Send_Char (C : Uint8) is
begin
    while (USART_SR and (2**SR_TXE)) = 0 loop
        null;
    end loop;
    USART_DR := Uint32 (C);

    while (USART_SR and (2**SR_TC)) = 0 loop
        null;
    end loop;
end Send_Char;
```

Listing 4.29: Transmisión de un byte con espera sobre los flags TXE y TC.

La espera sobre TXE garantiza que el registro DR puede aceptar el nuevo byte. La espera adicional sobre TC (*Transmission Complete*) asegura que el último bit ha salido físicamente por el pin TX antes de volver al código que hace la llamada, lo que hace que el envío de tramas sea correcto y seguro.

Las funciones `Send_String` y `Send_Line` se construyen sobre `Send_Char`. `Send_Line` añade los caracteres de control CR (0x0D) y LF (0x0A) al final de la cadena, produciendo un salto de línea compatible con terminales serie estándar.

Recepción

La recepción opera de forma similar a la transmisión. La función `Read_Char` bloquea la ejecución hasta que el flag `RXNE` (*Receive Data Register Not Empty*) se activa, indicando que el hardware ha completado la recepción y lo ha depositado en el registro `DR` [1]:

```
function Read_Char return UInt8 is
begin
    while (USART_SR and (2**SR_RXNE)) = 0 loop
        null;
    end loop;
    return UInt8 (USART_DR and 16#FF#);
end Read_Char;
```

Listing 4.30: Recepción bloqueante de un byte.

La máscara `and 16#FF#` extrae únicamente los 8 bits de datos, descartando los bits superiores del registro `DR` que el hardware mantiene a cero pero que podrían provocar un desbordamiento en la conversión.

La función `Data_Available` permite comprobar si hay un byte disponible sin bloquear la ejecución, lo que resulta útil para integrar la recepción en bucles que deben atender otras tareas:

```
function Data_Available return Boolean is
begin
    return (USART_SR and (2**SR_RXNE)) /= 0;
end Data_Available;
```

Listing 4.31: Comprobación no bloqueante de dato disponible en recepción.

Ejemplo de uso

El siguiente fragmento ilustra el uso típico del paquete :

```
with USART_Driver; use USART_Driver;

procedure Main is
begin
    Initialize (115_200);
    Send_Line ("Sistema iniciado");

    loop
```

```

    Send_Line ("Leyendo sensor...");
    [. . .]
    Send_String ("Valor: ");
    . . .
    Send_Line ("");
end loop;
end Main;

```

Listing 4.32: Uso del paquete USART_Driver para depuración por terminal serie.

La salida es visible en cualquier terminal serie, como Putty, conectado al puerto que expone el ST-LINK, configurado a 115200 baudios y raw.

4.5.2 I²C — Bus de Dos Hilos

El protocolo **I²C** (*Inter-Integrated Circuit*), desarrollado por Philips Semiconductors en 1982, es un protocolo serie síncrono de dos hilos que permite conectar múltiples dispositivos esclavos a un maestro usando únicamente las líneas **SCL** (*Serial Clock*) y **SDA** (*Serial Data*) [26]. Su bajo número de señales lo convierte en la interfaz estándar para sensores, pantallas y otros periféricos de baja velocidad en sistemas empotrados. El STM32F446 integra tres controladores I²C compatibles con los modos estándar (100 kbit/s) y rápido (400 kbit/s) [1].

Protocolo I²C

Una transacción I²C sigue siempre la misma estructura de señalización [26]:

1. **Condición de START:** El maestro baja SDA mientras SCLbus permanece alto. Es la única transición de SDA permitida con SCL alto junto a la condición de STOP.
2. **Dirección del esclavo:** Se compone de 7 bits de dirección más 1 bit R/̄W. El esclavo responde con **ACK** (SDA a bajo en el noveno ciclo de SCL), en caso de no recibirlo, el maestro interpreta un **NACK**. Adicionalmente, si la si el esclavo no reconoce la dirección o rechaza un byte, envía un **NACK** (SDA alto en el noveno ciclo). Todo **NACK** recibido o interpretado será tratado generando una señal de stop.
3. **Transferencia de datos:** Se envían los bytes cada uno confirmado con su **ACK** correspondiente.
4. **Condición de STOP:** El maestro sube SDA mientras SCL permanece alto, liberando el bus.

La Figura 4.2 muestra el diagrama de temporización de una transacción I²C completa. Las dos señales físicas del bus presentan el siguiente comportamiento [26]:

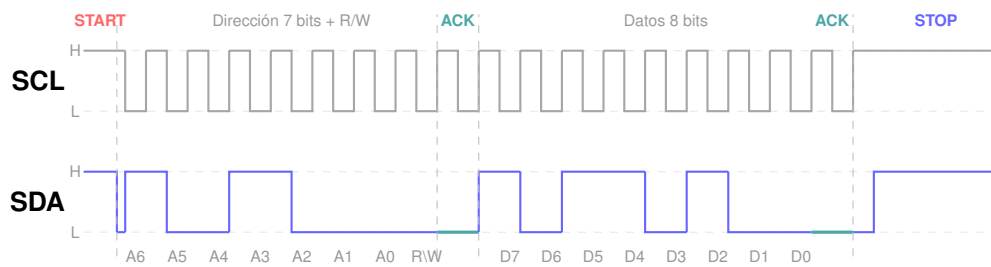


Figura 4.2: Diagrama de temporización de una transacción I²C: START, 7 bits de dirección, bit R/W, ACK del esclavo, byte de datos, ACK y STOP.

Diseño genérico y objeto protegido

Al igual que el paquete USART, el driver I²C se implementa como un **paquete genérico** con la dirección base del periférico y los pines GPIO:A diferencia de las anteriores instancias, esta no necesita las funciones alternativas como parámetro, ya que solo tienen un valor. La instancia concreta se declara en `i2c_driver.ads`:

```
package I2C_Driver is new I2C
(
  I2C_Base => I2C1_Base,
  SCL_PIN  => 8,
  SDA_PIN  => 9,
  GPIO_SCL => GPIOB,
  GPIO_SDA => GPIOB);
```

Listing 4.33: Instanciación del driver I2C para I2C1 con pines PB8/PB9.

Una característica del diseño es que toda la parte pública se encapsula dentro de un **objeto protegido** (protected Bus) [3]. Los objetos protegidos garantizan la exclusión mutua en el acceso a recursos compartidos: El runtime asegura que nunca dos tareas puedan ejecutar simultáneamente un subprograma del objeto protegido, haciendo el bus I²C seguro por construcción en un entorno multitarea [18].

Registros hardware

Los registros del periférico se declaran con `Address + Volatile` [1, 3]. Los más relevantes son [1]:

- **CR1:** Habilitación del periférico (PE), generación de START/STOP, control de ACK y reset software (SWRST).
- **CR2:** Frecuencia del APB1 en MHz, necesaria para calcular los tiempos internos del periférico.

- **SR1:** Flags de estado: SB (START generado), ADDR (Dirección reconocida), TXE (DR listo para escritura), RXNE (Dato disponible), BTF (*Byte Transfer Finished*), AF (NACK), BERR (Error de bus) y ARLO (Pérdida de arbitraje).
- **SR2:** Su lectura conjunta con SR1 limpia el flag ADDR.
- **CCR:** Divisor de reloj para la frecuencia de SCL.
- **TRISE:** Tiempo máximo de subida de SCL en ciclos de APB1

Inicialización

El bus I²C puede quedar bloqueado si una transacción fue interrumpida, dejando un esclavo con SDA a bajo. Para resolverlo, tras configurar los pines se realiza una **recuperación software** mediante 9 pulsos de SCL con SDA alto, garantizando que cualquier esclavo bloqueado completa el byte pendiente y libera la línea, seguida de una condición de STOP [1]:

```
GPIO_ODR_SCL := GPIO_ODR_SCL or Uint32(2**SCL_PIN);
GPIO_ODR_SDA := GPIO_ODR_SDA or Uint32(2**SDA_PIN);
for I in 1 .. 9 loop
    GPIO_ODR_SCL := GPIO_ODR_SCL and not(Uint32(2**SCL_PIN));
    for J in 1 .. 500 loop null; end loop;
    GPIO_ODR_SCL := GPIO_ODR_SCL or Uint32(2**SCL_PIN);
    for J in 1 .. 500 loop null; end loop;
end loop;
GPIO_ODR_SDA := GPIO_ODR_SDA and not(Uint32(2**SDA_PIN));
for J in 1 .. 500 loop null; end loop;
GPIO_ODR_SDA := GPIO_ODR_SDA or Uint32(2**SDA_PIN);
```

Listing 4.34: Recuperación del bus mediante 9 pulsos de SCL.

Tras la recuperación se aplica un reset por SWRST y se configura el periférico. Se emplea ODR en lugar de BSRR por legibilidad. El flag Initialized evita reconfiguraciones accidentales en llamadas sucesivas. Los valores de CCR y TRISE para modo estándar a 100 kHz con APB1 a 42 MHz son:

```
CR2 := (CR2 and not Uint32(16#3F#)) or Uint32(PCLK1_MHz);
CCR := (CCR and not Uint32(16#0FFF#)) or Uint32(210);
TRISE := (TRISE and not Uint32(16#3F#)) or Uint32(PCLK1_MHz + 1);
CR1 := CR1 or Uint32(2**I2C_CR1_PE);
```

Listing 4.35: Configuración de CCR y TRISE (100 kHz, APB1=42 MHz).

Herramientas de diagnóstico

El protocolo presenta una complejidad elevada por ello, para asegurar la correcta implementación y funcionamiento del mismo fue necesaria la implementación de tres procedimientos de diagnóstico que resultaron imprescindibles durante la puesta en marcha:

- **Test_Hardware:** Verifica el estado eléctrico de SCL y SDA leyendo el registro `ODR`. Una línea a bajo en reposo indica ausencia de pull-ups o cortocircuito.
- **Scan_I2C_Bus:** Escanea las 128 direcciones posibles e imprime por USART las que reciben ACK, verificando que los esclavos están correctamente conectados.
- **Test_Minimo:** Comprueba que el periférico puede generar START; si falla, el bus está bloqueado a nivel hardware.

Transmisión

`Write` transmite un byte siguiendo la secuencia descrita en 4.5.2: genera START, envía la dirección desplazada (`SlaveAddr * 2`, bit `R/W = 0`), carga el dato en `DR` y espera `BTF` antes del STOP, garantizando la serialización completa [1]. `Write_Buffer` extiende este esquema para arrays: itera enviando cada byte con espera de `TXE` y comprobación de `AF`, y genera el STOP al finalizar el array o ante cualquier error.

Recepción

La recepción de un byte único requiere la secuencia especial documentada en [1]: deshabilitar ACK *antes* de limpiar `ADDR` (lectura de `SR1/SR2`) y generar STOP *antes* de leer `DR`. `Read_Buffer` generaliza la recepción para N bytes: los primeros $N - 1$ se reciben con ACK activo y el último aplica la misma secuencia especial, restaurando ACK al concluir.

Herramientas de diagnóstico

Dada la complejidad del protocolo, se implementaron tres procedimientos de diagnóstico imprescindibles durante la puesta en marcha: `Test_Hardware`, que verifica el estado eléctrico de SCL y SDA leyendo `ODR` —una línea a bajo en reposo indica ausencia de pull-ups o cortocircuito—; `Scan_I2C_Bus`, que escanea las 128 direcciones posibles e imprime por USART las que reciben ACK; y `Test_Minimo`, que comprueba que el periférico puede generar START, indicando bloqueo hardware si falla.

Espera con timeout

La función auxiliar `Wait_Flag` espera a que un bit de `SR1` se active dentro del plazo indicado; ante `AF`, `BERR`, `ARLO` o expiración del timeout genera STOP y devuelve `False`. El uso de

Ada.Real_Time.Clock hace el timeout independiente de la frecuencia de reloj [18]:

```
function Wait_Flag (Flag_Mask : Uint32;
                   Timeout_MS : Positive) return Boolean is
    Deadline : constant Time := Clock + Milliseconds (Timeout_MS);
begin
    loop
        if (SR1 and Flag_Mask) /= 0 then return True;
        end if;
        if (SR1 and Uint32(2**I2C_SR1_AF)) /= 0 then
            SR1 := SR1 and not Uint32(2**I2C_SR1_AF);
            CR1 := CR1 or Uint32(2**I2C_CR1_STOP); return False;
        end if;
        if (SR1 and Uint32(2**I2C_SR1_BERR)) /= 0 then
            SR1 := SR1 and not Uint32(2**I2C_SR1_BERR);
            CR1 := CR1 or Uint32(2**I2C_CR1_STOP); return False;
        end if;
        if Clock > Deadline then
            CR1 := CR1 or Uint32(2**I2C_CR1_STOP); return False;
        end if;
    end loop;
end Wait_Flag;
```

Listing 4.36: Espera con timeout y detección de errores.

Ejemplo de uso

```
with I2C_Driver; use I2C_Driver;

procedure Main is
    SENSOR_ADDR : constant Uint8 := 16#48#;
    Reg_Value    : Uint8 := 0;
    Ok           : Boolean;
begin
    Bus.Initialize;
    Ok := Bus.Write (SENSOR_ADDR, 16#00#);
    Ok := Bus.Read  (SENSOR_ADDR, Reg_Value);
end Main;
```

Listing 4.37: Lectura de un registro de sensor mediante el driver I2C.

La notación `Bus.Write` / `Bus.Read` refleja que las operaciones se invocan empleando el objeto protegido, garantizando la exclusión mutua.

4.5.3 SPI — Interfaz Serie Síncrona

El protocolo **SPI** (*Serial Peripheral Interface*), introducido por Motorola en los años 80, es una interfaz serie síncrona *full-duplex* que opera con cuatro señales: **SCK** (*Serial Clock*), **MOSI** (*Master Out Slave In*), **MISO** (*Master In Slave Out*) y **CS** (*Chip Select*, activo a bajo nivel) [26]. A diferencia de I²C, SPI no requiere resistencias de *pull-up* ni mecanismo de ACK, lo que permite frecuencias de reloj mucho más elevadas, pudiendo alcanzar los **45 MHz** en el STM32F446 [1].

Protocolo SPI

SPI opera como un sistema maestro-esclavo donde el maestro genera siempre la señal de reloj (SCK). Cuando el maestro quiere iniciar una transacción, baja la línea CS a nivel bajo para seleccionar al esclavo concreto. A partir de ese momento, en cada ciclo de reloj ocurre la comunicación. El maestro envía un bit por MOSI (*Master Out Slave In*) y el esclavo responde simultáneamente por MISO (*Master In Slave Out*). Cuando el maestro vuelve a subir CS, el esclavo queda liberado y la transacción termina. La figura 4.3 muestra el diagrama de temporización de una transacción SPI.

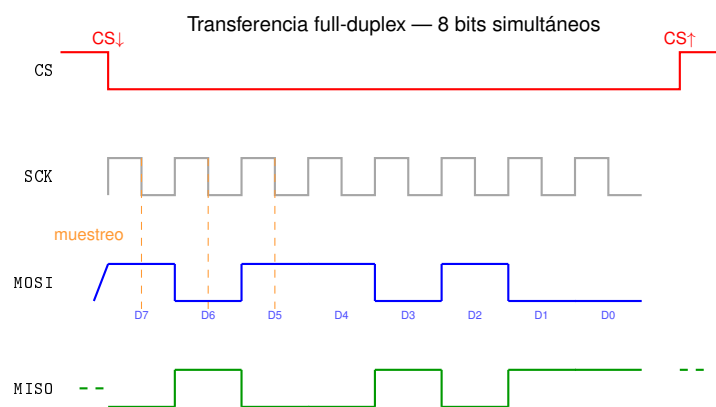


Figura 4.3: Transferencia SPI full-duplex

Diseño genérico del paquete

El paquete SPI sigue el mismo patrón genérico que USART e I2C: los parámetros de instanciación describen completamente el hardware sin necesidad de modificar el código del *driver*. A diferencia de los anteriores, dado el elevado número de parámetros, se ha implementado una función auxiliar para el cálculo automático de los valores alternativos de los pines, reduciendo el número de parámetros necesarios.

Una decisión de diseño relevante es que **CS se gestiona como un GPIO de propósito general**. Esto se consigue activando el modo SSM (*Software Slave Management*, CR1_SSM=1 y SSI CR1_SSI=1). La ventaja es que el código controla el momento exacto de activación y

desactivación del esclavo, necesario para el control de los paquetes referentes al W5500 [4.6.4], que requiere mantener /CS a nivel bajo durante transacciones de múltiples bytes.

A diferencia del paquete I2C, el *driver* SPI no emplea objetos protegidos de Ada. El protocolo SPI es un proceso avanzado con arbitraje, y una implementación basada en objetos protegidos ocasiona problemas de sincronización y rendimiento. Es decir, dada su alta velocidad y la necesidad de sincronizar las señales, el uso del *runtime* queda limitado y, por tanto, el uso de objetos protegidos podría llegar a bloquear el periférico.

Modos SPI y configuración CPOL/CPHA

El comportamiento del reloj SPI se define por dos bits del registro CR1 [1]:

- **CPOL** (*Clock Polarity*): indica el nivel del reloj en reposo. Si CPOL=0, SCK reposa a nivel bajo; si CPOL=1, SCK reposa a nivel alto.
- **CPHA** (*Clock Phase*): indica el flanco de muestreo del dato. Con CPHA=0, el muestreo se realiza en el flanco de subida; con CPHA=1, el muestreo se realiza en el flanco de bajada.

La combinación de ambos parámetros define cuatro modos (Tabla 4.3). Se utiliza el **Modo 0** (CPOL=0, CPHA=0), requerido por el W5500 [22] y el más habitual en periféricos comerciales.

Tabla 4.3: Modos SPI definidos por CPOL y CPHA [1].

Modo	CPOL	CPHA	Reposo SCK	Muestreo
0	0	0	Bajo	Flanco de subida
1	0	1	Bajo	Flanco de bajada
2	1	0	Alto	Flanco de bajada
3	1	1	Alto	Flanco de subida

Función alternativa automática

La función privada SPI_AF resuelve automáticamente el número de función alternativa GPIO correcto a partir de la combinación de periférico SPI, puerto GPIO y pin, evitando el elevado número de parámetros y el sobreacceso a la documentación.

```
function SPI_AF (SPIBase : Uint32; GPIOBase : Uint32;
                 Pin : Natural) return Uint32 is
begin
  case SPIBase is
    when SPI1_Base => return 5;      -- AF5 para todos los pines
    SPI1
```

```

when SPI2_Base =>
  case GPIOBase is
    when GPIOC =>
      case Pin is
        when 2|3 => return 5;
        when 1   => return 7;  -- PC1 MOSI: AF7
        when others => return 16#FFFF_FFFF#;
      end case;
    when others => return 5;
  end case;
when SPI3_Base =>
  [. . .]
when others => return 16#FFFF_FFFF#;
end case;
end SPI_AF;

```

Listing 4.38: Resolución automática de la función alternativa GPIO para cada periférico SPI.

El valor 16FFFF_FFFF actúa como valor para una combinación inválida. También es causado por la obligatoriedad de introducir el valor *when others*.

Inicialización

El procedimiento Initialize configura el periférico en seis pasos [1]:

1. **Habilitación de relojes.** SPI1 y SPI4 cuelgan del bus APB2 (84 MHz); SPI2 y SPI3 del APB1 (42 MHz). La selección es automática a partir de SPI_Base.
2. **CS alto antes de configurar.** Se fuerza CS a alto para evitar que el esclavo interprete ruido durante la configuración como inicio de transacción.
3. **GPIO en modo función alternativa.** SCK, MISO y MOSI se configuran con MODER=10 (AF), modo *push-pull*, velocidad alta y sin pull en SCK y MOSI. MISO recibe pull-up para mantener un nivel definido cuando ningún esclavo emplea la línea.
4. **CS como GPIO salida.** Sin función alternativa.
5. **Configuración del registro CR1:**

```

SPI_CR1 :=
  Uint32(2**SPI_CR1_MSTR)  -- Modo maestro
or Uint32(2**SPI_CR1_SSM)  -- CS por software
or Uint32(2**SPI_CR1_SSI)  -- SSI = 1

```

```
or BR_DIV256; -- 328 kHz (arranque seguro)
```

Listing 4.39: Configuración del registro CR1: maestro, Modo 0, 8 bits, MSB primero.

6. Habilitación del periférico mediante SPE=1.

Gestión dinámica de la velocidad

El driver permite cambiar la velocidad en tiempo de ejecución deshabilitando el periférico, modificando los bits BR[2:0] y rehabilitándolo [1]:

```
BR_DIV256 : constant Uint32 := 16#38#; -- 84/256 = 328 kHz
BR_DIV4   : constant Uint32 := 16#08#; -- 84/4   = 21 MHz

procedure Set_Speed_High is
begin
    SPI_CR1 := SPI_CR1 and not(Uint32(2**SPI_CR1_SPE));
    SPI_CR1 := (SPI_CR1 and not(Uint32(16#38#))) or BR_DIV4;
    SPI_CR1 := SPI_CR1 or Uint32(2**SPI_CR1_SPE);
end Set_Speed_High;
```

Listing 4.40: Cambio dinámico de velocidad entre 328 kHz y 21 MHz.

El arranque a 328 kHz es relevante para dispositivos como tarjetas SD que requieren inicialización lenta. El W5500 admite hasta 80 MHz [22], por lo que opera siempre a 21 MHz una vez inicializado

Transferencia full-duplex

La función principal del *driver* es `Transfer_8`, que realiza una transferencia *full-duplex* de un byte, esperando sobre los flags TXE y RXNE del registro SR [1]:

```
function Transfer_8 (Data : Uint8) return Uint8 is
begin
    loop exit when (SPI_SR and Uint32(2**SPI_SR_TXE)) /= 0; end loop;
    SPI_DR := Uint32(Data);
    loop exit when (SPI_SR and Uint32(2**SPI_SR_RXNE)) /= 0; end loop;
    return Uint8(SPI_DR and 16#FF#);
end Transfer_8;
```

Listing 4.41: Transferencia full-duplex de un byte con espera activa sobre TXE y RXNE.

La secuencia es el patrón principal de escritura y lectura. Finalmente, se espera sobre el flag BSY y se usa al final de transferencias de buffer para asegurar que SCK ha vuelto al reposo antes de subir =CS, evitando que el esclavo interprete la subida como fin prematuro de trama.

La función `Transfer_16` sigue la misma lógica, pero conmuta el bit DFF de `CR1` a 16 bits antes de la transferencia y lo restaura al terminar, deshabilitando y rehabilitando el periférico en cada cambio.

Lectura y escritura simplificadas

Las funciones `Write_8` y `Read_8` eliminan el carácter full-duplex para el programador:

```
procedure Write_8 (Data : Uint8) is
    Dummy : Uint8;
begin
    Dummy := Transfer_8 (Data);
end Write_8;

function Read_8 return Uint8 is
begin
    return Transfer_8 (16#FF#);
end Read_8;
```

Listing 4.42: Abstracciones de solo escritura y solo lectura sobre `Transfer_8`.

El envío de `0xFF` como *dummy byte* en `Read_8` es la convención estándar en SPI: Como el maestro genera siempre el reloj, debe enviar algo para recibir, y `0xFF` no afecta a la mayoría de periféricos. `Transfer_Buffer` generaliza la transferencia a arrays, iterando sobre el rango mínimo de TX y RX para evitar un `Constraint_Error` si tienen longitudes distintas. Adicionalmente, se han creado las abstracciones `Write_Buffer` y `Read_Buffer`.

```
procedure Transfer_Buffer (TX : Uint8_Array; RX : out
    Uint8_Array) is
    Min_Last : constant Positive :=
        (if TX'Last < RX'Last then TX'Last else RX'Last);
begin
    for I in TX'First .. Min_Last loop
        RX (I) := Transfer_8 (TX (I));
    end loop;
    Wait_Not_Busy;
end Transfer_Buffer;
```

Listing 4.43: Operación `Transfer_Buffer`.

Verificación: test de loopback

Durante el desarrollo se verificó el funcionamiento del *driver* mediante un test de *loopback*: con MOSI conectado físicamente a MISO, cualquier byte enviado debe recibirse idéntico. El programa envía `0x55` cada 500 ms y compara el resultado:


```

procedure Main is
[ . . . ]
begin
    USART_Driver.Initialize (115_200);
    SPI_Driver.Initialize;
    loop
        RX := SPI_Driver.Transfer_8 (TX);
        if RX = TX then
            USART_Driver.Send_Line ("SPI OK");
        else
            USART_Driver.Send_Line ("SPI ERROR");
        end if;
        Next_Time := Next_Time + Milliseconds (500);
        delay until Next_Time;
    end loop;
end Main;

```

Listing 4.44: Test de loopback SPI con temporización precisa mediante Ada.Real_Time.

El uso de `delay until Next_Time` en lugar de `delay 0.5` es característico de Ada [13] y tiene implicaciones importantes en sistemas de tiempo real. El primero constituye un retardo absoluto basado en el reloj de Ada.Real_Time: en cada iteración del bucle se incrementa `Next_Time` en un período fijo, de forma que el instante de despertar es siempre un múltiplo exacto del período respecto al instante de inicio. Esto garantiza que los errores de temporización de una iteración no se propaguen a las siguientes, evitando así la acumulación de error a lo largo del tiempo.

El segundo, `delay 0.5`, es un retardo relativo que suspende la tarea durante aproximadamente 0,5 segundos *a partir del momento en que se ejecuta la instrucción*. Dado que el tiempo de ejecución del cuerpo del bucle no es nulo, cada iteración introduce un pequeño error sistemático que se acumula de forma indefinida. En aplicaciones de larga duración o con requisitos estrictos de periodicidad, este comportamiento puede llegar a causar un mal funcionamiento.

4.6 Dispositivos externos

4.6.1 Pantalla OLED SSD1306

El **SSD1306** es un controlador de pantalla OLED (*Organic Light-Emitting Diode*) fabricado por Solomon Systech, capaz de gestionar paneles de hasta 128×64 píxeles unicolor [23]. A diferencia de las pantallas LCD, los píxeles OLED emiten luz propia y no requieren retroiluminación, lo que reduce el consumo cuando la mayoría de píxeles están apagados. El controlador se comunica con el microcontrolador a través del bus I²C (Dirección 0x3C) o SPI. En este proyecto se utiliza la interfaz I²C, conectada al driver `I2C_Driver` descrito en la sección 4.5.2.

Modelos soportados y constantes dependientes

El paquete SSD1306 soporta tres variantes del controlador mediante un tipo enumerado `Display_Model` y constantes calculadas en tiempo de compilación con expresiones `case` [3]:

```
type Display_Model is (SSD1306_128x64 , SSD1306_128x32 ,
    SSD1306_96x16);
Model : constant Display_Model := SSD1306_128x64;

WIDTH : constant Uint8 := (case Model is
    when SSD1306_128x64 => 128, when SSD1306_128x32 => 128,
    when SSD1306_96x16  => 96);

PAGES : constant Uint8 := (case Model is
    when SSD1306_128x64 => 8, when SSD1306_128x32 => 4,
    when SSD1306_96x16  => 2);

BUFFER_SIZE : constant Natural := (case Model is
    when SSD1306_128x64 => 1024, when SSD1306_128x32 => 512,
    when SSD1306_96x16  => 192);
```

Listing 4.45: Selección de modelo y constantes derivadas en `ssd1306.ads`.

Este diseño permite cambiar de modelo modificando una constante (`Model`). Los parámetros de inicialización que difieren entre modelos (`MULTIPLEX_VAL`, `COMPINS_VAL`, `CONTRAST_VAL`, `PRECHARGE_VAL`) se calculan de la misma forma.

Organización de la memoria de vídeo

El SSD1306 organiza su memoria de vídeo interna en **páginas** compuesta por 8 filas de altura [23]. Cada byte corresponde a una columna de 8 píxeles verticales contiguos (El bit 0 es el píxel superior y el bit 7 el inferior). La Figura 4.4 ilustra esta organización para el modelo 128×64.

La estructura de datos que en SRAM del STM32F446 que representará el contenido de la pantalla (Frame buffer) será la siguiente:

```
type Display_Buffer is array (0 .. BUFFER_SIZE - 1) of Uint8;
Frame_Buffer : Display_Buffer := (others => 0);
```

Listing 4.46: Declaración del frame buffer.

Para el modelo 128×64, el frame buffer ocupa **1024 bytes** de SRAM (128 columnas × 8 páginas). El índice de un byte en el buffer para la columna x y la página p se calculará mediante: $p \times 128 + x$.

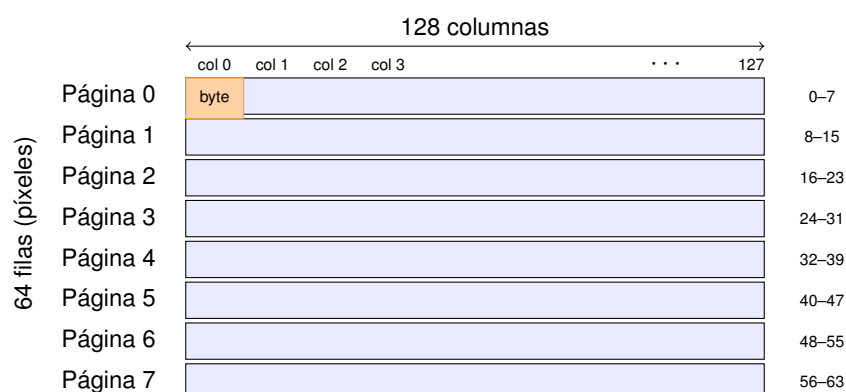


Figura 4.4: Organización de la memoria de vídeo del SSD1306 (128×64)

Protocolo de comunicación e inicialización

El SSD1306 distingue comandos de datos mediante un **byte de control** previo: 0x00 indica comando y 0x40 indica dato (o bloque de datos) [23]. Los procedimientos `Write_Command` y `Write_Data_Buffer` encapsulan esta lógica; el volcado completo de la VRAM se realiza en una única transacción I²C con `Write_Data_Buffer`, que antepone 0x40 y transfiere los 1024 bytes de forma consecutiva.

La inicialización envía la secuencia de comandos requerida por el datasheet [23]: apagado del panel, configuración de reloj, multiplexación, offset y línea de inicio, activación de la bomba de carga, modo de memoria horizontal (avance automático columna→página), remapeo de segmentos, parámetros de pines COM, contraste, precharge y VCOM según el modelo, limpieza del buffer y encendido final (0xAF). Las funciones `Display_On/Off`, `Set_Contrast` y `Set_Inverse` permiten ajustar el panel en tiempo de ejecución.

Operaciones de dibujo

Todas las operaciones gráficas actúan sobre el frame buffer en SRAM; la pantalla no se actualiza hasta llamar a `Update_Display`, evitando parpadeo por actualizaciones incompletas.

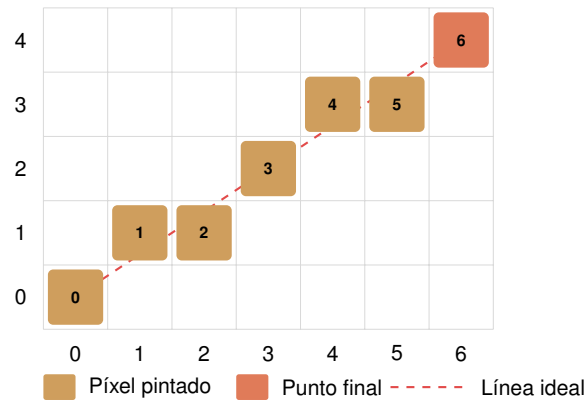
`Draw_Pixel` convierte las coordenadas (x, y) al índice $\lfloor y/8 \rfloor \times 128 + x$ y activa o desactiva el bit $y \bmod 8$ del byte correspondiente. `Draw_Line` implementa el **algoritmo de Bresenham** [23], aritmética entera pura, sin multiplicaciones ni punto flotante, cuya traza para la línea $(0, 0) \rightarrow (6, 4)$ se muestra en la Figura 4.5 y la Tabla 4.4 y siendo el código el siguiente [4.47]:

```

Err := DX - DY;
loop
  Draw_Pixel (Uint8 (X), Uint8 (Y), b);
  exit when X = Integer (X1) and Y = Integer (Y1);
  E2 := 2 * Err;
  if E2 > -DY then Err := Err - DY; X := X + SX; end if;
  if E2 < DX then Err := Err + DX; Y := Y + SY; end if;
end loop;

```

Listing 4.47: Algoritmo de Bresenham.

Figura 4.5: Rasterización de la línea $(0,0) \rightarrow (6,4)$ con Bresenham ($DX = 6$, $DY = 4$).

Paso	X	Y	Err	E2	Condición activa	Acción
0 (init)	0	0	2	4	$E2 > -DY$ y $E2 < DX$	$X++, Y++$
1	1	1	4	8	solo $E2 > -DY$	$X++ \rightarrow (2,1)$
2	2	1	2	4	$E2 > -DY$ y $E2 < DX$	$X++, Y++$
3	3	2	4	8	solo $E2 > -DY$	$X++ \rightarrow (4,2)$
4	4	2	2	4	$E2 > -DY$ y $E2 < DX$	$X++, Y++$
5	5	3	4	8	solo $E2 > -DY$	$X++ \rightarrow (6,3)$
6	6	3	2	4	$E2 > -DY$ y $E2 < DX$	$X++, Y++ \rightarrow (6,4)$
7 (fin)	6	4	—	—	$X = X1$ y $Y = Y1$	exit

Tabla 4.4: Trazas del algoritmo de Bresenham para la línea $(0,0) \rightarrow (6,4)$.

Draw_Rect dibuja el perímetro con cuatro llamadas a Draw_Line; Fill_Rect rellena el interior píxel a píxel. Update_Display configura la ventana de escritura (COLUMNADDR/PAGEADDR) y vuelca el frame buffer completo al controlador en una única transacción I²C.

4.6.2 Fuente de caracteres 5×7

El paquete incluye un bitmap de **5×7 píxeles** para los caracteres ASCII definidos como constantes estáticas [23]:

```
type Font_Col is array (0 .. 4) of Uint8;
type Font_Type is array (Character range '!' .. '~') of Font_Col;
Font : constant Font_Type := (
  'A' => (16#78#, 16#16#, 16#11#, 16#16#, 16#78#),
  'B' => (16#7F#, 16#49#, 16#49#, 16#49#, 16#36#),
  [ ... ]
);
```

Listing 4.48: Declaración de la fuente bitmap 5×7.

Al estar declarada como `constant`, el compilador sitúa en la memoria Flash del STM32F446 el bitmap, no consumiendo SRAM. `Put_Char` copia los 5 bytes directamente al frame buffer añadiendo un sexto byte nulo como separación. `Put_String` invoca `Put_Char` avanzando 6 píxeles por carácter, permitiendo hasta 21 caracteres por línea.

4.6.3 Programa de pruebas

El programa `test_ssd1306.adb` comprueba el funcionamiento mediante una secuencia de siete pruebas:

Tabla 4.5: Secuencia de pruebas del driver SSD1306.

Prueba	Descripción
Test_Text	Cadenas de texto en cuatro líneas
Test_Pixels	Contorno del panel píxel a píxel
Test_Lines	Bordes, diagonales y cruz central
Test_Rectangles	Tres rectángulos concéntricos
Test_Filled_Rectangles	Barras de progreso de distinta anchura
Test_Animation	Bloque 6×6 en trayectoria cuadrada
Test_Effects	Barrido de contraste (0–255) e inversión

La prueba de efectos recorre el rango completo de contraste verificando que el comando `SETCONTRAST` produce respuesta visible en el panel OLED.

4.6.4 Módulo Ethernet W5500

El **W5500** es un controlador Ethernet de WIZnet que integra en un único circuito la pila TCP/IP completa en hardware [22], exponiendo una interfaz de sockets sobre SPI. Esto libera al STM32F446 del procesamiento de tramas y simplifica el firmware Ada.

Funcionamiento general y trama SPI

Toda comunicación con el W5500 se encapsula en una **trama SPI de cuatro campos** [22]: dirección del registro (16 bits), byte de control y datos (Fig. 4.6).

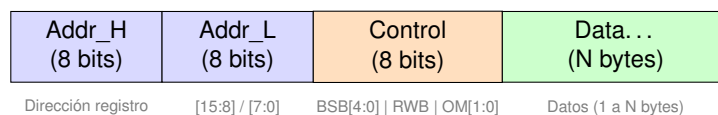


Figura 4.6: Estructura de la trama SPI del W5500. Elaboración propia a partir de [22].

El byte de control codifica: **BSB[7:3]** (bloque de memoria: registros comunes, registros/TX/RX del socket N), **RWB[2]** (0 = lectura, 1 = escritura) y **OM[1:0]** (modo de longitud: VDM variable o FDM fijo a 1/2/4 bytes). En el código se usa siempre VDM (OM=00):

```
COMMON_RD : constant UInt8 := 16#00#; -- BSB=00000 RWB=0 OM=00
COMMON_WR : constant UInt8 := 16#04#; -- BSB=00000 RWB=1 OM=00
SO_REG_RD  : constant UInt8 := 16#08#; -- BSB=00001 RWB=0 OM=00
SO_REG_WR  : constant UInt8 := 16#0C#; -- BSB=00001 RWB=1 OM=00
SO_TX_WR   : constant UInt8 := 16#14#; -- BSB=00010 RWB=1 OM=00
SO_RX_RD   : constant UInt8 := 16#18#; -- BSB=00011 RWB=0 OM=00
```

Listing 4.49: Bytes de control SPI del W5500 (selección).

Los accesos al chip se realizan mediante seis primitivas: Write_Reg/Read_Reg (1 byte), Write_Reg16/Read_Reg16 (2 bytes) y Write_Buf/Read_Buf (longitud variable), todas con la secuencia: CS↓ → Addr_H → Addr_L → Control → Dato(s) → CS↑.

Sockets TCP y UDP

El W5500 implementa 8 sockets hardware independientes, cada uno configurable en modo TCP, UDP, MACRAW o IPRAW, con buffers TX/RX de 2 KiB propios [22]. El microcontrolador los controla mediante los registros Sn_MR, Sn_CR y Sn_SR y los punteros de buffer Sn_TX_WR/Sn_RX_RD.

TCP

TCP garantiza la entrega ordenada y sin errores mediante confirmaciones (ACK), control de flujo por ventana deslizante y un *handshake* de 3 vías para establecer la conexión [1]. La Figura 4.7 muestra su cabecera.

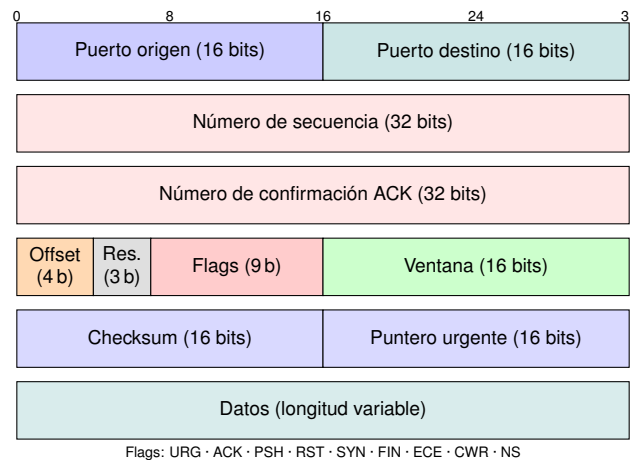


Figura 4.7: Estructura de la cabecera TCP.

El **handshake de 3 vías** funciona así: el cliente envía SYN con su número de secuencia inicial x (SYN_SENT); el servidor responde con SYN+ACK presentando su propio y y confirmando $x + 1$ (SYN_RCVD); el cliente confirma con $ack = y + 1$ y ambos pasan a ESTABLISHED. El cierre sigue la misma lógica con FIN/ACK, permitiendo el *medio cierre* (un lado deja de enviar mientras sigue recibiendo).

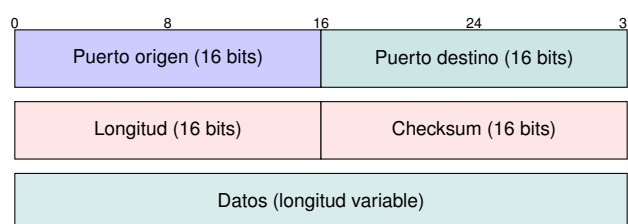
En el W5500, `Socket0_Open_TCP` configura el modo TCP y el puerto local; `Socket0_Connect` escribe la IP y puerto destino y lanza el SYN mediante el comando `CR_CONNECT`. El estado se consulta con `Socket0_Status` (registro `Sn_SR`); los valores relevantes se muestran en la Tabla 4.6.

Tabla 4.6: Estados del socket TCP en el W5500.

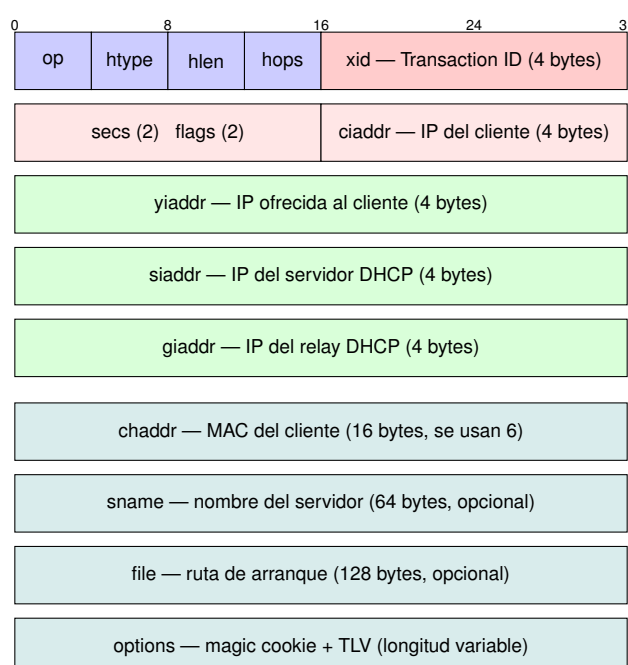
Constante	Valor	Significado
SOCK_CLOSED	0x00	Socket cerrado
SOCK_INIT	0x13	Esperando CONNECT/LISTEN
SOCK_SYNSENT	0x15	SYN enviado, esperando SYN-ACK
SOCK_ESTABLISHED	0x17	Conexión establecida
SOCK_CLOSE_WAIT	0x1C	FIN recibido, cerrando

UDP

UDP es un protocolo sin conexión ni garantía de entrega, adecuado cuando la latencia prima sobre la fiabilidad [22]. Su cabecera de solo 8 bytes (Fig. 4.8) carece de secuenciación ni control de flujo. En el proyecto el Socket 1 se usa para DNS y el Socket 2 (multicast) para mDNS. El W5500 antepone automáticamente 8 bytes de cabecera interna en recepción UDP (IP origen, puerto origen, tamaño) que el firmware debe descartar antes de parsear el payload.

**Figura 4.8:** Estructura de la cabecera UDP.

DHCP DHCP obtiene automáticamente IP, máscara, gateway y DNS desde un servidor de la red local [22], siguiendo la secuencia DORA descrita en 4.6.4. Opera sobre UDP en los puertos 67 (servidor) y 68 (cliente), con la estructura de mensaje mostrada en la Figura 4.9.

**Figura 4.9:** Estructura del mensaje DHCP.

Secuencia DORA

1. **Discover:** broadcast UDP a 255.255.255.255:67 con la MAC del cliente en chaddr y opción 53 = DHCPDISCOVER.
2. **Offer:** el servidor propone una IP en yiaddr junto con máscara, gateway y tiempo de concesión.
3. **Request:** el cliente acepta la oferta (opción 50 con la IP solicitada).

4. **ACK**: el servidor confirma la asignación; el cliente configura su interfaz.

La función `DHCP_Request` implementa el proceso completo en el Socket 3 (UDP broadcast). Devuelve un registro Ada con todos los parámetros; si `Success = False` (timeout o red sin DHCP) el firmware recurre a una dirección estática de *fallback*:

```
type DHCP_Result is record
    Success : Boolean;
    IP       : Uint8_Array (1 .. 4);
    Subnet   : Uint8_Array (1 .. 4);
    Gateway  : Uint8_Array (1 .. 4);
    DNS      : Uint8_Array (1 .. 4);
    Lease    : Uint32;
end record;

function DHCP_Request (Timeout_MS : Natural := 10_000)
    return DHCP_Result;
```

Listing 4.50: Tipo resultado de DHCP.

DNS

DNS traduce nombres de dominio en direcciones IPv4 [22]. El nombre se codifica en el campo QNAME como etiquetas precedidas de su longitud y terminadas en 0x00 (p. ej., `www.google.com` → 03 77 77 77 06 67 6F 6F 67 6C 65 03 63 6F 6D 00). La Figura 4.10 muestra la estructura del mensaje.

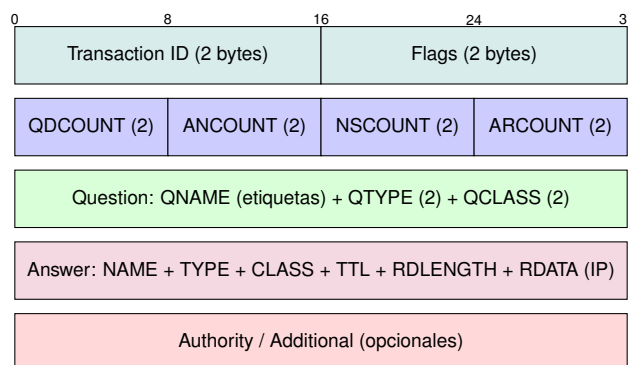


Figura 4.10: Estructura del mensaje DNS.

`Resolve_DNS` construye la query, la envía al servidor por UDP (puerto 53, Socket 1) y parsea la respuesta extrayendo el primer registro tipo A. El parser maneja compresión de nombres (0xC0 XX), salta la sección Question y recorre Answer hasta encontrar `RDLENGTH=4`:

```
function Resolve_DNS (Hostname      : String; DNS_Server : Uint8_Array;
                     IP_Out        : out Uint8_Array;
```

```
Timeout_MS : Natural := 3000) return Boolean;
```

Listing 4.51: Signatura de `Resolve_DNS`.

HTTP

`HTTP_Server` implementa un servidor HTTP/1.1 mínimo sobre el Socket 0 TCP (puerto 80): espera `SOCK_ESTABLISHED`, lee la petición GET y responde con una página HTML estática [22] (Fig. 4.11). La variante `HTTP_Server_mDNS` atiende simultáneamente las consultas mDNS mientras espera conexiones.

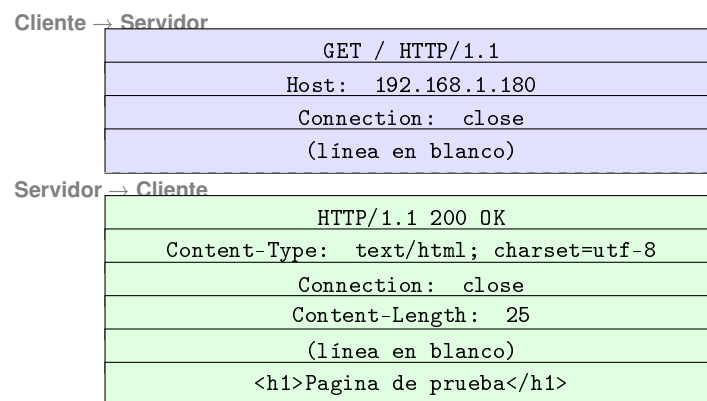


Figura 4.11: Transacción HTTP/1.1 básica implementada en el W5500.

Ping (ICMP)

El W5500 no implementa ICMP en hardware; el modo MACRAW del Socket 0 permite construir tramas Ethernet completas. `Send_Ping_Request` ensambla manualmente los 98 bytes de la trama (Ethernet 14 + IP 20 + ICMP 64) y calcula los checksums por complemento a uno [22]. Ping coordina apertura del socket, envío del *Echo Request* y espera del *Echo Reply* con timeout configurable (por defecto 3000 ms):

```
function Ping (Dest_IP      : Uint8_Array;
               Timeout_MS : Natural := 3000) return Boolean is
begin
    Socket0_Open_Raw;
    Send_Ping_Request (Dest_IP, Seq => 1);
    Success := Receive_Ping_Reply (Timeout_MS);
    Socket0_Close;
    return Success;
end Ping;
```

Listing 4.52: Función de ping con MACRAW.

mDNS

mDNS (RFC 6762) resuelve nombres `.local` sin servidor central, mediante mensajes DNS enviados a la dirección multicast 224.0.0.251 en el puerto UDP 5353 [22]. El Socket 2 se abre en modo UDP multicast (`Sn_MR = 0x82`) configurando previamente la MAC multicast destino 01:00:5E:00:00:FB en `Sn_DHAR`. El flujo consta de dos fases: **Anuncio inicial** (`mDNS_Announce`) al arrancar, que publica la asociación `stm32.local` → IP con TTL de 120 s; y **bucle de respuesta** (`mDNS_Loop`), que atiende queries cuyo QNAME coincida con `stm32.local` y responde con la respuesta correspondiente. El hostname se codifica con `Encode_DNS_Name`, que añade automáticamente el sufijo `.local`, permitiendo configurar cualquier nombre sin modificar el código de respuesta.

Integración y uso conjunto

El arranque sigue la secuencia: Verificación de hardware → DHCP → DNS → operación (mDNS+HTTP). La Tabla 4.7 resume la asignación de sockets; el Listado 4.53 ilustra el flujo completo.

```

if Read_Version /= 16#04# then ... end if;           -- W5500 = 0x04
if (Read_PHYCFGR and 1) = 0 then ... end if;         -- Link up = bit 0
declare
    R : constant DHCP_Result := DHCP_Request (10_000);
begin
    if R.Success then
        Set_IP (R.IP); Set_Subnet (R.Subnet); Set_Gateway (R.Gateway);
    end if;
end;
declare
    Server_IP : Uint8_Array (1 .. 4); Ok : Boolean;
begin
    Ok := Resolve_DNS ("api.ejemplo.com", R.DNS, Server_IP);
end;
mDNS_Announce ("stm32");
loop
    HTTP_Server_mDNS (80, "stm32");
end loop;

```

Listing 4.53: Secuencia de arranque de la pila de red con el W5500.

Tabla 4.7: Asignación de sockets del W5500 en el sistema.

Socket	Modo	Puerto	Uso
0	TCP / MACRAW	80 / –	HTTP servidor / Ping ICMP
1	UDP	1024	Cliente DNS
2	UDP multicast	5353	mDNS
3	UDP broadcast	68	Cliente DHCP

4.7 Prueba del sistema completo

Para validar el conjunto de bibliotecas desarrolladas, se ha implementado una simulación de un sistema embebido real que integra tres tareas concurrentes ejecutándose simultáneamente sobre el microcontrolador STM32F446:

- **Tarea Heartbeat (Blink_Task):** Indicador de vida del sistema mediante el parpadeo periódico de un LED cada 500,ms, implementado con temporización precisa mediante `Ada.Real_Time`.
- **Tarea de pantalla (Pintar_Task):** Secuencia de pruebas sobre la pantalla OLED SSD1306 vía I²C, que cubre renderizado de texto, píxeles, líneas, rectángulos, efectos de contraste e inversión, y animación de un objeto en movimiento.
- **Servidor HTTP con mDNS:** Tarea principal que lee la temperatura interna del microcontrolador a través del ADC, anuncia el dispositivo en la red local bajo el nombre `ejemplo.local` mediante mDNS, y sirve el valor de temperatura en tiempo real como respuesta HTML a cualquier cliente HTTP que acceda al puerto 80.

La ejecución concurrente de estas tres tareas demuestra la correcta integración de los subsistemas de GPIO, I²C (SSD1306), ADC, SPI y red (W5500) desarrollados a lo largo del proyecto. En la figura 4.12 se muestra el resultado de la prueba a nivel de servidor, donde se aprecia la respuesta HTTP con el valor de temperatura interna del microcontrolador al acceder desde un navegador a `ejemplo.local`.



Figura 4.12: Resultado prueba del sistema completo.

Los resultados confirman el funcionamiento estable del sistema bajo multitarea: el LED mantiene su cadencia sin interrupciones, la pantalla completa su secuencia de pruebas y el servidor responde correctamente a las peticiones HTTP, todo ello de forma simultánea y sin interferencias entre tareas.

5 Conclusiones

5.1 Conclusiones del trabajo

El presente proyecto ha demostrado la viabilidad del lenguaje Ada para el desarrollo de sistemas IoT, cumpliendo con los principales objetivos planteados.

En el ámbito del hardware, se ha diseñado y validado una plataforma basada en el microcontrolador STM32F446, integrando los periféricos de comunicación necesarios y los módulos externos. La placa de desarrollo Nucleo-F446RE ha demostrado ser una plataforma adecuada para la validación del sistema, con un coste reducido y un ecosistema de herramientas maduro.

En cuanto al software, se ha configurado completamente el entorno de desarrollo para Ada sobre arquitectura ARM Cortex-M4, empleando el runtime bajo el perfil Ravenscar-SFP, permitiendo programar de forma idiomática y obteniendo todas las ventajas del lenguaje. Se han desarrollado librerías funcionales para todos los periféricos básicos y para los protocolos de comunicación más complejos, siguiendo un enfoque modular y basado en paquetes genéricos que favorece la reutilización del código.

En el área IoT, se ha implementado una pila de comunicaciones completa utilizando el módulo Ethernet W5500, abarcando los protocolos TCP, UDP, DHCP, DNS, HTTP, ICMP (ping) y mDNS. Esta implementación ha permitido que el sistema se integre en redes, obteniendo configuración de red mediante DHCP, resolviendo nombres de dominio y sirviendo contenido web a través de un servidor HTTP básico.

La validación mediante pruebas ha verificado el correcto funcionamiento de cada capa del sistema, demostrando que Ada puede utilizarse como una alternativa real y viable frente a los lenguajes tradicionalmente empleados en sistemas empujados, como C o C++. De este modo, garantizando una mayor seguridad y facilidad de mantenimiento en los sistemas empujados.

5.2 Líneas de trabajo futuro

A partir del trabajo realizado, se identifican varias líneas de desarrollo futuro que podrían extenderse más allá del alcance inicial:

- **Ampliación de la pila IoT:** Incorporar protocolos de capa de aplicación adicionales como MQTT o CoAP, especialmente relevantes en entornos IoT, así como soporte para cifrado TLS.

- **Desarrollo de un depurador visual:** Construir una herramienta gráfica que permita monitorizar en tiempo real las variables del sistema, los estados de los sockets y las tramas de red, facilitando la depuración y la docencia.
- **Placa PCB definitiva:** Fabricar la placa de circuito impreso diseñada en KiCad y validar el firmware sobre el hardware final, realizando pruebas de consumo energético y robustez en condiciones reales.
- **Desarrollo de un sistema de archivos:** Uno de las mayores problemáticas fue la dificultad para implementar un sistema de archivos mediante Ada, por tanto, desarrollar esto sería una gran aportación al entorno creado.
- **Material docente:** Elaborar guías de laboratorio y materiales didácticos basados en este trabajo para su uso en asignaturas relacionadas con sistemas empuotrados y tiempo real.

Bibliografía

- [1] STMicroelectronics, *STM32F446xx Advanced Arm-based 32-bit MCUs Datasheet*, DS10693, 2024.
- [2] STMicroelectronics, *STM32F446 Product Overview*, Disponible en: <https://www.st.com/en/microcontrollers-microprocessors/stm32f446.html>
- [3] Ada Rapporteur Group, *Annotated Ada Reference Manual, Ada 2022 (ISO/IEC 8652:2023)*, Disponible en: <https://ada-lang.io/docs/arm/AA-0.2>
- [4] Ada Community, *ada-lang.io: A programming language for readable, correct, and performant software*, Disponible en: <https://ada-lang.io/>
- [5] AdaCore, *AdaCore Documentation*, Disponible en: <https://www.adacore.com/documentation>
- [6] Ada Information Clearinghouse, *Ada 2005 Reference Manual (ISO/IEC 8652:1995 with AMD 1:2007)*, Disponible en: https://www.adaic.org/resources/add_content/standards/05rm/html/RM-0-5.html
- [7] ARM Limited, *ARM Cortex-M4 Processor Datasheet*, Disponible en: <https://developer.arm.com/documentation/102832/0100>
- [8] STMicroelectronics, *STM32F446xx Reference Manual (RM0390)*, Rev. 7, 2023. Disponible en: https://www.st.com/resource/en/reference_manual/rm0390.pdf
- [9] ARM Holdings, *Cortex-M Processor Series*, ARM Developer Documentation, 2024. Disponible en: <https://developer.arm.com/Processors/Cortex-M4>
- [10] ARM Holdings, *ARMv7-M Architecture Reference Manual*, ARM DDI 0403E.e, 2021. Disponible en: <https://developer.arm.com/documentation/ddi0403/latest>
- [11] ARM Holdings, *Cortex-M4 Technical Reference Manual*, ARM DDI 0439D, Rev. r0p1, 2010. Disponible en: <https://developer.arm.com/documentation/ddi0439/latest>
- [12] IEEE, *IEEE Standard for Floating-Point Arithmetic*, IEEE Std 754-2019, 2019. doi:10.1109/IEEESTD.2019.8766229

- [13] Ada Resource Association, *Ada 2022 Reference Manual: Language and Standard Libraries*, ISO/IEC 8652:2023. Disponible en: <https://ada-auth.org/standards/22rm/html/RM-TTL.html>
- [14] Alire Contributors, *Alire: Ada Library REpository*, Disponible en: <https://alire.ada.dev>
- [15] AdaCore, *GNAT Pro: Industrial-Strength Ada Development Environment*, Disponible en: <https://www.adacore.com/gnat-pro>
- [16] OpenOCD Project, *OpenOCD: Open On-Chip Debugger User's Guide*, Disponible en: <https://openocd.org/doc/html/index.html>
- [17] STMicroelectronics, *STM32CubeProgrammer Software Description*, UM2237, 2024. Disponible en: <https://www.st.com/en/development-tools/stm32cubeprog.html>
- [18] Alan Burns, Brian Dobbing y Tullio Vardanega, *Guide for the Use of the Ada Ravenscar Profile in High Integrity Systems*, University of York, 2003. Disponible en: <https://www-users.cs.york.ac.uk/~burns/review.pdf>
- [19] AdaCore, *SPARK User's Guide*, AdaCore Documentation, 2024. Disponible en: <https://docs.adacore.com/spark2014-docs/html/ug/en/index.html>
- [20] Barr, M. y Massa, A., *Programming Embedded Systems in C and C++*, O'Reilly Media, 2006. ISBN: 978-0596009830.
- [21] KiCad EDA, *KiCad EDA: A Cross Platform and Open Source Electronics Design Automation Suite*, Disponible en: <https://www.kicad.org>
- [22] WIZnet, *W5500 Datasheet: Hardwired TCP/IP Embedded Ethernet Controller*, Ver. 1.0.9, 2023. Disponible en: <https://docs.wiznet.io/Product/iEthernet/W5500/datasheet>
- [23] Solomon Systech, *SSD1306: 128 x 64 Dot Matrix OLED/PLED Segment/Common Driver with Controller*, Ver. 1.1, 2008. Disponible en: <https://cdn-shop.adafruit.com/datasheets/SSD1306.pdf>
- [24] I. Sommerville, *Software Engineering*, 10th ed. Pearson, 2016.
- [25] AdaCore, *bb-runtimes: Bare-metal runtimes for Ada*, 2024. Disponible en: <https://github.com/AdaCore/bb-runtimes>

- [26] F. Leens, *An Introduction to ℓ^2C and SPI Protocols*, IEEE Instrumentation & Measurement Magazine, vol. 12, no. 1, pp. 8–13, Feb. 2009. Disponible en: <https://doi.org/10.1109/MIM.2009.4762946>
- [27] K. I. Joy, “Bresenham’s Algorithm,” *On-Line Computer Graphics Notes*, Visualization and Graphics Research Group, Dept. of Computer Science, University of California, Davis. Disponible en: <https://www.cs.put.poznan.pl/swilk/pmwiki/uploads/Dydaktyka/bresenham-int.pdf> [Acceso: mayo 2026].

Anexo I: Guía de instalación y configuración del entorno de desarrollo

Requisitos previos

Para poder emplear este código es imprescindible disponer de un sistema operativo que utilice el kernel de Linux. A continuación se detallan los pasos necesarios para configurar el entorno de desarrollo completo.

Instalación de Git

En primer lugar, se debe instalar `git` mediante el gestor de paquetes correspondiente al sistema operativo. En el siguiente ejemplo se emplea una distribución basada en Debian:

```
sudo apt install git -y
```

Listing 1: Instalación de Git en sistemas Debian/Ubuntu

Instalación de Alire

A continuación, se accederá a la página oficial de Alire, el compilador y gestor de versiones para Ada, disponible en:

<https://alire.ada.dev/>

Se descargará el binario correspondiente y se añadirá al `PATH` del sistema para poder invocarlo sin necesidad de especificar rutas absolutas. Posteriormente, se actualizarán los repositorios de índices para garantizar la compatibilidad con el *runtime*:

```
export PATH=$PATH:/ruta/a/alire/bin  
alr index --update-all
```

Listing 2: Configuración del `PATH` y actualización de repositorios de Alire

Clonado del repositorio

Una vez configurado el entorno, se clonará el repositorio del proyecto mediante el siguiente comando:

```
git clone https://github.com/j3ngibre/Ada_STM32F446_library.git
```

Listing 3: Clonado del repositorio `Ada_STM32F446_library`

Empleando el script `./Programador.sh` incluido en el repositorio, será posible compilar y cargar el código en la placa de forma automática.

Configuración opcional: Visual Studio Code

Para facilitar el desarrollo, se recomienda instalar Visual Studio Code como entorno de edición. Puede instalarse mediante:

```
sudo apt install code
```

Listing 4: Instalación de Visual Studio Code

Extensión para comentarios en bloque en Ada

Se ha desarrollado una extensión para Visual Studio Code que permite comentar bloques de código Ada de forma cómoda. El funcionamiento es el siguiente: se rodean las líneas que se desean comentar con marcadores `-/`, como se muestra en el siguiente ejemplo:

```
-/  
Hi  
How are you?  
-/
```

Listing 5: Sintaxis de la extensión antes de ejecutarse

Tras ejecutar la extensión (de forma automática o mediante comando), el bloque se transforma en:

```
-- -/  
-- Hi  
-- How are you?  
-- -/
```

Listing 6: Resultado tras aplicar la extensión de comentarios

Instrucciones de instalación:

1. Descargar el paquete `.vsix` desde el repositorio oficial:

```
https://github.com/j3ngibre/VsCodePlugins/tree/main/releases
```

2. En Visual Studio Code, acceder a *Extensiones* → *Instalar desde VSIX...* y seleccionar el archivo descargado.
3. La extensión quedará instalada y lista para su uso.

Anexo II: Código fuente

Todo el código fuente desarrollado para este trabajo se encuentra disponible en el siguiente repositorio: https://github.com/j3ngibre/Ada_STM32F446_library

Paquete STM32f446

```
package stm32f446 is

    type Uint8 is mod 2**8;
    type Uint16 is mod 2**16;
    type Uint32 is mod 2**32;
    type Uint8_Array is array (Positive range <>) of Uint8;

    -- Direcciones base
    RCC      : constant := 16#4002_3800#;
    GPIOA    : constant := 16#4002_0000#;
    GPIOB    : constant := 16#4002_0400#;
    GPIOC    : constant := 16#4002_0800#;
    GPIOD    : constant := 16#4002_0C00#;
    GPIOE    : constant := 16#4002_1000#;
    GPIOF    : constant := 16#4002_1400#;
    GPIOG    : constant := 16#4002_1800#;
    GPIOH    : constant := 16#4002_1C00#;

    SPI1_Base : constant := 16#4001_3000#;
    SPI2_Base : constant := 16#4000_3800#;
    SPI3_Base : constant := 16#4000_3C00#;
    SPI4_Base : constant := 16#4001_3400#;
    I2C1_Base : constant := 16#4000_5400#;
    I2C2_Base : constant := 16#4000_5800#;
    I2C3_Base : constant := 16#4000_5C00#;
    USART1_Base : constant := 16#4001_1000#;
    USART2_Base : constant := 16#4000_4400#;
    USART3_Base : constant := 16#4000_4800#;
    UART4_Base  : constant := 16#4000_4C00#;
    UART5_Base  : constant := 16#4000_5000#;
    USART6_Base : constant := 16#4001_1400#;
    I2C_OAR1    : constant := 16#08#;
    I2C_OAR2    : constant := 16#0C#;
    I2C_DR      : constant := 16#10#; -- 8bits los otros 8 reservados
    ADC1_Base   : constant := 16#4001_2000#;
    ADC2_Base   : constant := 16#4001_2100#;
```

```
ADC3_Base: constant := 16#4001_2200#;
ADC_Common_Registers: constant := 16#4001_2300#;
    ADC_CSR_Offset : constant := 16#00#;
    ADC_CCR_Offset : constant := 16#04#;
    ADC_CDR_Offset : constant := 16#08#;
```

```
GPIO_MODER_Offset: constant:=16#00#;
GPIO_AFR_L_Offset: constant:=16#20#;
GPIO_AFR_H_Offset: constant:=16#24#;
GPIO_OTYPER_Offset: constant:=16#04#;
GPIO_OSPEEDR_Offset: constant:=16#08#;
GPIO_PUPDR_Offset: constant:=16#0C#;
GPIO_IDR_Offset: constant:=16#10#;
GPIO_ODR_Offset: constant:=16#14#;
    GPIO_BSRR_Offset: constant:=16#18#;
```

```
AHB1ENR_A: constant := 16#30#;
    OTGHSULPIEN: constant:=30;
    OTGHSEN: constant:=29;
    DMA2EN: constant:=22;
    DMA1EN: constant:=21;
    BKPSRAMEN: constant:=18;
    CRCEN: constant:=12;
    GPIOHEN: constant:=7;
    GPIOGEN: constant:=6;
    GPIOFEN: constant:=5;
    GPIOEEN: constant:=4;
    GPIODEN: constant:=3;
    GPIOCEN: constant:=2;
    GPIOBEN: constant:=1;
    GPIOAEN: constant:=0;
```

```
APB1ENR_A: constant := 16#40#;
    DACEN: constant:=29;
    PWREN: constant:=28;
    CECEN: constant:=27;
    CAN2EN: constant:=26;
    CAN1EN: constant:=25;
    FMPI2C1EN: constant:=24;
    I2C3EN: constant:=23;
    I2C2EN : constant:=22;
    I2C1EN : constant:=21;
    USART5EN: constant:=20;
```

```
USART4EN: constant := 19;
USART3EN : constant := 18;
USART2EN : constant := 17;
SPDIFRXEN: constant := 16;
SPI3EN: constant := 15;
SPI2EN: constant := 14;
WWDEGEN: constant := 11;
TIM14EN: constant := 8;
TIM13EN: constant := 7;
TIM12EN: constant := 6;
TIM7EN: constant := 5;
TIM6EN: constant := 4;
TIM5EN: constant := 3;
TIM4EN: constant := 2;
TIM3EN: constant := 1;
TIM2EN: constant := 0;

APB2ENR_A : constant := 16#44#;
TIM1EN: constant := 0;
TIM8EN: constant := 1;
USART1EN: constant := 4;
USART6EN: constant := 5;
ADC1EN: constant := 8;
ADC2EN: constant := 9;
ADC3EN: constant := 10;
SDIOEN: constant := 11;
SPI1EN: constant := 12;
SPI4EN: constant := 13;
SYSCFGEN: constant := 14;
TIM9EN: constant := 16;
TIM10EN: constant := 17;
TIM11EN: constant := 18;
SAI2EN: constant := 22;
SAI1EN: constant := 23;
```

```
USART_DR_Offset : constant := 16#04#;
USART_BRR_Offset : constant := 16#08#;

USART_CR2_Offset : constant := 16#10#;
USART_CR3_Offset : constant := 16#14#;
USART_CR1_Offset : constant := 16#0C#;
CR1_UE          : constant := 13;
```

```
CR1_M      : constant := 12;
CR1_WAKE   : constant := 11;
CR1_PCE    : constant := 10;
CR1_PS     : constant := 9;
CR1_PEIE   : constant := 8;
CR1_TXEIE  : constant := 7;
CR1_TCIE   : constant := 6;
CR1_RXNEIE : constant := 5;
CR1_IDLEIE : constant := 4;
CR1_TE     : constant := 3;
CR1_RE     : constant := 2;
CR1_RWU    : constant := 1;
CR1_SBK    : constant := 0;

-- Bits de SR
USART_SR_Offset : constant := 16#00#;
SR_CTS          : constant := 9;
SR_LBD          : constant := 8;
SR_TXE          : constant := 7;
SR_TC           : constant := 6;
SR_RXNE         : constant := 5;
SR_IDLE         : constant := 4;
SR_ORE          : constant := 3;
SR_NF           : constant := 2;
SR_FE           : constant := 1;
SR_PE           : constant := 0;

-- Offsets de registros I2C
I2C_CR1        : constant := 16#00#;
I2C_CR1_PE     : constant := 0;
I2C_CR1_SMBUS  : constant := 1;
-- Bit 2 reserved
I2C_CR1_SMBTYPE : constant := 3;
I2C_CR1_ENARP   : constant := 4;
I2C_CR1_ENPEC   : constant := 5;
I2C_CR1_ENGC    : constant := 6;
I2C_CR1_NOSTRETCH : constant := 7;
I2C_CR1_START   : constant := 8;
I2C_CR1_STOP    : constant := 9;
I2C_CR1_ACK     : constant := 10;
I2C_CR1_POS     : constant := 11;
I2C_CR1_PEC     : constant := 12;
I2C_CR1_ALERT   : constant := 13;
```

```

--Bit 14 reserved
I2C_CR1_SWRST      : constant := 15;

I2C_CR2      : constant := 16#04#;
I2C_CR2_FREQ      : constant := 0;
-- bit 6 reserved
-- bit 7 reserved
I2C_CR2_ITERREN    : constant := 8;
I2C_CR2_ITEVTEN    : constant := 9;
I2C_CR2_ITBUFEN    : constant := 10;
I2C_CR2_DMAEN      : constant := 11;
I2C_CR2_LAST       : constant := 12;
--Bit 15:13 reserved

I2C_SR1 : constant := 16#14#;
I2C_SR1_SB      : constant := 0;
I2C_SR1_ADDR    : constant := 1;
I2C_SR1_BTF     : constant := 2;
I2C_SR1_ADD10   : constant := 3;
I2C_SR1_STOPF   : constant := 4;
-- bit 5 reserved
I2C_SR1_RxNE    : constant := 6;
I2C_SR1_TxE     : constant := 7;
I2C_SR1_BERR    : constant := 8;
I2C_SR1_ARLO    : constant := 9;
I2C_SR1_AF      : constant := 10;
I2C_SR1_OVR     : constant := 11;
I2C_SR1_PECERR  : constant := 12;
-- bit 13 reserved
I2C_SR1_TIMEOUT : constant := 14;
I2C_SR1_SMBALERT : constant := 15;

I2C_SR2 : constant := 16#18#;
I2C_SR2_MSL     : constant := 0;
I2C_SR2_BUSY    : constant := 1;
I2C_SR2_TRA     : constant := 2;
-- bit 3 reserved
I2C_SR2_GENCALL : constant := 4;
I2C_SR2_SMBDEFAULT : constant := 5;
I2C_SR2_SMBHOST : constant := 6;
I2C_SR2_DUALF   : constant := 7;
-- bits 15:8 PEC

```



```
I2C_CCR : constant := 16#1C#;
    I2C_CCR_CCR : constant := 0;
    -- bits 13:12 reserved
    I2C_CCR_DUTY : constant := 14;
    I2C_CCR_FS : constant := 15;

I2C_TRISE : constant := 16#20#;
    I2C_TRISE_TRISE : constant := 0;
    -- reservado hasta 15

-- Registros ADC
ADC_SR_Offset : constant := 16#00#;
    ADC_SR_AWD:constant:=0;
    ADC_SR_EOC:constant:= 1;
    ADC_SR_JEOC: constant := 2;
    ADC_SR_JSTRT:constant := 3;
    ADC_SR_STRT:constant := 4;
    ADC_SR_OVR:constant :=5;
ADC_CR1_Offset : constant := 16#04#;
    ADC_CR1_AWDCH:constant :=0;
    ADC_CR1_EOCIE : constant := 5;
    ADC_CR1_AWDIE :constant:=6;
    ADC_CR1_JEOCIE : constant := 7;
    ADC_CR1_SCAN : constant := 8;
    ADC_CR1_AWDSDL:constant:=9;
    ADC_CR1_DISCEN:constant:=11;
    ADC_CR1_JDISCEN:constant:=12;
    ADC_CR1_DISCNUM:constant:=13;
    ADC_CR1_JAWDEN : constant := 22;
    ADC_CR1_AWDEN : constant := 23;
    ADC_CR1_RES : constant := 24;
    ADC_CR1_OVRIE: constant:=26;
ADC_CR2_Offset : constant := 16#08#;
    ADC_CR2_ADON : constant := 0;
    ADC_CR2_CONT : constant := 1;
    ADC_CR2_DMA : constant := 8;
    ADC_CR2_DDS : constant := 9;
    ADC_CR2_EOCS : constant := 10;
    ADC_CR2_ALIGN : constant := 11;
```

```

ADC_CR2_JEXTSEL : constant := 16;
ADC_CR2_JEXTEN : constant := 20;
ADC_CR2_JSWSTART : constant := 22;
ADC_CR2_EXTSEL : constant := 24;
ADC_CR2_EXTEN : constant := 28;
ADC_CR2_SWSTART : constant := 30;
ADC_SMPR1_Offset : constant := 16#0C#;
ADC_SMPR2_Offset : constant := 16#10#;    -- 3BITS 0-9
ADC_JOFR1_Offset : constant := 16#14#;
ADC_JOFR2_Offset : constant := 16#18#;
ADC_JOFR3_Offset : constant := 16#1C#;
ADC_JOFR4_Offset : constant := 16#20#;
ADC_HTR_Offset : constant := 16#24#;    -- 11 BITS
ADC_LTR_Offset : constant := 16#28#;    -- 11BITS
ADC_SQR1_Offset : constant := 16#2C#;    -- 23BITS DE 5 EN 5
ADC_SQR2_Offset : constant := 16#30#;    -- 29 DE 5 EN 5
ADC_SQR3_Offset : constant := 16#34#;    -- 29
ADC_JSQR_Offset : constant := 16#38#;    -- 22 DE 5 EN 5
ADC_JDR1_Offset : constant := 16#3C#;    -- 15 BITS
ADC_JDR2_Offset : constant := 16#40#;
ADC_JDR3_Offset : constant := 16#44#;
ADC_JDR4_Offset : constant := 16#48#;
ADC_DR_Offset : constant := 16#4C#;

PCLK1_MHz : constant := 42;
I2C_AF : constant := 4;
I2C_SPEED : constant := 100_000;

-- SPI

-- Bits de CR1
SPI_CR1_CPHA : constant := 0;
SPI_CR1_CPOL : constant := 1;
SPI_CR1_MSTR : constant := 2;
SPI_CR1_BR0 : constant := 3;    -- Baudrate bits [5:3]
SPI_CR1_BR1 : constant := 4;
SPI_CR1_BR2 : constant := 5;
SPI_CR1_SPE : constant := 6;
SPI_CR1_LSBFIRST : constant := 7;
SPI_CR1_SSI : constant := 8;
SPI_CR1_SSM : constant := 9;
SPI_CR1_DFF : constant := 11;

-- Bits de SR

```

```
SPI_SR_RXNE      : constant := 0;
SPI_SR_TXE       : constant := 1;
SPI_SR_BSY       : constant := 7;

SPI1_EN: constant:=12;
SPI2_EN: constant:=14;
SPI3_EN: constant:=15;
SPI4_EN: constant:=13;

TIM1_Base  : constant := 16#4001_0000#;
TIM2_Base  : constant := 16#4000_0000#;
TIM3_Base  : constant := 16#4000_0400#;
TIM4_Base  : constant := 16#4000_0800#;
TIM5_Base  : constant := 16#4000_0C00#;
TIM6_Base  : constant := 16#4000_1000#;
TIM7_Base  : constant := 16#4000_1400#;
TIM8_Base  : constant := 16#4001_0400#;
TIM9_Base  : constant := 16#4001_4000#;
TIM10_Base : constant := 16#4001_4400#;
TIM11_Base : constant := 16#4001_4800#;
TIM12_Base : constant := 16#4000_1800#;
TIM13_Base : constant := 16#4000_1C00#;
TIM14_Base : constant := 16#4000_2000#;

TIM_CR1_Offset : constant := 16#00#;
  CR1_CEN      : constant := 0;
  CR1_UDIS     : constant := 1;
  CR1_URS      : constant := 2;
  CR1_OPM      : constant := 3;
  CR1_DIR      : constant := 4;
  CR1_CMS1     : constant := 5;
  CR1_CMS2     : constant := 6;
  CR1_ARPE     : constant := 7;

TIM_CR2_Offset : constant := 16#04#;
TIM_SMCR_Offset : constant := 16#08#;

TIM_DIER_Offset : constant := 16#0C#;
  DIER_UIE     : constant := 0;
  DIER_CC1IE   : constant := 1;
  DIER_CC2IE   : constant := 2;
  DIER_CC3IE   : constant := 3;
  DIER_CC4IE   : constant := 4;
```

```
TIM_SR_Offset      : constant := 16#10#;
  SR_UIF           : constant := 0;
  SR_CC1IF         : constant := 1;
  SR_CC2IF         : constant := 2;
  SR_CC3IF         : constant := 3;
  SR_CC4IF         : constant := 4;

TIM_EGR_Offset     : constant := 16#14#;
  EGR_UG           : constant := 0;

TIM_CCMR1_Offset   : constant := 16#18#;
TIM_CCMR2_Offset   : constant := 16#1C#;

TIM_CCER_Offset    : constant := 16#20#;
  CCER_CC1E        : constant := 0;
  CCER_CC1P        : constant := 1;
  CCER_CC2E        : constant := 4;
  CCER_CC2P        : constant := 5;
  CCER_CC3E        : constant := 8;
  CCER_CC3P        : constant := 9;
  CCER_CC4E        : constant := 12;
  CCER_CC4P        : constant := 13;

TIM_CNT_Offset     : constant := 16#24#;
TIM_PSC_Offset     : constant := 16#28#;
TIM_ARR_Offset     : constant := 16#2C#;

TIM_CCR1_Offset    : constant := 16#34#;
TIM_CCR2_Offset    : constant := 16#38#;
TIM_CCR3_Offset    : constant := 16#3C#;
TIM_CCR4_Offset    : constant := 16#40#;

TIM_BDTR_Offset    : constant := 16#44#;
  BDTR_MOE         : constant := 15;

end stm32f446;
```

Listing 7: stm32f446.ads

Paquete GPIO

```

with stm32f446; use stm32f446;
with System;

package GPIO is

    type GPIO_Port is
        (PORT_A, PORT_B, PORT_C, PORT_D,
         PORT_E, PORT_F, PORT_G, PORT_H);

    subtype Pin_Number is Integer range 0 .. 15;

    type GPIO_Point is record
        Port : GPIO_Port;
        Pin  : Pin_Number;
    end record;

    type GPIO_Pull is (No_Pull, Pull_Up, Pull_Down);
    procedure Config_Analog(GPIO_P:GPIO_Point;
                           Pull :GPIO_Pull :=No_Pull);

    procedure Config_Output (GPIO_P : GPIO_Point);
    procedure Config_Input  (GPIO_P : GPIO_Point;
                           Pull   : GPIO_Pull := No_Pull);

    procedure Set      (GPIO_P : GPIO_Point);
    procedure Clear    (GPIO_P : GPIO_Point);
    procedure Toggle   (GPIO_P : GPIO_Point);
    function  Read     (GPIO_P : GPIO_Point) return Boolean;

    RCC_AHB1ENR : Uint32 with
        Volatile,
        Address => System'To_Address (RCC + AHB1ENR_A);

end GPIO;

```

Listing 8: gpio.ads — Especificación del paquete GPIO

```

with stm32f446; use stm32f446;
with System;
with System.Storage_Elements; use System.Storage_Elements;
with System.Address_To_Access_Conversions;

```

```
package body GPIO is

    --conversion de punteros
    package Conv is new System.Address_To_Access_Conversions (Uint32);

function Bit (N : Natural) return Uint32 is
begin
    if N > 31 then
        raise Constraint_Error;
    end if;

    return Uint32 (2 ** N);
end Bit;

function Bit2_Mask (N : Natural) return Uint32 is
begin
    return Uint32 (3) * Uint32 (2 ** (2 * N));
end Bit2_Mask;

-- Valor V colocado en el campo de 2 bits en posición N
function Bit2_Val (N : Natural; V : Uint32) return Uint32 is
begin
    return (V and 3) * Uint32 (2 ** (2 * N));
end Bit2_Val;

function Port_Base (P : GPIO_Port) return Storage_Offset is
begin
    case P is
        when PORT_A => return Storage_Offset (GPIOA);
        when PORT_B => return Storage_Offset (GPIOB);
        when PORT_C => return Storage_Offset (GPIOC);
        when PORT_D => return Storage_Offset (GPIOD);
        when PORT_E => return Storage_Offset (GPIOE);
        when PORT_F => return Storage_Offset (GPIOF);
        when PORT_G => return Storage_Offset (GPIOG);
        when PORT_H => return Storage_Offset (GPIOH);
    end case;
end Port_Base;
```

```
function Reg (Base      : Storage_Offset;
              Offset    : Storage_Offset) return access Uint32 is
begin
    return Conv.To_Pointer (System.To_Address (Base + Offset));
end Reg;

function MODER (P : GPIO_Port) return access Uint32 is
begin
    return Reg (Port_Base (P), Storage_Offset (GPIO_MODER_Offset));
end MODER;

function PUPDR (P : GPIO_Port) return access Uint32 is
begin
    return Reg (Port_Base (P), Storage_Offset (GPIO_PUPDR_Offset));
end PUPDR;

function ODR (P : GPIO_Port) return access Uint32 is
begin
    return Reg (Port_Base (P), Storage_Offset (GPIO_ODR_Offset));
end ODR;

function IDR (P : GPIO_Port) return access Uint32 is
begin
    return Reg (Port_Base (P), Storage_Offset (GPIO_IDR_Offset));
end IDR;

function BSRR (P : GPIO_Port) return access Uint32 is
begin
    return Reg (Port_Base (P), Storage_Offset (GPIO_BSRR_Offset));
end BSRR;

procedure Enable_Clock (P : GPIO_Port) is
begin
    case P is
        when PORT_A => RCC_AHB1ENR := RCC_AHB1ENR or Bit (0);
        when PORT_B => RCC_AHB1ENR := RCC_AHB1ENR or Bit (1);
        when PORT_C => RCC_AHB1ENR := RCC_AHB1ENR or Bit (2);
        when PORT_D => RCC_AHB1ENR := RCC_AHB1ENR or Bit (3);
        when PORT_E => RCC_AHB1ENR := RCC_AHB1ENR or Bit (4);
        when PORT_F => RCC_AHB1ENR := RCC_AHB1ENR or Bit (5);
        when PORT_G => RCC_AHB1ENR := RCC_AHB1ENR or Bit (6);
        when PORT_H => RCC_AHB1ENR := RCC_AHB1ENR or Bit (7);
```

```
        end case;
    end Enable_Clock;

    procedure Config_Output (GPIO_P : GPIO_Point) is
        P      : constant GPIO_Port := GPIO_P.Port;
        Pin    : constant Natural   := GPIO_P.Pin;
    begin
        Enable_Clock (P);
        MODER (P).all :=
            (MODER (P).all and not Bit2_Mask (Pin))
            or Bit2_Val (Pin, 1);
    end Config_Output;

    procedure Config_Analog (GPIO_P : GPIO_Point
        ;Pull:GPIO_Pull:=No_Pull) is
        P      : constant GPIO_Port := GPIO_P.Port;
        Pin    : constant Natural   := GPIO_P.Pin;
        Val    : Uint32;
    begin
        Enable_Clock (P);
        MODER (P).all :=
            (MODER (P).all and not Bit2_Mask (Pin))
            or Bit2_Val (Pin, 3);

        case Pull is
            when No_Pull    => Val := Bit2_Val (Pin, 0);
            when Pull_Up    => Val := Bit2_Val (Pin, 1);
            when Pull_Down  => Val := Bit2_Val (Pin, 2);
        end case;
        PUPDR (P).all :=
            (PUPDR (P).all and not Bit2_Mask (Pin)) or Val;
    end Config_Analog;

    procedure Config_Input (GPIO_P : GPIO_Point;
        Pull : GPIO_Pull := No_Pull) is
        P      : constant GPIO_Port := GPIO_P.Port;
        Pin    : constant Natural   := GPIO_P.Pin;
        Val    : Uint32;
    begin
```



```
Enable_Clock (P);

MODER (P).all := MODER (P).all and not Bit2_Mask (Pin);

case Pull is
    when No_Pull    => Val := Bit2_Val (Pin, 0);
    when Pull_Up    => Val := Bit2_Val (Pin, 1);
    when Pull_Down  => Val := Bit2_Val (Pin, 2);
end case;
PUPDR (P).all :=
    (PUPDR (P).all and not Bit2_Mask (Pin)) or Val;
end Config_Input;

procedure Set (GPIO_P : GPIO_Point) is
begin
    BSRR (GPIO_P.Port).all := 2**GPIO_P.Pin;
end Set;

procedure Clear (GPIO_P : GPIO_Point) is
begin
    BSRR (GPIO_P.Port).all := 2**(GPIO_P.Pin+16);
end Clear;

procedure Toggle (GPIO_P : GPIO_Point) is
begin
    if (ODR (GPIO_P.Port).all and Bit (GPIO_P.Pin)) /= 0 then
        Clear (GPIO_P);
    else
        Set (GPIO_P);
    end if;
end Toggle;

function Read (GPIO_P : GPIO_Point) return Boolean is
begin
    return (IDR (GPIO_P.Port).all and Bit (GPIO_P.Pin)) /= 0;
end Read;
```

```
end GPIO;
```

Listing 9: gpio.adb — Cuerpo del paquete GPIO

Paquete ADC

```
with stm32f446; use stm32f446;

package ADC is

    subtype ADC_Channel      is Natural range 0 .. 18;
    subtype ADC_Channel_Regular is Natural range 0 .. 16;

    ADC_CH_Temp      : constant ADC_Channel := 18;
    ADC_CH_Vrefint    : constant ADC_Channel := 17;
    ADC_CH_Vbat       : constant ADC_Channel := 18;

    type ADC_Resolution is
        (Res_12bit,
         Res_10bit,
         Res_8bit,
         Res_6bit);

    type ADC_Alignment is (Right, Left);

    type ADC_Instance is (ADC_1, ADC_2, ADC_3);

    type ADC_Mode is
        (Single,
         Continuous);

    procedure Init (ADC : ADC_Instance;
                   Res  : ADC_Resolution := Res_12bit;
                   Align: ADC_Alignment  := Right;
                   Mode : ADC_Mode       := Single);

    function Read      (ADC : ADC_Instance;
                       Ch   : ADC_Channel) return Uint32;

    function Read_mV   (ADC : ADC_Instance;
                       Ch   : ADC_Channel) return Natural;

    procedure Start_Continuous (ADC : ADC_Instance;
                               Ch   : ADC_Channel);

    function Read_Continuous (ADC : ADC_Instance) return Uint32;

    procedure Stop_Continuous (ADC : ADC_Instance);
```

```

    procedure Enable_Temp_Sensor;
    function Read_Temperature return Integer;
    procedure Enable_Vrefint;
    function Read_Vrefint_mV return Natural;

end ADC;

```

Listing 10: adc.ads — Especificación del paquete ADC

```

with stm32f446;                                use stm32f446;
with System;
with System.Storage_Elements;                  use System.Storage_Elements;
with System.Address_To_Access_Conversions;

package body ADC is

    package Conv is new System.Address_To_Access_Conversions (Uint32);

    Bit_Table : constant array (0 .. 31) of Uint32 :=
        (0 => 16#0000_0001#, 1 => 16#0000_0002#,
         2 => 16#0000_0004#, 3 => 16#0000_0008#,
         4 => 16#0000_0010#, 5 => 16#0000_0020#,
         6 => 16#0000_0040#, 7 => 16#0000_0080#,
         8 => 16#0000_0100#, 9 => 16#0000_0200#,
         10 => 16#0000_0400#, 11 => 16#0000_0800#,
         12 => 16#0000_1000#, 13 => 16#0000_2000#,
         14 => 16#0000_4000#, 15 => 16#0000_8000#,
         16 => 16#0001_0000#, 17 => 16#0002_0000#,
         18 => 16#0004_0000#, 19 => 16#0008_0000#,
         20 => 16#0010_0000#, 21 => 16#0020_0000#,
         22 => 16#0040_0000#, 23 => 16#0080_0000#,
         24 => 16#0100_0000#, 25 => 16#0200_0000#,
         26 => 16#0400_0000#, 27 => 16#0800_0000#,
         28 => 16#1000_0000#, 29 => 16#2000_0000#,
         30 => 16#4000_0000#, 31 => 16#8000_0000#);

    function Bit (N : Natural) return Uint32 is
    begin
        return Bit_Table (N);
    end Bit;

    function Reg (Base : Storage_Offset;

```

```
        Offset : Storage_Offset) return access Uint32 is
begin
    return Conv.To_Pointer (System'To_Address (Base + Offset));
end Reg;

function ADC_Base (A : ADC_Instance) return Storage_Offset is
begin
    case A is
        when ADC_1 => return Storage_Offset (ADC1_Base);
        when ADC_2 => return Storage_Offset (ADC2_Base);
        when ADC_3 => return Storage_Offset (ADC3_Base);
    end case;
end ADC_Base;

function SR      (A : ADC_Instance) return access Uint32 is
begin return Reg (ADC_Base (A), Storage_Offset (ADC_SR_Offset));
end SR;

function CR1     (A : ADC_Instance) return access Uint32 is
begin return Reg (ADC_Base (A), Storage_Offset (ADC_CR1_Offset));
end CR1;

function CR2     (A : ADC_Instance) return access Uint32 is
begin return Reg (ADC_Base (A), Storage_Offset (ADC_CR2_Offset));
end CR2;

function SMPR1   (A : ADC_Instance) return access Uint32 is
begin return Reg (ADC_Base (A), Storage_Offset
    (ADC_SMPR1_Offset)); end SMPR1;
function SMPR2   (A : ADC_Instance) return access Uint32 is
begin return Reg (ADC_Base (A), Storage_Offset
    (ADC_SMPR2_Offset)); end SMPR2;
function SQR1    (A : ADC_Instance) return access Uint32 is
begin return Reg (ADC_Base (A), Storage_Offset (ADC_SQR1_Offset));
end SQR1;
function SQR3    (A : ADC_Instance) return access Uint32 is
begin return Reg (ADC_Base (A), Storage_Offset (ADC_SQR3_Offset));
end SQR3;
function DR      (A : ADC_Instance) return access Uint32 is
begin return Reg (ADC_Base (A), Storage_Offset (ADC_DR_Offset));
end DR;

function CCR return access Uint32 is
begin
```

```

        return Reg (Storage_Offset (ADC_Common_Registers),
                    Storage_Offset (ADC_CCR_Offset));
end CCR;

procedure Enable_ADC_Clock (A : ADC_Instance) is
    RCC_APB2ENR : constant access Uint32 :=
        Reg (Storage_Offset (RCC), Storage_Offset (APB2ENR_A));
begin
    case A is
        when ADC_1 => RCC_APB2ENR.all := RCC_APB2ENR.all or Bit
            (ADC1EN);
        when ADC_2 => RCC_APB2ENR.all := RCC_APB2ENR.all or Bit
            (ADC2EN);
        when ADC_3 => RCC_APB2ENR.all := RCC_APB2ENR.all or Bit
            (ADC3EN);
    end case;
end Enable_ADC_Clock;

procedure Set_Sample_Time (A : ADC_Instance; Ch : ADC_Channel) is
    Val : constant Uint32 := 7;
    Shift : Natural;
    Mask : Uint32;
begin
    if Ch <= 9 then
        Shift := Ch * 3;
        Mask := Bit (Shift) or Bit (Shift + 1) or Bit (Shift + 2);
        SMPR2 (A).all := (SMPR2 (A).all and not Mask)
            or (Val * Bit (Shift));
    else
        Shift := (Ch - 10) * 3;
        Mask := Bit (Shift) or Bit (Shift + 1) or Bit (Shift + 2);
        SMPR1 (A).all := (SMPR1 (A).all and not Mask)
            or (Val * Bit (Shift));
    end if;
end Set_Sample_Time;

procedure Set_Channel (A : ADC_Instance; Ch : ADC_Channel) is
    Ch_Mask : constant Uint32 :=
        Bit (0) or Bit (1) or Bit (2) or Bit (3) or Bit (4);
begin
    Set_Sample_Time (A, Ch);
    SQR3 (A).all := (SQR3 (A).all and not Ch_Mask) or Uint32 (Ch);
end Set_Channel;

```

```
procedure Init (ADC : ADC_Instance;
               Res : ADC_Resolution := Res_12bit;
               Align: ADC_Alignment := Right;
               Mode : ADC_Mode      := Single) is
    Res_Val : Uint32;
begin
    Enable_ADC_Clock (ADC);

    CR2 (ADC).all := CR2 (ADC).all and not Bit (ADC_CR2_ADON);

    case Res is
        when Res_12bit => Res_Val := 0;
        when Res_10bit => Res_Val := 1;
        when Res_8bit  => Res_Val := 2;
        when Res_6bit  => Res_Val := 3;
    end case;

    CR1 (ADC).all :=
        (CR1 (ADC).all
         and not (Bit (ADC_CR1_RES) or Bit (ADC_CR1_RES + 1))
         and not Bit (ADC_CR1_SCAN))
        or (Res_Val * Bit (ADC_CR1_RES));

    if Align = Left then
        CR2 (ADC).all := CR2 (ADC).all or Bit (ADC_CR2_ALIGN);
    else
        CR2 (ADC).all := CR2 (ADC).all and not Bit (ADC_CR2_ALIGN);
    end if;

    if Mode = Continuous then
        CR2 (ADC).all := CR2 (ADC).all or Bit (ADC_CR2_CONT);
    else
        CR2 (ADC).all := CR2 (ADC).all and not Bit (ADC_CR2_CONT);
    end if;

    CR2 (ADC).all := CR2 (ADC).all and not Bit (ADC_CR2_DMA);
```

```
SQR1 (ADC).all :=
    SQR1 (ADC).all
    and not (Bit (20) or Bit (21) or Bit (22) or Bit (23));

CR2 (ADC).all := CR2 (ADC).all or Bit (ADC_CR2_ADON);

for I in 1 .. 1000 loop
    null;
end loop;

end Init;

function Read (ADC : ADC_Instance;
               Ch  : ADC_Channel) return Uint32 is
begin
    Set_Channel (ADC, Ch);

    SR (ADC).all := SR (ADC).all and not Bit (ADC_SR_EOC);

    CR2 (ADC).all := CR2 (ADC).all or Bit (ADC_CR2_SWSTART);

    while (SR (ADC).all and Bit (ADC_SR_EOC)) = 0 loop
        null;
    end loop;

    return DR (ADC).all and 16#0FFF#;
end Read;

function Read_mV (ADC : ADC_Instance;
                  Ch  : ADC_Channel) return Natural is
    Raw : constant Uint32 := Read (ADC, Ch);
begin
    return Natural ((Raw * 3300) / 4095);
end Read_mV;
```



```
procedure Start_Continuous (ADC : ADC_Instance;  
                           Ch   : ADC_Channel) is  
begin  
    CR2 (ADC).all := CR2 (ADC).all or Bit (ADC_CR2_CONT);  
  
    Set_Channel (ADC, Ch);  
  
    SR (ADC).all := SR (ADC).all and not Bit (ADC_SR_EOC);  
  
    CR2 (ADC).all := CR2 (ADC).all or Bit (ADC_CR2_SWSTART);  
end Start_Continuous;  
  
function Read_Continuous (ADC : ADC_Instance) return Uint32 is  
begin  
    return DR (ADC).all and 16#0FFF#;  
end Read_Continuous;  
  
procedure Stop_Continuous (ADC : ADC_Instance) is  
begin  
    CR2 (ADC).all := CR2 (ADC).all and not Bit (ADC_CR2_CONT);  
    CR2 (ADC).all := CR2 (ADC).all and not Bit (ADC_CR2_ADON);  
end Stop_Continuous;  
  
CCR_TSVREFE : constant := 23;  
CCR_VBATE   : constant := 22;  
  
procedure Enable_Temp_Sensor is  
begin  
    CCR.all := CCR.all or Bit (CCR_TSVREFE);  
    for I in 1 .. 10_000 loop  
        null;  
    end loop;  
end Enable_Temp_Sensor;
```

```
function Read_Temperature return Integer is
    Raw      : constant Uint32 := Read (ADC_1, ADC_CH_Temp);
    V25      : constant Integer := 943;
    Raw_Int  : constant Integer := Integer (Raw);
    begin
        return ((V25 - Raw_Int) * 1000) / 31 + 2500;
    end Read_Temperature;

procedure Enable_Vrefint is
begin
    CCR.all := CCR.all or Bit (CCR_TSVREFE);
    for I in 1 .. 10_000 loop
        null;
    end loop;
end Enable_Vrefint;

function Read_Vrefint_mV return Natural is
    Raw : constant Uint32 := Read (ADC_1, ADC_CH_Vrefint);
begin
    return Natural ((Raw * 3300) / 4095);
end Read_Vrefint_mV;

end ADC;
```

Listing 11: adc.adb — Cuerpo del paquete ADC

Paquete USART (Driver genérico)

```
with System;
with stm32f446; use stm32f446;

generic
    GPIO_RX: Uint32;
    GPIO_TX: Uint32;
    RX_PIN: Natural;
    TX_PIN: Natural;
    AF_RX: Natural;
    AF_TX: Natural;
    USART_Base: Uint32;

package USART is

    procedure Initialize (Baudrate : Uint32);

    procedure Send_Char (C : Uint8);
    procedure Send_String (S : String);
    procedure Send_Line (S : String);
    function Read_Char return Uint8;
    function Data_Available return Boolean;
private

    function GPIO_To_AHB1_Bit (Base : Uint32) return Uint32 is
    (case Base is
        when GPIOA => 2**GPIOAEN,
        when GPIOB => 2**GPIOBEN,
        when GPIOC => 2**GPIOCEN,
        when GPIOD => 2**GPIODEN,
        when GPIOE => 2**GPIOEEN,
        when GPIOF => 2**GPIOFEN,
        when GPIOG => 2**GPIOGEN,
        when GPIOH => 2**GPIOHEN,
        when others => 0);

    function USART_To_APB_Bit (Base : Uint32) return Uint32 is
    (case Base is
        when USART1_Base => 2**USART1EN,
```

```

    when USART2_Base => 2**USART2EN,
    when USART3_Base => 2**USART3EN,
    when UART4_Base => 2**USART4EN,
    when UART5_Base => 2**USART5EN,
    when USART6_Base => 2**USART6EN,
    when others      => 0);

AHB1_TX_Bit : constant Uint32 := GPIO_To_AHB1_Bit (GPIO_TX);
AHB1_RX_Bit : constant Uint32 := GPIO_To_AHB1_Bit (GPIO_RX);
APB_USART_Bit : constant Uint32 := USART_To_APB_Bit
    (USART_Base);

RCC_AHB1ENR : Uint32 with
    Volatile,
    Address => System'To_Address (RCC + AHB1ENR_A);

RCC_APB2ENR: Uint32 with
    Volatile,
    Address => System'To_Address(RCC+APB2ENR_A);

RCC_APB1ENR : Uint32 with
    Volatile,
    Address => System'To_Address (RCC + APB1ENR_A);

GPIO_MODER_TX : Uint32 with
    Volatile,
    Address => System'To_Address (GPIO_TX + GPIO_MODER_Offset);

GPIO_MODER_RX : Uint32 with
    Volatile,
    Address => System'To_Address (GPIO_RX + GPIO_MODER_Offset);

GPIO_AFRL_RX : Uint32 with
    Volatile,
    Address => System'To_Address (GPIO_RX + GPIO_AFRL_Offset);

GPIO_AFRL_TX : Uint32 with
    Volatile,
    Address => System'To_Address (GPIO_TX + GPIO_AFRL_Offset);

GPIO_AFRH_RX : Uint32 with

```

```
    Volatile,
    Address => System'To_Address (GPIO_RX + GPIO_AFRH_Offset);

GPIO_AFRH_TX : Uint32 with
    Volatile,
    Address => System'To_Address (GPIO_TX + GPIO_AFRH_Offset);

GPIO_PUPDR_RX : Uint32 with
    Volatile,
    Address => System'To_Address (GPIO_RX + GPIO_PUPDR_Offset);

GPIO_PUPDR_TX : Uint32 with
    Volatile,
    Address => System'To_Address (GPIO_TX + GPIO_PUPDR_Offset);

USART_SR : Uint32 with
    Volatile,
    Address => System'To_Address (USART_Base + USART_SR_Offset);

USART_DR : Uint32 with
    Volatile,
    Address => System'To_Address (USART_Base + USART_DR_Offset);

USART_BRR : Uint32 with
    Volatile,
    Address => System'To_Address (USART_Base + USART_BRR_Offset);

USART_CR1 : Uint32 with
    Volatile,
    Address => System'To_Address (USART_Base + USART_CR1_Offset);

USART_CR2 : Uint32 with
    Volatile,
    Address => System'To_Address (USART_Base + USART_CR2_Offset);

USART_CR3 : Uint32 with
    Volatile,
    Address => System'To_Address (USART_Base + USART_CR3_Offset);

end USART;
```

Listing 12: usart.ads — Especificación del driver USART

```

with System;
with Ada.Real_Time; use Ada.Real_Time;
with stm32f446; use stm32f446;
package body USART is

  procedure Delay_Loop (Cycles : Uint32) is
    C : Uint32 := Cycles;
  begin
    while C > 0 loop
      C := C - 1;
    end loop;
  end Delay_Loop;

  procedure Initialize (Baudrate : Uint32) is
    PCLK1 : constant Uint32 := 42_000_000;
    Div: Uint32;
  begin
    RCC_AHB1ENR := RCC_AHB1ENR or AHB1_RX_Bit or AHB1_TX_Bit;

    case USART_Base is
      when USART1_Base=>
        RCC_APB2ENR:= RCC_APB2ENR or
          APB_USART_Bit;

        when USART6_Base=>RCC_APB2ENR:= RCC_APB2ENR or
          APB_USART_Bit;
      when others => RCC_APB1ENR := RCC_APB1ENR or APB_USART_Bit;
    end case;

    delay(0.001);
    GPIO_MODER_TX := GPIO_MODER_TX and not (3 * 4**(TX_PIN));
    GPIO_MODER_TX := GPIO_MODER_TX or (2 * 4**(TX_PIN));

    GPIO_MODER_RX := GPIO_MODER_RX and not (3 * 4**(RX_PIN));
    GPIO_MODER_RX := GPIO_MODER_RX or (2 * 4**(RX_PIN));
    if TX_PIN < 8 then
      GPIO_AFRL_TX := (GPIO_AFRL_TX
and not(Uint32(2#1111# * (2**(4*TX_PIN))))) or Uint32(AF_TX *
      (2**(4*(TX_PIN))));

```

```

else
    GPIO_AFRH_TX := (GPIO_AFRH_TX
    and not(Uint32(2#1111# * (2**(4*(TX_PIN-8))))) or Uint32(AF_TX*
        (2**(4*(TX_PIN-8))));
end if;

    if RX_PIN < 8 then
        GPIO_AFRL_RX := (GPIO_AFRL_RX
    and not(Uint32(2#1111# * (2**(4*(RX_PIN))))) or Uint32(AF_RX *
        (2**(4*(RX_PIN))));
    else
        GPIO_AFRH_RX := (GPIO_AFRH_RX
    and not(Uint32(2#1111# * (2**(4*(RX_PIN-8))))) or Uint32(AF_RX*
        (2**(4*(RX_PIN-8))));
    end if;

    GPIO_PUPDR_TX := GPIO_PUPDR_TX or ( 4**(TX_PIN ));
    GPIO_PUPDR_RX := GPIO_PUPDR_RX or ( 4**(RX_PIN));

    Div:=PCLK1 / Baudrate;
    USART_BRR := Div;
    USART_CR1 := (2**CR1_TE) or (2**CR1_RE) or (2**CR1_UE);  --
        TE, RE, UE

    USART_CR2 := 0;
    USART_CR3 := 0;

    delay(0.01);
end Initialize;

function Data_Available return Boolean is
begin

```

```
    return (USART_SR and (2**SR_RXNE)) /= 0;
end Data_Available;

function Read_Char return Uint8 is
begin
    while (USART_SR and (2**SR_RXNE)) = 0 loop
        null;
    end loop;
    return Uint8 (USART_DR and 16#FF#);
end Read_Char;

procedure Send_Char (C : Uint8) is
begin
    while (USART_SR and (2**SR_TXE)) = 0 loop
        null;
    end loop;
    USART_DR := Uint32 (C); --A 32 bits
                           0000_0000_0000_0000_0000_0000_1111_1111

    while (USART_SR and (2**SR_TC)) = 0 loop
        null;
    end loop;
end Send_Char;

procedure Send_String (S : String) is
begin
    for I in S'Range loop
        Send_Char (Character'Pos (S (I)));
    end loop;
end Send_String;

procedure Send_Line (S : String) is
begin
    Send_String (S);
    Send_Char (13);
    Send_Char (10);
end Send_Line;
```



```
end USART;
```

Listing 13: usart.adb — Cuerpo del driver USART

Paquete I2C

```
with System;
with stm32f446; use stm32f446;

generic
    I2C_Base : Uint32;
    SCL_PIN : Natural;
    SDA_PIN : Natural;
    GPIO_SCL: Uint32;
    GPIO_SDA: Uint32;

package I2C is

protected Bus is
    procedure Initialize;

    function Write (SlaveAddr : Uint8;
                   Data : Uint8) return Boolean;

    function Read (SlaveAddr : Uint8;
                  Data : out Uint8) return Boolean;

    function Write_Buffer (SlaveAddr : Uint8;
                           Data : Uint8_Array) return Boolean;

    function Read_Buffer (SlaveAddr : Uint8;
                           Data : out Uint8_Array) return Boolean;

    function Wait_Flag (Flag_Mask : Uint32; Timeout_MS : Positive)
        return Boolean;
    procedure Clear_Errors;
    procedure Test_Hardware;
    procedure Scan_I2C_Bus;
    procedure Test_Minimo;
    procedure Mostrar_Config;
private
    Initialized: Boolean:=false;

end Bus;

function GPIO_To_AHB1_Bit (Base : Uint32) return Uint32 is
```

```

(case Base is
  when GPIOA => 2**GPIOAEN,
  when GPIOB => 2**GPIOBEN,
  when GPIOC => 2**GPIOCEN,
  when GPIOD => 2**GPIODEN,
  when GPIOE => 2**GPIOEEN,
  when GPIOF => 2**GPIOFEN,
  when GPIOG => 2**GPIOGEN,
  when GPIOH => 2**GPIOHEN,
  when others => 0);

function I2C_To_APB1_Bit (Base : Uint32) return Uint32 is
(case Base is
  when I2C1_Base => 2**I2C1EN,
  when I2C2_Base => 2**I2C2EN,
  when I2C3_Base => 2**I2C3EN,
  when others    => 0);

AHB1_SCL_Bit : constant Uint32 := GPIO_To_AHB1_Bit (GPIO_SCL);
AHB1_SDA_Bit : constant Uint32 := GPIO_To_AHB1_Bit (GPIO_SDA);
APB1_I2C_Bit : constant Uint32 := I2C_To_APB1_Bit  (I2C_Base);

RCC_AHB1ENR : Uint32 with
  Volatile,
  Address => System'To_Address (RCC + AHB1ENR_A);

RCC_APB1ENR : Uint32 with
  Volatile,
  Address => System'To_Address (RCC + APB1ENR_A);

GPIO_MODER_SCL : Uint32 with
  Volatile,
  Address => System'To_Address (GPIO_SCL + GPIO_MODER_Offset);

GPIO_MODER_SDA : Uint32 with
  Volatile,
  Address => System'To_Address (GPIO_SDA + GPIO_MODER_Offset);

GPIO_AFRL_SCL : Uint32 with

```

```
    Volatile ,
    Address => System'To_Address (GPIO_SCL + GPIO_AFRL_Offset);

    GPIO_AFRL_SDA : Uint32 with
    Volatile ,
    Address => System'To_Address (GPIO_SDA + GPIO_AFRL_Offset);

GPIO_AFRH_SCL : Uint32 with
Volatile ,
Address => System'To_Address (GPIO_SCL + GPIO_AFRH_Offset);

GPIO_AFRH_SDA : Uint32 with
Volatile ,
Address => System'To_Address (GPIO_SDA + GPIO_AFRH_Offset);


GPIO_OTYPER_SCL : Uint32 with
Volatile ,
Address => System'To_Address (GPIO_SCL + GPIO_OTYPER_Offset);

GPIO_OTYPER_SDA : Uint32 with
Volatile ,
Address => System'To_Address (GPIO_SDA + GPIO_OTYPER_Offset);


GPIO_OSPEEDR_SCL : Uint32 with
Volatile ,
Address => System'To_Address (GPIO_SCL + GPIO_OTYPER_Offset);

GPIO_OSPEEDR_SDA : Uint32 with
Volatile ,
Address => System'To_Address (GPIO_SDA + GPIO_OTYPER_Offset);


GPIO_PUPDR_SCL : Uint32 with
Volatile ,
Address => System'To_Address (GPIO_SCL + GPIO_PUPDR_Offset);
GPIO_PUPDR_SDA : Uint32 with
Volatile ,
```

```
Address => System'To_Address (GPIO_SDA + GPIO_PUPDR_Offset);
```

```
GPIO_ODR_SCL : Uint32 with  
Volatile, Address => System'To_Address (GPIO_SCL+  
    GPIO_ODR_Offset);
```

```
GPIO_ODR_SDA : Uint32 with  
Volatile, Address => System'To_Address (GPIO_SDA+  
    GPIO_ODR_Offset);
```

```
-- Registros I2C
```

```
CR1 : Uint32 with  
Volatile,  
Address => System'To_Address (I2C_Base + I2C_CR1);
```

```
CR2 : Uint32 with  
Volatile,  
Address => System'To_Address (I2C_Base + I2C_CR2);
```

```
CCR : Uint32 with  
Volatile,  
Address => System'To_Address (I2C_Base + I2C_CCR);
```

```
TRISE : Uint32 with  
Volatile,  
Address => System'To_Address (I2C_Base + I2C_TRISE);
```

```
SR1 : Uint32 with  
Volatile,  
Address => System'To_Address (I2C_Base + I2C_SR1);
```

```
SR2 : Uint32 with  
Volatile,  
Address => System'To_Address (I2C_Base + I2C_SR2);
```

```
DR : Uint32 with  
Volatile,
```

```

        Address => System'To_Address (I2C_Base + I2C_DR);

end I2C;

```

Listing 14: i2c.ads — Especificación del driver I2C

```

with System;
with Ada.Real_Time; use Ada.Real_Time;
with stm32f446; use stm32f446;
with I2C; use I2C;
with USART;
with USART_Driver; use USART_Driver;

package body I2C is

    protected body Bus is

        procedure Initialize is
            tmp , tmp2 :Integer;
        begin

            if Initialized then
                return;
            end if;
            RCC_AHB1ENR := RCC_AHB1ENR or AHB1_SCL_Bit or AHB1_SDA_Bit;
            RCC_APB1ENR := RCC_APB1ENR or APB1_I2C_Bit;

            GPIO_MODER_SCL := (GPIO_MODER_SCL
                and not(Uint32(2**(2*SCL_Pin))) and
                not(Uint32(2**(2*(SCL_Pin+1)))));

            GPIO_MODER_SDA := (GPIO_MODER_SDA
                and not(Uint32(2**(2*SDA_Pin))) and
                not(Uint32(2**(2*(SDA_Pin+1)))));

            GPIO_MODER_SCL:=(GPIO_MODER_SCL or Uint32(2**(2*SCL_Pin)));
            GPIO_MODER_SDA:=(GPIO_MODER_SDA or Uint32(2**(2*SDA_Pin)));

```

```

-- Open-drain
GPIO_OTYPER_SCL := GPIO_OTYPER_SCL
    or Uint32(2**SCL_PIN);

GPIO_OTYPER_SCL := GPIO_OTYPER_SCL
    or Uint32(2**SDA_PIN);

-- Pull-up activo
GPIO_PUPDR_SCL := (GPIO_PUPDR_SCL
    and not(Uint32(2**((2*SCL_PIN)+1))));

GPIO_PUPDR_SDA := (GPIO_PUPDR_SDA
    and not(Uint32(2**((2*SDA_PIN)+1))));

GPIO_PUPDR_SCL := GPIO_PUPDR_SCL
    or Uint32(2**(2*SCL_PIN));

GPIO_PUPDR_SDA := GPIO_PUPDR_SDA
    or Uint32(2**(2*SDA_PIN));

-- Ambas líneas altas inicialmente
GPIO_ODR_SCL := GPIO_ODR_SCL or Uint32(2**SCL_PIN);
GPIO_ODR_SDA := GPIO_ODR_SDA or Uint32(2**SDA_PIN);
for J in 1 .. 500 loop null; end loop;

for I in 1 .. 9 loop
    GPIO_ODR_SCL := GPIO_ODR_SCL and not(Uint32(2**SCL_PIN)); --
        SCL bajo
    for J in 1 .. 500 loop null; end loop;
    GPIO_ODR_SCL := GPIO_ODR_SCL or Uint32(2**SCL_PIN); --
        SCL alto
    for J in 1 .. 500 loop null; end loop;
end loop;

GPIO_ODR_SDA := GPIO_ODR_SDA and not(Uint32(2**SDA_PIN)); -- SDA
    bajo
for J in 1 .. 500 loop null; end loop;
GPIO_ODR_SCL := GPIO_ODR_SCL or Uint32(2**SCL_PIN); --
    SCL alto
for J in 1 .. 500 loop null; end loop;

```

```

GPIO_ODR_SDA := GPIO_ODR_SDA or Uint32(2**SDA_PIN);           --
    SDA alto = STOP
for J in 1 .. 500 loop null; end loop;

CR1 := CR1 or Uint32(2**I2C_CR1_SWRST);
for I in 1 .. 200 loop null; end loop;
CR1 := CR1 and not(Uint32(2**I2C_CR1_SWRST));

GPIO_MODER_SCL := (GPIO_MODER_SCL
    and not(Uint32(4**SCL_PIN)) and not(Uint32((4**SCL_PIN)+1)));

GPIO_MODER_SDA := (GPIO_MODER_SDA
    and not(Uint32(4**SDA_PIN)) and not(Uint32((4**SDA_PIN)+1)));

tmp := (2*SCL_PIN)+1;
GPIO_MODER_SCL := (GPIO_MODER_SCL or Uint32(2**tmp));
tmp2 := (2*SDA_PIN)+1;
GPIO_MODER_SDA := (GPIO_MODER_SDA or Uint32(2**tmp2));

GPIO_OTYPER_SCL := GPIO_OTYPER_SCL
    or Uint32(2**SCL_PIN);

GPIO_OTYPER_SDA := GPIO_OTYPER_SDA
    or Uint32(2**SDA_PIN);

GPIO_OSPEEDR_SCL := (GPIO_OSPEEDR_SCL
    and not(Uint32(4**SCL_PIN)));

GPIO_OSPEEDR_SDA := (GPIO_OSPEEDR_SDA
    and not(Uint32(4**SDA_PIN)));

GPIO_OSPEEDR_SCL := (GPIO_OSPEEDR_SCL or Uint32(2**tmp));
GPIO_OSPEEDR_SDA := (GPIO_OSPEEDR_SDA or Uint32(2**tmp2));

GPIO_PUPDR_SCL := (GPIO_PUPDR_SCL
    and not(Uint32(2**((2*SCL_PIN)+1))));

```



```

GPIO_PUPDR_SDA := (GPIO_PUPDR_SDA
    and not(Uint32(2**((2*SDA_PIN)+1))));

GPIO_PUPDR_SCL := GPIO_PUPDR_SCL
    or Uint32(2**(2*SCL_PIN));

GPIO_PUPDR_SDA := GPIO_PUPDR_SDA
    or Uint32(2**(2*SDA_PIN));

if SCL_PIN < 8 then GPIO_AFRL_SCL := (GPIO_AFRL_SCL
    and not(Uint32(2#1111# * (2**(4*SCL_PIN))))) or Uint32(I2C_AF *
    (2**(4*(SCL_PIN))));
else
GPIO_AFRH_SCL := (GPIO_AFRH_SCL
    and not(Uint32(2#1111# * (2**(4*(SCL_PIN-8)))))) or
    Uint32(I2C_AF* (2**(4*(SCL_PIN-8))));
end if;

if SDA_PIN < 8 then GPIO_AFRL_SDA := (GPIO_AFRL_SDA
    and not(Uint32(2#1111# * (2**(4*SDA_PIN))))) or Uint32(I2C_AF *
    (2**(4*SDA_PIN)));
else
GPIO_AFRH_SDA := (GPIO_AFRH_SDA
    and not(Uint32(2#1111# * (2**(4*(SDA_PIN-8)))))) or
    Uint32(I2C_AF * (2**(4*(SDA_PIN-8))));
end if;

CR1 := CR1 and not(Uint32(2**I2C_CR1_PE));

CR2 := (CR2 and not(Uint32(16#3F#)))
    or Uint32(PCLK1_MHz);

tmp:= (PCLK1_MHz*10 )/ (2*(I2C_SPEED/100_000));

CCR := (CCR
    and not(Uint32(2**I2C_CCR_FS))
    and not(Uint32(2**I2C_CCR_DUTY))
    and not(Uint32(16#0FFF#)))
    or Uint32(tmp);

```

```
TRISE := (TRISE and not(Uint32(16#3F#)))
        or Uint32(PCLK1_MHz+1);

CR1 := CR1 or Uint32(2**I2C_CR1_PE);

for I in 1 .. 10_000 loop null; end loop;
if (SR2 and Uint32(2)) /= 0 then
    Send_Line ("WARN: bus sigue BUSY tras init");
else
    Send_Line ("OK: bus libre tras init");
end if;
Initialized:=true;

end Initialize;

procedure Clear_Errors is
    Tmp : Uint32;
begin
    Tmp := SR1;

    if (Tmp and Uint32 (2 ** I2C_SR1_AF)) /= 0 then
        SR1 := Tmp and not (Uint32 (2 ** I2C_SR1_AF));
    end if;

    if (Tmp and Uint32 (2 ** I2C_SR1_STOPF)) /= 0 then
        Tmp := SR1;
        CR1 := CR1;
    end if;

    if (Tmp and Uint32 (2 ** I2C_SR1_BERR)) /= 0 then
        SR1 := Tmp and not (Uint32 (2 ** I2C_SR1_BERR));
    end if;

    if (Tmp and Uint32 (2 ** I2C_SR1_ARLO)) /= 0 then
        SR1 := Tmp and not (Uint32 (2 ** I2C_SR1_ARLO));
    end if;

    if (Tmp and Uint32 (2 ** I2C_SR1_OVR)) /= 0 then
        Tmp := SR1;
```

```

        Tmp := DR;
    end if;
end Clear_Errors;

function Wait_Flag (Flag_Mask : Uint32;
                    Timeout_MS : Positive) return Boolean is
    Deadline : constant Time := Clock + Milliseconds
        (Timeout_MS);
begin
    loop
        if (SR1 and Flag_Mask) /= 0 then
            return True;
        end if;
        if (SR1 and Uint32 (2 ** I2C_SR1_AF)) /= 0 then
            SR1 := SR1 and not (Uint32 (2 ** I2C_SR1_AF));
            CR1 := CR1 or Uint32 (2 ** I2C_CR1_STOP);
            return False;
        end if;
        if (SR1 and Uint32 (2 ** I2C_SR1_BERR)) /= 0 then
            SR1 := SR1 and not (Uint32 (2 ** I2C_SR1_BERR));
            CR1 := CR1 or Uint32 (2 ** I2C_CR1_STOP);
            return False;
        end if;
        if (SR1 and Uint32 (2 ** I2C_SR1_ARLO)) /= 0 then
            SR1 := SR1 and not (Uint32 (2 ** I2C_SR1_ARLO));
            CR1 := CR1 or Uint32 (2 ** I2C_CR1_STOP);
            return False;
        end if;
        if Clock > Deadline then
            CR1 := CR1 or Uint32 (2 ** I2C_CR1_STOP);
            return False;
        end if;
    end loop;
end Wait_Flag;

function Write (SlaveAddr : Uint8;
                Data : Uint8) return Boolean is
    Status : Uint32 with Volatile;
begin
    CR1 := CR1 or Uint32 (2 ** I2C_CR1_START);

    if not Wait_Flag (Uint32 (2 ** I2C_SR1_SB), 100) then

```

```

        Send_Line ("FAIL: SB timeout");
    return False;
end if;

DR := Uint32 (SlaveAddr) * 2;

if not Wait_Flag (Uint32 (2 ** I2C_SR1_ADDR), 100) then
    Send_Line ("FAIL: ADDR timeout");
    CR1 := CR1 or Uint32 (2 ** I2C_CR1_STOP);
    return False;
end if;

Status := SR1;
Status := SR2;

if not Wait_Flag (Uint32 (2 ** I2C_SR1_TXE), 100) then
    Send_Line ("FAIL: TXE timeout");
    CR1 := CR1 or Uint32 (2 ** I2C_CR1_STOP);
    return False;
end if;

DR := Uint32 (Data);

if not Wait_Flag (Uint32 (2 ** I2C_SR1_BTF), 100) then
    Send_Line ("FAIL: BTF timeout");
    CR1 := CR1 or Uint32 (2 ** I2C_CR1_STOP);
    return False;
end if;

CR1 := CR1 or Uint32 (2 ** I2C_CR1_STOP);
return True;
end Write;

function Write_Buffer (SlaveAddr : Uint8;
                       Data       : Uint8_Array) return
                       Boolean is
    Status : Uint32 with Volatile;
    Timeout : Integer;
begin
    SR1 := SR1
    and not (Uint32 (2 ** I2C_SR1_AF))
    and not (Uint32 (2 ** I2C_SR1_BERR))
    and not (Uint32 (2 ** I2C_SR1_ARLO));

```

```
Status := SR1;
Status := SR2;

Timeout := 100;
while (SR2 and Uint32 (2)) /= 0 loop
    Timeout := Timeout - 1;
    if Timeout = 0 then
        Send_Line ("FAIL: bus BUSY antes de START");
        return False;
    end if;
end loop;

if (CR1 and Uint32 (2 ** I2C_CR1_PE)) = 0 then
    Send_Line ("FAIL: PE no activo");
    return False;
end if;

CR1 := CR1 or Uint32 (2 ** I2C_CR1_START);

if not Wait_Flag (Uint32 (2 ** I2C_SR1_SB), 100) then
    Send_Line ("FAIL: SB timeout");
    return False;
end if;

Status := SR1;
DR      := Uint32 (SlaveAddr) * 2;

if not Wait_Flag (Uint32 (2 ** I2C_SR1_ADDR), 100) then
    if (SR1 and Uint32 (2 ** I2C_SR1_AF)) /= 0 then
        Send_Line ("FAIL: NACK en direccion");
        SR1 := SR1 and not (Uint32 (2 ** I2C_SR1_AF));
    else
        Send_Line ("FAIL: ADDR timeout");
    end if;
    CR1 := CR1 or Uint32 (2 ** I2C_CR1_STOP);
    return False;
end if;

Status := SR1;
Status := SR2;

for Byte of Data loop
    if not Wait_Flag (Uint32 (2 ** I2C_SR1_TXE), 100) then
        Send_Line ("FAIL: TXE timeout");
```

```

        CR1 := CR1 or Uint32 (2 ** I2C_CR1_STOP);
        return False;
    end if;
    if (SR1 and Uint32 (2 ** I2C_SR1_AF)) /= 0 then
        Send_Line ("FAIL: NACK durante envio");
        SR1 := SR1 and not (Uint32 (2 ** I2C_SR1_AF));
        CR1 := CR1 or Uint32 (2 ** I2C_CR1_STOP);
        return False;
    end if;
    DR := Uint32 (Byte);
end loop;

if not Wait_Flag (Uint32 (2 ** I2C_SR1_BTF), 100) then
    Send_Line ("FAIL: BTF timeout");
    CR1 := CR1 or Uint32 (2 ** I2C_CR1_STOP);
    return False;
end if;

CR1 := CR1 or Uint32 (2 ** I2C_CR1_STOP);
return True;
end Write_Buffer;

function Read (SlaveAddr : Uint8;
               Data       : out Uint8) return Boolean is
    Status : Uint32;
    Time    : constant Positive := 100;
begin
    CR1 := CR1 or Uint32 (2 ** I2C_CR1_ACK);
    CR1 := CR1 or Uint32 (2 ** I2C_CR1_START);

    if not Wait_Flag (Uint32 (2 ** I2C_SR1_SB), Time) then
        return False;
    end if;

    DR := (Uint32 (SlaveAddr) * 2) or 16#01#;    -- R/W=1

    if not Wait_Flag (Uint32 (2 ** I2C_SR1_ADDR), Time) then
        CR1 := CR1 or Uint32 (2 ** I2C_CR1_STOP);
        return False;
    end if;

    CR1 := CR1 and not (Uint32 (2 ** I2C_CR1_ACK));

```

```
Status := SR1;
Status := SR2;

if not Wait_Flag (Uint32 (2 ** I2C_SR1_RxNE), Time) then
    CR1 := CR1 or Uint32 (2 ** I2C_CR1_STOP);
    return False;
end if;

CR1 := CR1 or Uint32 (2 ** I2C_CR1_STOP);
Data := Uint8 (DR and 16#FF#);

CR1 := CR1 or Uint32 (2 ** I2C_CR1_ACK);
return True;
end Read;

function Read_Buffer (SlaveAddr : Uint8;
                      Data       : out Uint8_Array) return
                      Boolean is
    Status : Uint32;
    Time   : constant Positive := 100;
begin
    if Data'Length = 0 then
        return True;
    end if;

    CR1 := CR1 or Uint32 (2 ** I2C_CR1_ACK);
    CR1 := CR1 or Uint32 (2 ** I2C_CR1_START);

    if not Wait_Flag (Uint32 (2 ** I2C_SR1_SB), Time) then
        return False;
    end if;

    DR := (Uint32 (SlaveAddr) * 2) or 16#01#;  -- R/W=1

    if not Wait_Flag (Uint32 (2 ** I2C_SR1_ADDR), Time) then
        CR1 := CR1 or Uint32 (2 ** I2C_CR1_STOP);
        return False;
    end if;

    if Data'Length = 1 then
```

```

        CR1 := CR1 and not (Uint32 (2 ** I2C_CR1_ACK));
    end if;

    Status := SR1;
    Status := SR2;

    for I in Data'Range loop
        if I = Data'Last then

            CR1 := CR1 and not (Uint32 (2 ** I2C_CR1_ACK));

            if not Wait_Flag (Uint32 (2 ** I2C_SR1_RxNE), Time)
            then
                CR1 := CR1 or Uint32 (2 ** I2C_CR1_STOP);
                return False;
            end if;

            CR1 := CR1 or Uint32 (2 ** I2C_CR1_STOP);
            Data (I) := Uint8 (DR and 16#FF#);
        else
            if not Wait_Flag (Uint32 (2 ** I2C_SR1_RxNE), Time)
            then
                CR1 := CR1 or Uint32 (2 ** I2C_CR1_STOP);
                return False;
            end if;
            Data (I) := Uint8 (DR and 16#FF#);
        end if;
    end loop;

    CR1 := CR1 or Uint32 (2 ** I2C_CR1_ACK);  -- restaurar ACK
    return True;
end Read_Buffer;

```

--Procedimientos de diagnostico en caso de fallos

```

procedure Test_Hardware is
    Scl_State, Sda_State : Uint32;
    Scl_Shift, Sda_Shift : Uint32;
begin

```



```
Send_Line ("=== TEST HARDWARE ===");

Scl_Shift := Uint32(2**SCL_PIN);
Sda_Shift := Uint32(2**SDA_PIN);

Scl_State := (GPIO_ODR_SCL / Scl_Shift) and 1;
Sda_State := (GPIO_ODR_SDA / Sda_Shift) and 1;

Send_Line ("Estado actual - SCL=" & Uint32'Image(Scl_State) &
           " SDA=" & Uint32'Image(Sda_State));

if Scl_State = 0 then
    Send_Line (" ALERTA ! SCL está BAJO - posible
               cortocircuito");
end if;

if Sda_State = 0 then
    Send_Line (" ALERTA ! SDA está BAJO - posible
               cortocircuito");
end if;

if Scl_State = 1 and Sda_State = 1 then
    Send_Line ("OK: Ambas líneas ALTAS - pull-ups funcionando");
else
    Send_Line ("ERROR: Pull-ups ausentes, dañados o dispositivo
               roto");
end if;
end Test_Hardware;

procedure Scan_I2C_Bus is
    Status : Uint32;
    Found : Boolean := False;
begin
    Send_Line ("=== ESCANEO I2C ===");

    for Addr in 0 .. 127 loop

        CR1 := CR1 or Uint32(2**I2C_CR1_START);

        if Wait_Flag(Uint32(2**I2C_SR1_SB), 10) then

            DR := Uint32(Addr) * 2;
```

```

        if Wait_Flag(Uint32(2**I2C_SR1_ADDR), 10) then
            Send_Line ("Dispositivo encontrado en 0x" &
                        Uint8'Image(Uint8(Addr)));
            Found := True;

            Status := SR1;
            Status := SR2;
        end if;
    end if;

    -- STOP
    CR1 := CR1 or Uint32(2**I2C_CR1_STOP);
    for I in 1 .. 1000 loop null; end loop;
end loop;

if not Found then
    Send_Line ("NINGÚN dispositivo encontrado - posible:");
    Send_Line (" 1. Dispositivo roto/desconectado");
    Send_Line (" 2. Dirección incorrecta");
    Send_Line (" 3. Sin alimentación en el dispositivo");
    Send_Line (" 4. Pull-ups ausentes");
end if;
end Scan_I2C_Bus;

procedure Test_Minimo is
    Success : Boolean;
    Dummy_Data : Uint8 := 0;
begin
    Send_Line ("=== TEST MÍNIMO ===");

    CR1 := CR1 or Uint32(2**I2C_CR1_START);

    if Wait_Flag(Uint32(2**I2C_SR1_SB), 100) then
        Send_Line ("OK: START generado");
        CR1 := CR1 or Uint32(2**I2C_CR1_STOP);
        Send_Line ("OK: STOP generado");
    else
        Send_Line ("FAIL: No se pudo generar START");
        Send_Line (" -> Bus bloqueado por hardware");
    end if;
end Test_Minimo;

```

```
procedure Mostrar_Config is
  function To_String (V : Uint32) return String is
  begin
    return Long_Long_Integer'Image(Long_Long_Integer(V));
  end To_String;

  function To_String (V : Natural) return String is
  begin
    return Integer'Image(Integer(V));
  end To_String;

begin
  Send_Line("---- Configuracion I2C ----");
  Send_Line("I2C_Base: " & To_String(I2C_Base));
  Send_Line("SCL_PIN : " & To_String(SCL_PIN));
  Send_Line("SDA_PIN : " & To_String(SDA_PIN));
  Send_Line("GPIO_SCL: " & To_String(GPIO_SCL));
  Send_Line("GPIO_SDA: " & To_String(GPIO_SDA));
  Send_Line("-----");
end Mostrar_Config;

end Bus;
end I2C;
```

Listing 15: i2c.adb — Cuerpo del driver I2C

Paquete SPI

```
with System;
with stm32f446; use stm32f446;
generic
    SPI_Base      : Uint32;
    GPIO_SCK      : Uint32;
    GPIO_MISO     : Uint32;
    GPIO_MOSI     : Uint32;
    GPIO_CS       : Uint32;
    SCK_Pin       : Natural;
    MISO_Pin      : Natural;
    MOSI_Pin      : Natural;
    CS_Pin        : Natural;
package SPI is

    procedure Initialize;

    function Transfer_8  (Data : Uint8) return Uint8;
    function Transfer_16 (Data : Uint16) return Uint16;

    procedure Write_8  (Data : Uint8);
    procedure Write_16 (Data : Uint16);

    function Read_8  return Uint8;
    function Read_16 return Uint16;

    procedure Write_Buffer  (Data : Uint8_Array);
    procedure Read_Buffer   (Data : out Uint8_Array);
    procedure Transfer_Buffer(TX : Uint8_Array; RX : out Uint8_Array);

    procedure Set_Speed_Low;    -- 400 kHz para init SD
    procedure Set_Speed_High;   -- máxima velocidad

    procedure CS_Low;
    procedure CS_High;

private
```

```

function SPI_AF (SPIBase : Uint32; GPIOBase : Uint32; Pin :
    Natural) return Uint32;

function GPIO_To_AHB1_Bit (Base : Uint32) return Uint32 is
    (case Base is
        when GPIOA => 2**GPIOAEN,
        when GPIOB => 2**GPIOBEN,
        when GPIOC => 2**GPIOCEN,
        when GPIOD => 2**GPIODEN,
        when GPIOE => 2**GPIOEEN,
        when GPIOF => 2**GPIOFEN,
        when GPIOG => 2**GPIOGEN,
        when GPIOH => 2**GPIOHEN,
        when others => 0);

function SPI_To_APB_Bit (Base : Uint32) return Uint32 is
    (case Base is
        when SPI1_Base => 2**SPI1_EN,
        when SPI2_Base => 2**SPI2_EN,
        when SPI3_Base => 2**SPI3_EN,
        when SPI4_Base => 2**SPI4_EN,
        when others    => 0);

RCC_AHB1ENR : Uint32 with
    Volatile, Address => System'To_Address (RCC + 16#30#);
RCC_APB2ENR : Uint32 with
    Volatile, Address => System'To_Address (RCC + 16#44#);  --
    SPI1 en APB2
RCC_APB1ENR : Uint32 with
    Volatile, Address => System'To_Address (RCC + 16#40#);

-- GPIO SCK
GPIO_SCK_MODER : Uint32 with
    Volatile, Address => System'To_Address (GPIO_SCK + 16#00#);
GPIO_SCK_OTYPER : Uint32 with
    Volatile, Address => System'To_Address (GPIO_SCK + 16#04#);
GPIO_SCK_OSPEEDR : Uint32 with
    Volatile, Address => System'To_Address (GPIO_SCK + 16#08#);
GPIO_SCK_PUPDR : Uint32 with

```

```

    Volatile, Address => System'To_Address (GPIO_SCK + 16#0C#);
GPIO_SCK_ODR : Uint32 with
    Volatile, Address => System'To_Address (GPIO_SCK + 16#14#);
GPIO_SCK_AFRL : Uint32 with
    Volatile, Address => System'To_Address (GPIO_SCK + 16#20#);
GPIO_SCK_AFRH : Uint32 with
    Volatile, Address => System'To_Address (GPIO_SCK + 16#24#);

-- GPIO MISO
GPIO_MISO_MODER : Uint32 with
    Volatile, Address => System'To_Address (GPIO_MISO + 16#00#);
GPIO_MISO_OTYPER : Uint32 with
    Volatile, Address => System'To_Address (GPIO_MISO + 16#04#);
GPIO_MISO_OSPEEDR : Uint32 with
    Volatile, Address => System'To_Address (GPIO_MISO + 16#08#);
GPIO_MISO_PUPDR : Uint32 with
    Volatile, Address => System'To_Address (GPIO_MISO + 16#0C#);
GPIO_MISO_ODR : Uint32 with
    Volatile, Address => System'To_Address (GPIO_MISO + 16#14#);
GPIO_MISO_AFRL : Uint32 with
    Volatile, Address => System'To_Address (GPIO_MISO + 16#20#);
GPIO_MISO_AFRH : Uint32 with
    Volatile, Address => System'To_Address (GPIO_MISO + 16#24#);

-- GPIO MOSI
GPIO_MOSI_MODER : Uint32 with
    Volatile, Address => System'To_Address (GPIO_MOSI + 16#00#);
GPIO_MOSI_OTYPER : Uint32 with
    Volatile, Address => System'To_Address (GPIO_MOSI + 16#04#);
GPIO_MOSI_OSPEEDR : Uint32 with
    Volatile, Address => System'To_Address (GPIO_MOSI + 16#08#);
GPIO_MOSI_PUPDR : Uint32 with
    Volatile, Address => System'To_Address (GPIO_MOSI + 16#0C#);
GPIO_MOSI_ODR : Uint32 with
    Volatile, Address => System'To_Address (GPIO_MOSI + 16#14#);
GPIO_MOSI_AFRL : Uint32 with
    Volatile, Address => System'To_Address (GPIO_MOSI + 16#20#);
GPIO_MOSI_AFRH : Uint32 with
    Volatile, Address => System'To_Address (GPIO_MOSI + 16#24#);

-- GPIO CS
GPIO_CS_MODER : Uint32 with
    Volatile, Address => System'To_Address (GPIO_CS + 16#00#);
GPIO_CS_OSPEEDR : Uint32 with

```

```

        Volatile, Address => System'To_Address (GPIO_CS + 16#08#);
GPIO_CS_PUPDR : Uint32 with
        Volatile, Address => System'To_Address (GPIO_CS + 16#0C#);
GPIO_CS_ODR : Uint32 with
        Volatile, Address => System'To_Address (GPIO_CS + 16#14#);

-- Registros SPI
SPI_CR1 : Uint32 with
        Volatile, Address => System'To_Address (SPI_Base + 16#00#);
SPI_CR2 : Uint32 with
        Volatile, Address => System'To_Address (SPI_Base + 16#04#);
SPI_SR : Uint32 with
        Volatile, Address => System'To_Address (SPI_Base + 16#08#);
SPI_DR : Uint32 with
        Volatile, Address => System'To_Address (SPI_Base + 16#0C#);

BR_DIV256 : constant Uint32 := 16#38#;  -- 111 en bits [5:3] =
        328 kHz
BR_DIV4   : constant Uint32 := 16#08#;  -- 001 en bits [5:3] = 21
        MHz

end SPI;
```

Listing 16: spi.ads — Especificación del driver SPI

```

with System;
with Ada.Real_Time; use Ada.Real_Time;
with stm32f446;      use stm32f446;

package body SPI is

function SPI_AF (SPIBase : Uint32; GPIOBase : Uint32; Pin :
        Natural) return Uint32 is
begin
    case SPIBase is

        when SPI1_Base =>
            return 5;
```

```
when SPI2_Base =>
  case GPIOBase is
    when GPIOC =>
      case Pin is
        when 2 | 3 => return 5;
        when 1    => return 7;
        when others => return 16#FFFF_FFFF#;
      end case;
    when others =>
      return 5;
    end case;

when SPI3_Base =>
  case GPIOBase is
    when GPIOB =>
      case Pin is
        when 0 | 2    => return 7;
        when 3 | 4 | 5 => return 6;
        when others   => return 16#FFFF_FFFF#;
      end case;
    when GPIOC =>
      case Pin is
        when 1          => return 5;
        when 10 | 11 | 12 => return 6;
        when others     => return 16#FFFF_FFFF#;
      end case;
    when GPIOD =>
      case Pin is
        when 0    => return 6;
        when 6    => return 5;
        when others => return 16#FFFF_FFFF#;
      end case;
    when others =>
      return 16#FFFF_FFFF#;
    end case;

when SPI4_Base =>
  case GPIOBase is
    when GPIOD => return 5;
    when GPIOE => return 5;
    when GPIOG => return 6;
```



```
        when others => return 16#FFFF_FFFF#;
    end case;

    when others =>
        return 16#FFFF_FFFF#;
    end case;
end SPI_AF;

procedure Initialize is
    AF_MISO : Uint32 := SPI_AF (SPI_Base, GPIO_MISO, MISO_Pin);
    AF_MOSI : Uint32 := SPI_AF (SPI_Base, GPIO_MOSI, MOSI_Pin);
    AF_SCK   : Uint32 := SPI_AF (SPI_Base, GPIO_SCK,  SCK_Pin);
begin

    case SPI_Base is
        when SPI1_Base =>
            RCC_APB2ENR := RCC_APB2ENR or Uint32 (2**SPI1_EN);
        when SPI2_Base =>
            RCC_APB1ENR := RCC_APB1ENR or Uint32 (2**SPI2_EN);
        when SPI3_Base =>
            RCC_APB1ENR := RCC_APB1ENR or Uint32 (2**SPI3_EN);
        when SPI4_Base =>
            RCC_APB2ENR := RCC_APB2ENR or Uint32 (2**SPI4_EN);
        when others =>
            RCC_APB2ENR := RCC_APB2ENR or Uint32 (2**SPI1_EN);
    end case;

    RCC_AHB1ENR := RCC_AHB1ENR
        or GPIO_To_AHB1_Bit (GPIO_MISO)
        or GPIO_To_AHB1_Bit (GPIO_MOSI)
        or GPIO_To_AHB1_Bit (GPIO_SCK)
        or GPIO_To_AHB1_Bit (GPIO_CS);

    GPIO_CS_ODR := GPIO_CS_ODR or Uint32 (2**CS_Pin);

    GPIO_MISO_MODER := (GPIO_MISO_MODER
```

```
and not (Uint32 (2**((2*MISO_Pin))))
and not (Uint32 (2**((2*MISO_Pin)+1))))
or Uint32 (2**((2*MISO_Pin)+1));

GPIO_MOSI_MODER := (GPIO_MOSI_MODER
and not (Uint32 (2**((2*MOSI_Pin))))
and not (Uint32 (2**((2*MOSI_Pin)+1))))
or Uint32 (2**((2*MOSI_Pin)+1));

GPIO_SCK_MODER := (GPIO_SCK_MODER
and not (Uint32 (2**((2*SCK_Pin))))
and not (Uint32 (2**((2*SCK_Pin)+1))))
or Uint32 (2**((2*SCK_Pin)+1));

GPIO_MISO_OTYPER := GPIO_MISO_OTYPER and not (Uint32
(2**MISO_Pin));
GPIO_MOSI_OTYPER := GPIO_MOSI_OTYPER and not (Uint32
(2**MOSI_Pin));
GPIO_SCK_OTYPER := GPIO_SCK_OTYPER and not (Uint32
(2**SCK_Pin));

GPIO_MOSI_OSPEEDR := (GPIO_MOSI_OSPEEDR
and not (Uint32 (2**((2*MOSI_Pin))))
or Uint32 (2**((2*MOSI_Pin)+1));

GPIO_MISO_OSPEEDR := (GPIO_MISO_OSPEEDR
and not (Uint32 (2**((2*MISO_Pin))))
or Uint32 (2**((2*MISO_Pin)+1));

GPIO_SCK_OSPEEDR := (GPIO_SCK_OSPEEDR
and not (Uint32 (2**((2*SCK_Pin))))
or Uint32 (2**((2*SCK_Pin)+1));

GPIO_SCK_PUPDR := GPIO_SCK_PUPDR
and not (Uint32 (2**((2*SCK_Pin))))
and not (Uint32 (2**((2*SCK_Pin)+1))));
```

```
GPIO_MOSI_PUPDR := GPIO_MOSI_PUPDR
    and not (Uint32 (2**(2*MOSI_Pin)))
    and not (Uint32 (2**((2*MOSI_Pin)+1)));

GPIO_MISO_PUPDR := (GPIO_MISO_PUPDR
    and not (Uint32 (2**(2*MISO_Pin)))
    and not (Uint32 (2**((2*MISO_Pin)+1))))
or Uint32 (2**(2*MISO_Pin));

if MISO_Pin <= 7 then
    GPIO_MISO_AFRL := (GPIO_MISO_AFRL
        and not (Uint32 (2#1111# * (2**(4*MISO_Pin)))))
        or Uint32 (AF_MISO * (2**(4*MISO_Pin)));
else
    GPIO_MISO_AFRH := (GPIO_MISO_AFRH
        and not (Uint32 (2#1111# * (2**(4*(MISO_Pin-8))))))
        or Uint32 (AF_MISO * (2**(4*(MISO_Pin-8))));
end if;

if MOSI_Pin <= 7 then
    GPIO_MOSI_AFRL := (GPIO_MOSI_AFRL
        and not (Uint32 (2#1111# * (2**(4*MOSI_Pin)))))
        or Uint32 (AF_MOSI * (2**(4*MOSI_Pin)));
else
    GPIO_MOSI_AFRH := (GPIO_MOSI_AFRH
        and not (Uint32 (2#1111# * (2**(4*(MOSI_Pin-8))))))
        or Uint32 (AF_MOSI * (2**(4*(MOSI_Pin-8))));
end if;

if SCK_Pin <= 7 then
    GPIO_SCK_AFRL := (GPIO_SCK_AFRL
        and not (Uint32 (2#1111# * (2**(4*SCK_Pin)))))
        or Uint32 (AF_SCK * (2**(4*SCK_Pin)));
else
    GPIO_SCK_AFRH := (GPIO_SCK_AFRH
        and not (Uint32 (2#1111# * (2**(4*(SCK_Pin-8))))))
        or Uint32 (AF_SCK * (2**(4*(SCK_Pin-8))));
end if;
```

```
GPIO_CS_MODER := (GPIO_CS_MODER
    and not (Uint32 (2**(2*CS_Pin)))
    and not (Uint32 (2**((2*CS_Pin)+1))))
    or Uint32 (2**(2*CS_Pin));

GPIO_CS_OSPEEDR := (GPIO_CS_OSPEEDR
    and not (Uint32 (2**(2*CS_Pin)))
    or Uint32 (2**((2*CS_Pin)+1)));

GPIO_CS_PUPDR := GPIO_CS_PUPDR
    and not (Uint32 (2**(2*CS_Pin)))
    and not (Uint32 (2**((2*CS_Pin)+1)));

SPI_CR1 := SPI_CR1 and not (Uint32 (2**SPI_CR1_SPE));

SPI_CR1 :=
    Uint32 (2**SPI_CR1_MSTR)
    or Uint32 (2**SPI_CR1_SSM)
    or Uint32 (2**SPI_CR1_SSI)
    or BR_DIV256;

SPI_CR2 := 0;

SPI_CR1 := SPI_CR1 or Uint32 (2**SPI_CR1_SPE);

end Initialize;

procedure Set_Speed_Low is
begin
    SPI_CR1 := SPI_CR1 and not (Uint32 (2**SPI_CR1_SPE));
    SPI_CR1 := (SPI_CR1 and not (Uint32 (16#38#))) or BR_DIV256;
    SPI_CR1 := SPI_CR1 or Uint32 (2**SPI_CR1_SPE);
```

```
end Set_Speed_Low;
```

```
procedure Set_Speed_High is
```

```
begin
```

```
    SPI_CR1 := SPI_CR1 and not (Uint32 (2**SPI_CR1_SPE));
```

```
    SPI_CR1 := (SPI_CR1 and not (Uint32 (16#38#))) or BR_DIV4;
```

```
    SPI_CR1 := SPI_CR1 or Uint32 (2**SPI_CR1_SPE);
```

```
end Set_Speed_High;
```

```
procedure CS_Low is
```

```
begin
```

```
    GPIO_CS_ODR := GPIO_CS_ODR and not (Uint32 (2**CS_Pin));
```

```
end CS_Low;
```

```
procedure CS_High is
```

```
begin
```

```
    GPIO_CS_ODR := GPIO_CS_ODR or Uint32 (2**CS_Pin);
```

```
end CS_High;
```

```
procedure Wait_TXE is
```

```
begin
```

```
    loop
```

```
        exit when (SPI_SR and Uint32 (2**SPI_SR_TXE)) /= 0;
```

```
    end loop;
```

```
end Wait_TXE;
```

```
procedure Wait_RXNE is
```

```
begin
```

```
    loop
```

```
        exit when (SPI_SR and Uint32 (2**SPI_SR_RXNE)) /= 0;
```

```
    end loop;
```

```
end Wait_RXNE;
```

```
procedure Wait_Not_Busy is
```

```
begin
```

```
    loop
```

```
        exit when (SPI_SR and Uint32 (2**SPI_SR_BSY)) = 0;
```

```
    end loop;
```

```
end Wait_Not_Busy;
```

```
function Transfer_8 (Data : Uint8) return Uint8 is
begin
    Wait_TXE;
    SPI_DR := Uint32 (Data);
    Wait_RXNE;
    return Uint8 (SPI_DR and 16#FF#);
end Transfer_8;
```

```
function Transfer_16 (Data : Uint16) return Uint16 is
begin
    SPI_CR1 := SPI_CR1 and not (Uint32 (2**SPI_CR1_SPE));
    SPI_CR1 := SPI_CR1 or Uint32 (2**SPI_CR1_DFF);
    SPI_CR1 := SPI_CR1 or Uint32 (2**SPI_CR1_SPE);

    Wait_TXE;
    SPI_DR := Uint32 (Data);
    Wait_RXNE;

    declare
        Result : constant Uint16 := Uint16 (SPI_DR and 16#FFFF#);
    begin
        SPI_CR1 := SPI_CR1 and not (Uint32 (2**SPI_CR1_SPE));
        SPI_CR1 := SPI_CR1 and not (Uint32 (2**SPI_CR1_DFF));
        SPI_CR1 := SPI_CR1 or Uint32 (2**SPI_CR1_SPE);
        return Result;
    end;
end Transfer_16;
```

```
procedure Write_8 (Data : Uint8) is
    Dummy : Uint8;
begin
    Dummy := Transfer_8 (Data);
end Write_8;
```

```
procedure Write_16 (Data : Uint16) is
    Dummy : Uint16;
begin
    Dummy := Transfer_16 (Data);
end Write_16;

function Read_8 return Uint8 is
begin
    return Transfer_8 (16#FF#);
end Read_8;

function Read_16 return Uint16 is
begin
    return Transfer_16 (16#FFFF#);
end Read_16;

procedure Write_Buffer (Data : Uint8_Array) is
begin
    for Byte of Data loop
        Write_8 (Byte);
    end loop;
    Wait_Not_Busy;
end Write_Buffer;

procedure Read_Buffer (Data : out Uint8_Array) is
begin
    for I in Data'Range loop
        Data (I) := Read_8;
    end loop;
end Read_Buffer;

procedure Transfer_Buffer (TX : Uint8_Array; RX : out Uint8_Array)
    is
    Min_Last : constant Positive :=
        (if TX'Last < RX'Last then TX'Last else RX'Last);
begin
    for I in TX'First .. Min_Last loop
        RX (I) := Transfer_8 (TX (I));
    end loop;
```

```
        Wait_Not_Busy;  
    end Transfer_Buffer;  
  
end SPI;
```

Listing 17: spi.adb — Cuerpo del driver SPI

Paquete W5500 (Ethernet)

```
with SPI_Driver;
with STM32F446; use STM32F446;

package W5500 is

    function Read_Version return Uint8;
    function Read_PHYCFGR return Uint8;

    procedure Set_MAC      (Mac : Uint8_Array);
    procedure Set_IP       (IP  : Uint8_Array);
    procedure Set_Subnet   (SN  : Uint8_Array);
    procedure Set_Gateway  (GW  : Uint8_Array);
    procedure Get_MAC      (Mac : out Uint8_Array);
    procedure Get_IP       (IP  : out Uint8_Array);

    procedure Socket0_Open_TCP (Local_Port : Uint16);
    procedure Socket0_Connect (Dest_IP : Uint8_Array; Dest_Port :
        Uint16);
    function Socket0_Status return Uint8;
    procedure Socket0_Close;

    function Ping (Dest_IP      : Uint8_Array;
        Timeout_MS : Natural := 3000) return Boolean;

    SOCK_CLOSED      : constant Uint8 := 16#00#;
    SOCK_INIT        : constant Uint8 := 16#13#;
    SOCK_LISTEN      : constant Uint8 := 16#14#;
    SOCK_SYNSENT     : constant Uint8 := 16#15#;
    SOCK_ESTABLISHED : constant Uint8 := 16#17#;
    SOCK_CLOSE_WAIT  : constant Uint8 := 16#1C#;
    SOCK_IPRAW       : constant Uint8 := 16#42#;

    procedure Print_Hex (Label : String; Val : Uint8);

    function Resolve_DNS (Hostname : String;
        DNS_Server : Uint8_Array;
        IP_Out     : out Uint8_Array;
        Timeout_MS : Natural := 3000) return Boolean;

    procedure HTTP_Server (Port : Uint16 := 80);
```

```

    procedure HTTP_Server_mDNS (Port : Uint16 := 80 ;
        Hostname:String;Content:String);

procedure mDNS_Announce (Hostname:string);

procedure mDNS_Loop(Hostname :string);

type DHCP_Result is record
    Success : Boolean;
    IP       : Uint8_Array (1 .. 4);
    Subnet   : Uint8_Array (1 .. 4);
    Gateway  : Uint8_Array (1 .. 4);
    DNS      : Uint8_Array (1 .. 4);
    Lease    : Uint32;
end record;

function DHCP_Request (Timeout_MS : Natural := 10_000) return
    DHCP_Result;

private

    procedure Write_Reg (Addr : Uint16; Block : Uint8; Data : Uint8);
    function  Read_Reg   (Addr : Uint16; Block : Uint8) return Uint8;
    procedure Write_Reg16 (Addr : Uint16; Block : Uint8; Data :
        Uint16);
    function  Read_Reg16  (Addr : Uint16; Block : Uint8) return
        Uint16;
    procedure Write_Buf  (Addr : Uint16; Block : Uint8; Data :
        Uint8_Array);
    procedure Read_Buf   (Addr : Uint16; Block : Uint8; Data : out
        Uint8_Array);

    COMMON_RD : constant Uint8 := 16#00#;
    COMMON_WR : constant Uint8 := 16#04#;
    SO_REG_RD  : constant Uint8 := 16#08#;
    SO_REG_WR  : constant Uint8 := 16#0C#;
    SO_TX_WR   : constant Uint8 := 16#14#;
    SO_RX_RD   : constant Uint8 := 16#18#;

```

```

S1_REG_RD : constant UInt8 := 16#28#;
S1_REG_WR : constant UInt8 := 16#2C#;
S1_TX_WR  : constant UInt8 := 16#34#;
S1_RX_RD  : constant UInt8 := 16#38#;
S1_TX_FSR : constant UInt16 := 16#0020#;
S1_TX_WR_R: constant UInt16 := 16#0024#;
S1_RX_RSR : constant UInt16 := 16#0026#;
S1_RX_RD_R: constant UInt16 := 16#0028#;
S2_REG_RD : constant UInt8 := 16#48#;
S2_REG_WR : constant UInt8 := 16#4C#;
S2_TX_WR  : constant UInt8 := 16#54#;
S2_RX_RD  : constant UInt8 := 16#58#;
S3_REG_RD : constant UInt8 := 16#68#;
S3_REG_WR : constant UInt8 := 16#6C#;
S3_TX_WR  : constant UInt8 := 16#74#;
S3_RX_RD  : constant UInt8 := 16#78#;
MDNS_Buffer : UInt8_Array (1 .. 256);

-- Registros comunes
REG_MR      : constant UInt16 := 16#0000#;
REG_GAR     : constant UInt16 := 16#0001#;
REG_SUBR    : constant UInt16 := 16#0005#;
REG_SHAR    : constant UInt16 := 16#0009#;
REG_SIPR    : constant UInt16 := 16#000F#;
REG_PHYCFGR : constant UInt16 := 16#002E#;
REG_VERSIONR : constant UInt16 := 16#0039#;

-- Registro de socket
Sn_MR      : constant UInt16 := 16#0000#;
Sn_CR      : constant UInt16 := 16#0001#;
Sn_SR      : constant UInt16 := 16#0003#;
Sn_PORT    : constant UInt16 := 16#0004#;
Sn_DIPR    : constant UInt16 := 16#000C#;
Sn_DPORT   : constant UInt16 := 16#0010#;
Sn_TX_FSR  : constant UInt16 := 16#0020#;
Sn_TX_RD   : constant UInt16 := 16#0022#;
Sn_TX_WR   : constant UInt16 := 16#0024#;
Sn_RX_RSR  : constant UInt16 := 16#0026#;
Sn_RX_RD   : constant UInt16 := 16#0028#;

-- Modos Sn_MR
MR_TCP : constant UInt8 := 16#01#;
MR_UDP : constant UInt8 := 16#02#;
MR_RAW : constant UInt8 := 16#03#; -- IPRAW en W5500 (con

```

```

    Sn_PROTO=0x001C)

-- Comandos Sn_CR
CR_OPEN      : constant Uint8 := 16#01#;
CR_LISTEN    : constant Uint8 := 16#02#;
CR_CONNECT    : constant Uint8 := 16#04#;
CR_DISCON    : constant Uint8 := 16#08#;
CR_CLOSE     : constant Uint8 := 16#10#;
CR_SEND      : constant Uint8 := 16#20#;
CR_RECV      : constant Uint8 := 16#40#;

ICMP_ECHO_REQUEST : constant Uint8 := 8;
ICMP_ECHO_REPLY   : constant Uint8 := 0;

Ping_Buffer : Uint8_Array (1 .. 128);

procedure Socket0_Open_Raw;
procedure Send_Ping_Request (Dest_IP : Uint8_Array; Seq :
    Uint16);
function Receive_Ping_Reply (Timeout_MS : Natural) return
    Boolean;
Sn_DHAR : constant Uint16 := 16#0006#;

SOCK_UDP      : constant Uint8 := 16#22#;

DNS_Buffer    : Uint8_Array (1 .. 256);

procedure Socket1_Open_UDP (Local_Port : Uint16);
procedure Socket1_Close;
function Socket1_Status return Uint8;

HTTP_Buffer : Uint8_Array (1 .. 512);

procedure Socket0_Listen (Local_Port : Uint16);
procedure Socket0_Send (Data : Uint8_Array);
function Socket0_Recv (Max_Len : Natural) return Natural;
procedure Socket2_Open_mDNS;
procedure Socket2_Close;
procedure mDNS_Send_Response (Query_ID : Uint16 ; Hostname:string);
function mDNS_Parse_Query return Boolean;

```

```

procedure Encode_DNS_Name (Hostname : String ; Result      : out
    Uint8_Array; Result_Len  : out Natural) ;

DHCP_Buffer : Uint8_Array (1 .. 548);

procedure Socket3_Open_DHCP;
procedure Socket3_Close;
procedure DHCP_Send (Data : Uint8_Array; Len : Natural);
function  DHCP_Recv (Timeout_MS : Natural) return Natural;

end W5500;

```

Listing 18: w5500.ads — Especificación del controlador Ethernet

```

with Ada.Real_Time; use Ada.Real_Time;
with SPI_Driver;
with USART_Driver; use USART_Driver;

package body W5500 is

    procedure Write_Reg (Addr : Uint16; Block : Uint8; Data : Uint8)
    is
    begin
        SPI_Driver.CS_Low;
        SPI_Driver.Write_8 (Uint8 (Addr / 256));
        SPI_Driver.Write_8 (Uint8 (Addr mod 256));
        SPI_Driver.Write_8 (Block);
        SPI_Driver.Write_8 (Data);
        SPI_Driver.CS_High;
    end Write_Reg;

    procedure Print_Hex (Label : String; Val : Uint8) is
        Hex : constant array (0 .. 15) of Character :=
            "0123456789ABCDEF";
    begin
        USART_Driver.Send_Line (Label & "0x"
            & Hex (Natural (Val / 16))
            & Hex (Natural (Val mod 16)));
    end Print_Hex;

```

```
end Print_Hex;

function Read_Reg (Addr : Uint16; Block : Uint8) return Uint8 is
    Result : Uint8;
begin
    SPI_Driver.CS_Low;
    SPI_Driver.Write_8 (Uint8 (Addr / 256));
    SPI_Driver.Write_8 (Uint8 (Addr mod 256));
    SPI_Driver.Write_8 (Block);
    Result := SPI_Driver.Read_8;
    SPI_Driver.CS_High;
    return Result;
end Read_Reg;

procedure Write_Buf (Addr : Uint16; Block : Uint8; Data :
    Uint8_Array) is
begin
    SPI_Driver.CS_Low;
    SPI_Driver.Write_8 (Uint8 (Addr / 256));
    SPI_Driver.Write_8 (Uint8 (Addr mod 256));
    SPI_Driver.Write_8 (Block);
    SPI_Driver.Write_Buffer (Data);
    SPI_Driver.CS_High;
end Write_Buf;

procedure Read_Buf (Addr : Uint16; Block : Uint8; Data : out
    Uint8_Array) is
begin
    SPI_Driver.CS_Low;
    SPI_Driver.Write_8 (Uint8 (Addr / 256));
    SPI_Driver.Write_8 (Uint8 (Addr mod 256));
    SPI_Driver.Write_8 (Block);
    SPI_Driver.Read_Buffer (Data);
    SPI_Driver.CS_High;
end Read_Buf;

function Read_Reg16 (Addr : Uint16; Block : Uint8) return Uint16
    is
begin
    return Uint16 (Read_Reg (Addr,      Block)) * 256
        + Uint16 (Read_Reg (Addr + 1, Block));
end Read_Reg16;

procedure Write_Reg16 (Addr : Uint16; Block : Uint8; Data :
```

```
    Uint16) is
begin
    Write_Reg (Addr,      Block, Uint8 (Data / 256));
    Write_Reg (Addr + 1, Block, Uint8 (Data mod 256));
end Write_Reg16;

function Read_Version return Uint8 is
begin
    return Read_Reg (REG_VERSIONR, COMMON_RD);
end Read_Version;

function Read_PHYCFGR return Uint8 is
begin
    return Read_Reg (REG_PHYCFGR, COMMON_RD);
end Read_PHYCFGR;

procedure Set_Gateway (GW : Uint8_Array) is
begin
    Write_Buf (REG_GAR, COMMON_WR, GW);
end Set_Gateway;

procedure Set_Subnet (SN : Uint8_Array) is
begin
    Write_Buf (REG_SUBR, COMMON_WR, SN);
end Set_Subnet;

procedure Set_MAC (Mac : Uint8_Array) is
begin
    Write_Buf (REG_SHAR, COMMON_WR, Mac);
end Set_MAC;

procedure Set_IP (IP : Uint8_Array) is
begin
    Write_Buf (REG_SIPR, COMMON_WR, IP);
end Set_IP;

procedure Get_MAC (Mac : out Uint8_Array) is
begin
    Read_Buf (REG_SHAR, COMMON_RD, Mac);
end Get_MAC;
```

```
procedure Get_IP (IP : out Uint8_Array) is
begin
    Read_Buf (REG_SIPR, COMMON_RD, IP);
end Get_IP;

procedure Socket0_Open_TCP (Local_Port : Uint16) is
begin
    Write_Reg (Sn_CR, SO_REG_WR, CR_CLOSE);
    Write_Reg (Sn_MR, SO_REG_WR, MR_TCP);
    Write_Reg16 (Sn_PORT, SO_REG_WR, Local_Port);
    Write_Reg (Sn_CR, SO_REG_WR, CR_OPEN);
end Socket0_Open_TCP;

procedure Socket0_Connect (Dest_IP : Uint8_Array; Dest_Port :
    Uint16) is
begin
    Write_Buf (Sn_DIPR, SO_REG_WR, Dest_IP);
    Write_Reg16 (Sn_DPORT, SO_REG_WR, Dest_Port);
    Write_Reg (Sn_CR, SO_REG_WR, CR_CONNECT);
end Socket0_Connect;

function Socket0_Status return Uint8 is
begin
    return Read_Reg (Sn_SR, SO_REG_RD);
end Socket0_Status;

procedure Socket0_Close is
begin
    Write_Reg (Sn_CR, SO_REG_WR, CR_DISCON);
    Write_Reg (Sn_CR, SO_REG_WR, CR_CLOSE);
end Socket0_Close;

procedure Socket0_Open_Raw is
    T : Time;
begin
    Write_Reg (Sn_CR, SO_REG_WR, CR_CLOSE);
    T := Clock + Milliseconds (10);
    delay until T;
    Write_Reg (Sn_MR, SO_REG_WR, 16#04#);
    Write_Reg (Sn_CR, SO_REG_WR, CR_OPEN);
```



```

    T := Clock + Milliseconds (50);
    delay until T;
end Socket0_Open_Raw;

procedure Send_Ping_Request (Dest_IP : Uint8_Array; Seq : Uint16) is
    -- Frame completo: Ethernet(14) + IP(20) + ICMP(64) = 98 bytes
    Identifier : constant Uint16 := 16#1234#;
    ICMP_Len    : constant Natural := 64;    -- 8 header + 56 data
    IP_Len      : constant Natural := 20 + ICMP_Len; -- 84
    Frame_Len   : constant Natural := 14 + IP_Len;   -- 98

    My_MAC : constant Uint8_Array (1 .. 6) :=
        (16#DE#, 16#AD#, 16#BE#, 16#EF#, 16#FE#, 16#ED#);
    GW_MAC : constant Uint8_Array (1 .. 6) :=
        (16#C0#, 16#FD#, 16#84#, 16#D6#, 16#97#, 16#5F#);
    My_IP   : constant Uint8_Array (1 .. 4) :=
        (192, 168, 1, 180);

    F      : Uint8_Array (1 .. Frame_Len) := (others => 0);
    Chk     : Uint16;
    TX_Ptr  : Uint16;
    IR      : Uint8;
    Timeout : Natural;

    function IP_Checksum (Data : Uint8_Array; From, Len : Natural)
        return Uint16 is
            Sum : Uint32 := 0;
            I   : Natural := From;
        begin
            while I + 1 <= From + Len - 1 loop
                Sum := Sum + Uint32 (Data (I)) * 256 + Uint32 (Data (I +
                    1));
                I := I + 2;
            end loop;
            if I <= From + Len - 1 then
                Sum := Sum + Uint32 (Data (I)) * 256;
            end if;
            while Sum > 16#FFFF# loop
                Sum := (Sum and 16#FFFF#) + (Sum / 16#1_0000#);
            end loop;
            return Uint16 ((not Sum) and 16#FFFF#);
        end IP_Checksum;

begin

```

```

F (1) := GW_MAC (1); F (2) := GW_MAC (2); F (3) := GW_MAC (3);
F (4) := GW_MAC (4); F (5) := GW_MAC (5); F (6) := GW_MAC (6);

F (7) := My_MAC (1); F (8) := My_MAC (2); F (9) := My_MAC (3);
F (10) := My_MAC (4); F (11) := My_MAC (5); F (12) := My_MAC (6);

F (13) := 16#08#; F (14) := 16#00#;

F (15) := 16#45#; -- Version=4, IHL=5
F (16) := 0; -- DSCP/ECN
F (17) := Uint8 (IP_Len / 256); -- Total length H
F (18) := Uint8 (IP_Len mod 256); -- Total length L (=84)
F (19) := 16#00#; F (20) := 16#01#; -- ID = 1
F (21) := 0; F (22) := 0; -- Flags/Fragment = 0
F (23) := 64; -- TTL
F (24) := 1; -- Protocol = ICMP
F (25) := 0; F (26) := 0; -- IP checksum
    (provisional)
-- IP origen
F (27) := My_IP (1); F (28) := My_IP (2);
F (29) := My_IP (3); F (30) := My_IP (4);
-- IP destino
F (31) := Dest_IP (1); F (32) := Dest_IP (2);
F (33) := Dest_IP (3); F (34) := Dest_IP (4);

-- Calcular checksum IP
Chk := IP_Checksum (F, 15, 20);
F (25) := Uint8 (Chk / 256);
F (26) := Uint8 (Chk mod 256);

F (35) := 8; -- Type = Echo Request
F (36) := 0; -- Code
F (37) := 0; F (38) := 0;
F (39) := Uint8 (Identifier / 256);
F (40) := Uint8 (Identifier mod 256);
F (41) := Uint8 (Seq / 256);
F (42) := Uint8 (Seq mod 256);
for I in 1 .. 56 loop
    F (42 + I) := Uint8 (I mod 256); -- datos
end loop;

```

```

-- Calcular checksum ICMP (sobre bytes 35..98)
Chk := IP_Checksum (F, 35, ICMP_Len);
F (37) := Uint8 (Chk / 256);
F (38) := Uint8 (Chk mod 256);

-- === Escribir en buffer TX ===
TX_Ptr := Read_Reg16 (Sn_TX_WR, SO_REG_RD);
Write_Buf (TX_Ptr and 16#07FF#, SO_TX_WR, F);
Write_Reg16 (Sn_TX_WR, SO_REG_WR, TX_Ptr + Uint16 (Frame_Len));
Write_Reg (Sn_CR, SO_REG_WR, CR_SEND);

-- Esperar CR = 0
Timeout := 0;
loop
  IR := Read_Reg (Sn_CR, SO_REG_RD);
  exit when IR = 0;
  Timeout := Timeout + 1;
  exit when Timeout > 1000;
  delay until Clock + Milliseconds (1);
end loop;

IR := Read_Reg (16#0002#, SO_REG_RD);
Print_Hex ("DBG Sn_IR: ", IR);
if (IR and 16#10#) /= 0 then
  USART_Driver.Send_Line ("DBG: SEND_OK");
end if;
if (IR and 16#08#) /= 0 then
  USART_Driver.Send_Line ("DBG: SEND_TIMEOUT");
end if;
Write_Reg (16#0002#, SO_REG_WR, 16#FF#);

end Send_Ping_Request;

function Receive_Ping_Reply (Timeout_MS : Natural) return Boolean is
  Deadline : constant Time := Clock + Milliseconds (Timeout_MS);
  RX_Size   : Uint16;
  RX_Ptr    : Uint16;
  Read_Len  : Natural;
  Got_Data  : Boolean := False;

  ICMP_TYPE_OFF : constant Natural := 37;
begin
  loop

```

```

    exit when Clock >= Deadline;

    if Socket0_Status = SOCK_CLOSED then
        USART_Driver.Send_Line ("DBG: socket cerrado");
        return False;
    end if;

    RX_Size := Read_Reg16 (Sn_RX_RSR, SO_REG_RD);

    if RX_Size > 0 then
        Got_Data := True;
        Read_Len := Natural (RX_Size);
        if Read_Len > Ping_Buffer'Length then
            Read_Len := Ping_Buffer'Length;
        end if;

        RX_Ptr := Read_Reg16 (Sn_RX_RD, SO_REG_RD);
        Read_Buf (RX_Ptr, SO_RX_RD, Ping_Buffer (1 .. Read_Len));
        Write_Reg16 (Sn_RX_RD, SO_REG_WR, RX_Ptr + Uint16
            (Read_Len));
        Write_Reg (Sn_CR, SO_REG_WR, CR_RECV);

        Print_Hex ("DBG RX bytes: ", Uint8 (Read_Len));

        for I in 1 .. Natural'Min (Read_Len, 40) loop
            Print_Hex ("DBG[" & Integer'Image (I) & "]: ",
                Ping_Buffer (I));
        end loop;

        if Read_Len >= ICMP_TYPE_OFF then
            Print_Hex ("DBG ICMP_TYPE: ", Ping_Buffer
                (ICMP_TYPE_OFF));
            if Ping_Buffer (ICMP_TYPE_OFF) = ICMP_ECHO_REPLY then
                return True;
            end if;
        end if;
    end if;

    delay until Clock + Milliseconds (10);
end loop;

if not Got_Data then
    USART_Driver.Send_Line ("DBG: no llego ningun dato al RX");
end if;

```

```
    return False;
end Receive_Ping_Reply;

function Ping (Dest_IP      : Uint8_Array;
               Timeout_MS   : Natural := 3000) return Boolean is
    Success : Boolean;
    Status   : Uint8;
begin
    USART_Driver.Send_Line ("=== PING ===");

    Socket0_Open_Raw;

    Status := Socket0_Status;
    Print_Hex ("Socket Raw status: ", Status);
    if Status /= SOCK_IPRAW then
        USART_Driver.Send_Line ("ERROR: socket Raw no abrio
                                (esperado 0x32)");
        Socket0_Close;
        return False;
    end if;

    Send_Ping_Request (Dest_IP, 1);
    USART_Driver.Send_Line ("Ping request enviado");

    Success := Receive_Ping_Reply (Timeout_MS);

    Socket0_Close;

    if Success then
        USART_Driver.Send_Line ("Ping OK!");
    else
        USART_Driver.Send_Line ("Ping timeout");
    end if;

    return Success;
end Ping;

-- Socket 1 UDP (para DNS)

procedure Socket1_Open_UDP (Local_Port : Uint16) is
    T : constant Time := Clock + Milliseconds (10);
```

```

begin
    Write_Reg    (Sn_CR,    S1_REG_WR, CR_CLOSE);
    delay until T;
    Write_Reg    (Sn_MR,    S1_REG_WR, MR_UDP);
    Write_Reg16  (Sn_PORT, S1_REG_WR, Local_Port);
    Write_Reg    (Sn_CR,    S1_REG_WR, CR_OPEN);
    delay until Clock + Milliseconds (10);
end Socket1_Open_UDP;

procedure Socket1_Close is
begin
    Write_Reg (Sn_CR, S1_REG_WR, CR_CLOSE);
end Socket1_Close;

function Socket1_Status return Uint8 is
begin
    return Read_Reg (Sn_SR, S1_REG_RD);
end Socket1_Status;

--  DNS

function Resolve_DNS (Hostname    : String;
                      DNS_Server  : Uint8_Array;
                      IP_Out       : out Uint8_Array;
                      Timeout_MS  : Natural := 3000) return Boolean is

    --  Construye la sección QNAME del DNS a partir del hostname
    --  "www.google.com" -> 03 77 77 77 06 67 6F 6F 67 6C 65 03 63 6F
    --  6D 00
    procedure Encode_Name (Buf      : in out Uint8_Array;
                          Offset    : in out Natural;
                          Name      : String) is
        Label_Start : Natural := Name'First;
        Len          : Natural;
    begin
        for I in Name'First .. Name'Last + 1 loop
            if I = Name'Last + 1 or else Name (I) = '.' then
                Len := I - Label_Start;
                Buf (Offset) := Uint8 (Len);
                Offset := Offset + 1;
                for J in Label_Start .. I - 1 loop
                    Buf (Offset) := Character'Pos (Name (J));

```

```

        Offset := Offset + 1;
    end loop;
    Label_Start := I + 1;
end if;
end loop;
Buf (Offset) := 0;    -- terminador
Offset := Offset + 1;
end Encode_Name;

TX_ID      : constant Uint16 := 16#ABCD#;
Pkt_Len    : Natural;
TX_Ptr     : Uint16;
RX_Size    : Uint16;
RX_Ptr     : Uint16;
Read_Len   : Natural;
Deadline   : constant Time := Clock + Milliseconds (Timeout_MS);
Off        : Natural;
Ancount    : Uint16;
Qdcount    : Uint16;
Rtype      : Uint16;
Rdlen      : Uint16;
Found      : Boolean := False;

begin
    IP_Out := (others => 0);
    Socket1_Open_UDP (16#0400#);    -- puerto local 1024

    if Socket1_Status /= SOCK_UDP then
        USART_Driver.Send_Line ("DNS: error abriendo socket UDP");
        return False;
    end if;

    Write_Buf (Sn_DIPR, S1_REG_WR, DNS_Server);
    Write_Reg16 (Sn_DPORT, S1_REG_WR, 53);

    DNS_Buffer := (others => 0);

    DNS_Buffer (1) := Uint8 (TX_ID / 256);
    DNS_Buffer (2) := Uint8 (TX_ID mod 256);
    DNS_Buffer (3) := 16#01#;    -- QR=0 query, OPCODE=0, RD=1
    DNS_Buffer (4) := 16#00#;

```

```

DNS_Buffer (5) := 16#00#; DNS_Buffer (6) := 16#01#; --
  QDCOUNT=1
DNS_Buffer (7) := 16#00#; DNS_Buffer (8) := 16#00#; --
  ANCOUNT=0
DNS_Buffer (9) := 16#00#; DNS_Buffer (10) := 16#00#; --
  NSCOUNT=0
DNS_Buffer (11) := 16#00#; DNS_Buffer (12) := 16#00#; --
  ARCOUNT=0

-- QNAME
Off := 13;
Encode_Name (DNS_Buffer, Off, Hostname);

-- QTYPE=A (0x0001), QCLASS=IN (0x0001)
DNS_Buffer (Off) := 16#00#; DNS_Buffer (Off + 1) := 16#01#;
DNS_Buffer (Off + 2) := 16#00#; DNS_Buffer (Off + 3) := 16#01#;
Pkt_Len := Off + 3;

-- Enviar por socket 1 UDP
TX_Ptr := Read_Reg16 (Sn_TX_WR, S1_REG_RD);
Write_Buf (TX_Ptr and 16#07FF#, S1_TX_WR, DNS_Buffer (1 ..
  Pkt_Len));
Write_Reg16 (Sn_TX_WR, S1_REG_WR, TX_Ptr + Uint16 (Pkt_Len));
Write_Reg (Sn_CR, S1_REG_WR, CR_SEND);

USART_Driver.Send_Line ("DNS: query enviada para " & Hostname);

-- Esperar respuesta
loop
  exit when Clock >= Deadline;

  RX_Size := Read_Reg16 (S1_RX_RSR, S1_REG_RD);

  if RX_Size > 0 then
    Read_Len := Natural (RX_Size);
    if Read_Len > DNS_Buffer'Length then
      Read_Len := DNS_Buffer'Length;
    end if;

    RX_Ptr := Read_Reg16 (S1_RX_RD_R, S1_REG_RD);
    Read_Buf (RX_Ptr, S1_RX_RD, DNS_Buffer (1 .. Read_Len));
    Write_Reg16 (S1_RX_RD_R, S1_REG_WR, RX_Ptr + Uint16
      (Read_Len));
    Write_Reg (Sn_CR, S1_REG_WR, CR_RECV);
  end if;
end loop

```



```
Off := 9;

if Read_Len < Off + 12 then
    USART_Driver.Send_Line ("DNS: respuesta demasiado
        corta");
    Socket1_Close;
    return False;
end if;

declare
    RX_ID : constant Uint16 :=
        Uint16 (DNS_Buffer (Off)) * 256 +
        Uint16 (DNS_Buffer (Off + 1));
begin
    if RX_ID /= TX_ID then
        USART_Driver.Send_Line ("DNS: ID no coincide");
        Socket1_Close;
        return False;
    end if;
end;

Ancount := Uint16 (DNS_Buffer (Off + 6)) * 256 +
    Uint16 (DNS_Buffer (Off + 7));
Qdcount := Uint16 (DNS_Buffer (Off + 4)) * 256 +
    Uint16 (DNS_Buffer (Off + 5));

Print_Hex ("DNS ANCOUNT: ", Uint8 (Ancount));

if Ancount = 0 then
    USART_Driver.Send_Line ("DNS: sin respuestas
        (NXDOMAIN?)");
    Socket1_Close;
    return False;
end if;

Off := Off + 12;  -- saltar cabecera

for Q in 1 .. Natural (Qdcount) loop
```

```

loop
  exit when Off > Read_Len;
  declare
    Label_Len : constant Uint8 := DNS_Buffer (Off);
  begin
    if Label_Len = 0 then
      Off := Off + 1;
      exit;
    elsif (Label_Len and 16#C0#) = 16#C0# then
      Off := Off + 2;
      exit;
    else
      Off := Off + 1 + Natural (Label_Len);
    end if;
  end;
end loop;
Off := Off + 4;
end loop;

for A in 1 .. Natural (Ancount) loop
  exit when Off > Read_Len;

  if (DNS_Buffer (Off) and 16#C0#) = 16#C0# then
    Off := Off + 2;
  else
    loop
      exit when Off > Read_Len;
      declare
        L : constant Uint8 := DNS_Buffer (Off);
      begin
        if L = 0 then
          Off := Off + 1; exit;
        elsif (L and 16#C0#) = 16#C0# then
          Off := Off + 2; exit;
        else
          Off := Off + 1 + Natural (L);
        end if;
      end;
    end loop;
  end if;

  exit when Off + 10 > Read_Len;

```

```
Rtype := Uint16 (DNS_Buffer (Off))      * 256 +
        Uint16 (DNS_Buffer (Off + 1));

Rdlen := Uint16 (DNS_Buffer (Off + 8)) * 256 +
        Uint16 (DNS_Buffer (Off + 9));
Off := Off + 10;

if Rtype = 1 and then Rdlen = 4 then
    -- Tipo A, IPv4
    IP_Out (1) := DNS_Buffer (Off);
    IP_Out (2) := DNS_Buffer (Off + 1);
    IP_Out (3) := DNS_Buffer (Off + 2);
    IP_Out (4) := DNS_Buffer (Off + 3);
    Found := True;
    exit;
end if;

Off := Off + Natural (Rdlen);
end loop;

Socket1_Close;

if Found then
    USART_Driver.Send_Line ("DNS OK: " &
        Integer'Image (Natural (IP_Out (1))) & "." &
        Integer'Image (Natural (IP_Out (2))) & "." &
        Integer'Image (Natural (IP_Out (3))) & "." &
        Integer'Image (Natural (IP_Out (4))));
    return True;
else
    USART_Driver.Send_Line ("DNS: no se encontro registro
        A");
    return False;
end if;
end if;

delay until Clock + Milliseconds (10);
end loop;

USART_Driver.Send_Line ("DNS: timeout esperando respuesta");
Socket1_Close;
return False;
end Resolve_DNS;
```

```

procedure Socket0_Listen (Local_Port : Uint16) is
begin
    Write_Reg (Sn_CR, SO_REG_WR, CR_CLOSE);
    delay until Clock + Milliseconds (5);
    Write_Reg (Sn_MR, SO_REG_WR, MR_TCP);
    Write_Reg16 (Sn_PORT, SO_REG_WR, Local_Port);
    Write_Reg (Sn_CR, SO_REG_WR, CR_OPEN);
    delay until Clock + Milliseconds (5);
    Write_Reg (Sn_CR, SO_REG_WR, CR_LISTEN);
    -- CR_LISTEN = 0x02
end Socket0_Listen;

procedure Socket0_Send (Data : Uint8_Array) is
    TX_Ptr : Uint16;
    TX_Free : Uint16;
    Sent : Natural := 0;
    Chunk : Natural;
    Timeout : Natural;
    IR : Uint8;
begin
    while Sent < Data'Length loop

        loop
            TX_Free := Read_Reg16 (Sn_TX_FSR, SO_REG_RD);
            exit when TX_Free > 0;
            delay until Clock + Milliseconds (1);
        end loop;

        Chunk := Natural'Min (Data'Length - Sent, Natural (TX_Free));

        TX_Ptr := Read_Reg16 (Sn_TX_WR, SO_REG_RD);
        Write_Buf (TX_Ptr and 16#07FF#, SO_TX_WR,
            Data (Data'First + Sent .. Data'First + Sent +
                Chunk - 1));
        Write_Reg16 (Sn_TX_WR, SO_REG_WR, TX_Ptr + Uint16 (Chunk));
        Write_Reg (Sn_CR, SO_REG_WR, CR_SEND);

        Timeout := 0;
        loop
            IR := Read_Reg (16#0002#, SO_REG_RD);
            exit when (IR and 16#10#) /= 0;

```

```

        exit when (IR and 16#08#) /= 0;
        exit when Timeout > 2000;
        Timeout := Timeout + 1;
        delay until Clock + Milliseconds (1);
    end loop;
    Write_Reg (16#0002#, SO_REG_WR, 16#FF#);

    Sent := Sent + Chunk;
end loop;
end Socket0_Send;

function Socket0_Recv (Max_Len : Natural) return Natural is
    RX_Size : Uint16;
    RX_Ptr   : Uint16;
    Read_Len : Natural;
begin
    RX_Size := Read_Reg16 (Sn_RX_RSR, SO_REG_RD);
    if RX_Size = 0 then
        return 0;
    end if;

    Read_Len := Natural'Min (Natural (RX_Size), Max_Len);
    RX_Ptr   := Read_Reg16 (Sn_RX_RD, SO_REG_RD);
    Read_Buf (RX_Ptr, SO_RX_RD, HTTP_Buffer (1 .. Read_Len));
    Write_Reg16 (Sn_RX_RD, SO_REG_WR, RX_Ptr + Uint16 (Read_Len));
    Write_Reg   (Sn_CR,      SO_REG_WR, CR_RECV);
    return Read_Len;
end Socket0_Recv;

-- server http

procedure HTTP_Server (Port : Uint16 := 80) is

    HTML : constant String :=
        "HTTP/1.1 200 OK" & ASCII.CR & ASCII.LF &
        "Content-Type: text/html; charset=utf-8" & ASCII.CR & ASCII.LF
        &
        "Connection: close" & ASCII.CR & ASCII.LF &
        "Content-Length: 31" & ASCII.CR & ASCII.LF &
        ASCII.CR & ASCII.LF &
        "<h1>Pagina de prueba</h1>";
    -- Content-Length = 25 chars de <h1>Pagina de prueba</h1>
    -- Ajusta si cambias el body

```

```
HTML_Buf : Uint8_Array (1 .. HTML'Length);
Status    : Uint8;
Timeout   : Natural;
Got_Data  : Boolean;

begin

  for I in HTML'Range loop
    HTML_Buf (I - HTML'First + 1) := Character'Pos (HTML (I));
  end loop;

  USART_Driver.Send_Line ("HTTP: escuchando en puerto " &
    Integer'Image (Natural (Port)));

  Socket0_Listen (Port);

  Timeout := 0;
  loop
    Status := Socket0_Status;
    exit when Status = SOCK_ESTABLISHED;
    exit when Timeout > 30_000;
    Timeout := Timeout + 1;
    delay until Clock + Milliseconds (1);
  end loop;

  if Socket0_Status /= SOCK_ESTABLISHED then
    USART_Driver.Send_Line ("HTTP: timeout esperando conexion");
    Socket0_Close;
    return;
  end if;

  USART_Driver.Send_Line ("HTTP: cliente conectado");

  Got_Data := False;
  Timeout := 0;
  loop
    if Socket0_Recv (HTTP_Buffer'Length) > 0 then
      Got_Data := True;
      exit;
    end if;

    Status := Socket0_Status;
```

```

        exit when Status = SOCK_CLOSE_WAIT;
        exit when Timeout > 5000;
        Timeout := Timeout + 1;
        delay until Clock + Milliseconds (1);
    end loop;

    if Got_Data then

        USART_Driver.Send_Line ("HTTP: peticion recibida");
    end if;

    Socket0_Send (HTML_Buf);
    USART_Driver.Send_Line ("HTTP: respuesta enviada");

    delay until Clock + Milliseconds (10);
    Socket0_Close;
    USART_Driver.Send_Line ("HTTP: conexion cerrada");

end HTTP_Server;

procedure HTTP_Server_mDNS (Port : Uint16 := 80 ;
    Hostname:String;Content:String) is

    HTML : constant String :=
        "HTTP/1.1 200 OK" & ASCII.CR & ASCII.LF &
        "Content-Type: text/html; charset=utf-8" & ASCII.CR & ASCII.LF
        &
        "Connection: close" & ASCII.CR & ASCII.LF &
        "Content-Length: 31" & ASCII.CR & ASCII.LF &
        ASCII.CR & ASCII.LF &
        Content;
    -- Content-Length = 25 chars de <h1>Pagina de prueba</h1>
    -- Ajusta si cambias el body

    HTML_Buf : Uint8_Array (1 .. HTML'Length);
    Status    : Uint8;
    Timeout   : Natural;
    Got_Data  : Boolean;

```

```
begin
  -- Convertir String a Uint8_Array
  for I in HTML'Range loop
    HTML_Buf (I - HTML'First + 1) := Character'Pos (HTML (I));
  end loop;

  USART_Driver.Send_Line ("HTTP: escuchando en puerto " &
    Integer'Image (Natural (Port)));

  Socket0_Listen (Port);

  Timeout := 0;
  loop
    Status := Socket0_Status;
    exit when Status = SOCK_ESTABLISHED;
    exit when Timeout > 30_000;    -- 30 segundos máximo
    Timeout := Timeout + 1;
    mDNS_Loop(Hostname);
    delay until Clock + Milliseconds (1);
  end loop;

  if Socket0_Status /= SOCK_ESTABLISHED then
    USART_Driver.Send_Line ("HTTP: timeout esperando conexion");
    Socket0_Close;
    return;
  end if;

  USART_Driver.Send_Line ("HTTP: cliente conectado");

  Got_Data := False;
  Timeout := 0;
  loop
    if Socket0_Recv (HTTP_Buffer'Length) > 0 then
      Got_Data := True;
      exit;
    end if;

    Status := Socket0_Status;
    exit when Status = SOCK_CLOSE_WAIT;
    exit when Timeout > 5000;
    Timeout := Timeout + 1;
    delay until Clock + Milliseconds (1);
```



```
end loop;

if Got_Data then

    USART_Driver.Send_Line ("HTTP: peticion recibida");
end if;

Socket0_Send (HTML_Buf);
USART_Driver.Send_Line ("HTTP: respuesta enviada");

delay until Clock + Milliseconds (10);
Socket0_Close;
USART_Driver.Send_Line ("HTTP: conexion cerrada");

end HTTP_Server_mDNS;

procedure Socket2_Open_mDNS is
    -- IP multicast mDNS: 224.0.0.251
    MDNS_IP : constant Uint8_Array (1 .. 4) := (224, 0, 0, 251);
    -- MAC multicast derivada de IP: 01:00:5E:00:00:FB
    MDNS_MAC : constant Uint8_Array (1 .. 6) :=
        (16#01#, 16#00#, 16#5E#, 16#00#, 16#00#, 16#FB#);
begin
    Write_Reg (Sn_CR, S2_REG_WR, CR_CLOSE);
    delay until Clock + Milliseconds (5);

    Write_Reg (Sn_MR, S2_REG_WR, 16#82#);
    Write_Reg16 (Sn_PORT, S2_REG_WR, 5353);
    Write_Buf (Sn_DHAR, S2_REG_WR, MDNS_MAC);
    Write_Buf (Sn_DIPR, S2_REG_WR, MDNS_IP);
    Write_Reg16 (Sn_DPORT, S2_REG_WR, 5353);
    Write_Reg (Sn_CR, S2_REG_WR, CR_OPEN);
    delay until Clock + Milliseconds (10);
end Socket2_Open_mDNS;

procedure Socket2_Close is
begin
    Write_Reg (Sn_CR, S2_REG_WR, CR_CLOSE);
end Socket2_Close;
```

```

procedure mDNS_Send_Response (Query_ID : Uint16; Hostname : String)
is
  MDNS_IP   : constant Uint8_Array (1 .. 4) := (224, 0, 0, 251);
  MDNS_MAC  : constant Uint8_Array (1 .. 6) :=
    (16#01#, 16#00#, 16#5E#, 16#00#, 16#00#, 16#FB#);
  My_IP     : constant Uint8_Array (1 .. 4) := (192, 168, 1, 180);

  Name_Buf  : Uint8_Array (1 .. 64);
  Name_Len  : Natural;

  B         : Uint8_Array (1 .. 128) := (others => 0);
  Off       : Natural := 1;
  TX_Ptr    : Uint16;

  procedure Put16 (V : Uint16) is
  begin
    B (Off) := Uint8 (V / 256); B (Off + 1) := Uint8 (V mod 256);
    Off := Off + 2;
  end Put16;

  procedure Put32 (V : Uint32) is
  begin
    B (Off)      := Uint8 ((V / 16#10000000#) mod 256);
    B (Off + 1) := Uint8 ((V /      16#100000#) mod 256);
    B (Off + 2) := Uint8 ((V /      16#100#) mod 256);
    B (Off + 3) := Uint8 (V mod 256);
    Off := Off + 4;
  end Put32;

begin
  -- Codificar el hostname ("stm32.local" -> 05 stm32 05 local 00)
  Encode_DNS_Name (Hostname, Name_Buf, Name_Len);

  Put16 (Query_ID);      -- ID
  Put16 (16#8400#);      -- Flags: QR=1, AA=1
  Put16 (0);             -- QDCOUNT = 0
  Put16 (1);             -- ANCOUNT = 1
  Put16 (0);             -- NSCOUNT = 0
  Put16 (0);             -- ARCOUNT = 0

  for I in 1 .. Name_Len loop
    B (Off) := Name_Buf (I); Off := Off + 1;
  end loop;

```

```

Put16 (1);          -- TYPE = A
Put16 (16#8001#);   -- CLASS = IN | FLUSH
Put32 (120);        -- TTL = 120s
Put16 (4);          -- RDLENGTH = 4

B (Off) := My_IP (1); B (Off + 1) := My_IP (2);
B (Off + 2) := My_IP (3); B (Off + 3) := My_IP (4);
Off := Off + 4;

-- Enviar por socket 2 UDP multicast
Write_Buf (Sn_DHAR, S2_REG_WR, MDNS_MAC);
Write_Buf (Sn_DIPR, S2_REG_WR, MDNS_IP);
Write_Reg16 (Sn_DPORT, S2_REG_WR, 5353);

TX_Ptr := Read_Reg16 (Sn_TX_WR, S2_REG_RD);
Write_Buf (TX_Ptr and 16#07FF#, S2_TX_WR, B (1 .. Off - 1));
Write_Reg16 (Sn_TX_WR, S2_REG_WR, TX_Ptr + Uint16 (Off - 1));
Write_Reg (Sn_CR, S2_REG_WR, CR_SEND);

delay until Clock + Milliseconds (5);
Write_Reg (16#0002#, S2_REG_WR, 16#FF#);

end mDNS_Send_Response;

function mDNS_Parse_Query return Boolean is

    DNS      : Natural := 9;
    Flags    : Uint16;
    Qdcount  : Uint16;
    Off      : Natural;
    Len      : Uint8;

    -- Nombre esperado codificado: 05 stm32 05 local 00
    Target : constant Uint8_Array (1 .. 13) :=
        (16#05#,
         16#73#, 16#74#, 16#6D#, 16#33#, 16#32#,
         16#05#,
         16#6C#, 16#6F#, 16#63#, 16#61#, 16#6C#,
         16#00#);

begin
    if MDNS_Buffer'Length < DNS + 12 then

```

```
        return False;
    end if;

    Flags := Uint16 (MDNS_Buffer (DNS + 2)) * 256 +
              Uint16 (MDNS_Buffer (DNS + 3));
    if (Flags and 16#8000#) /= 0 then
        return False;
    end if;

    Qdcount := Uint16 (MDNS_Buffer (DNS + 4)) * 256 +
                Uint16 (MDNS_Buffer (DNS + 5));
    if Qdcount = 0 then
        return False;
    end if;

    Off := DNS + 12;

    for I in Target'Range loop
        exit when Off > MDNS_Buffer'Length;
        if MDNS_Buffer (Off) /= Target (I) then
            return False;
        end if;
        Off := Off + 1;
    end loop;

    if Off + 1 > MDNS_Buffer'Length then
        return False;
    end if;
    declare
        Qtype : constant Uint16 :=
            Uint16 (MDNS_Buffer (Off) and 16#7F#) * 256 +
            Uint16 (MDNS_Buffer (Off + 1));

    begin
        if Qtype /= 1 then
            return False;
        end if;
    end;

    return True;
end mDNS_Parse_Query;
```

```

procedure mDNS_Announce (Hostname:String )is
begin
    Socket2_Open_mDNS;

    mDNS_Send_Response (0, Hostname);
    delay until Clock + Milliseconds (250);
    mDNS_Send_Response (0 ,Hostname);
    USART_Driver.Send_Line ("mDNS: anunciado como " & Hostname);
end mDNS_Announce;

procedure mDNS_Loop (Hostname: String) is
    RX_Size  : Uint16;
    RX_Ptr    : Uint16;
    Read_Len  : Natural;
    Query_ID  : Uint16;
begin
    RX_Size := Read_Reg16 (Sn_RX_RSR, S2_REG_RD);
    if RX_Size = 0 then
        return;
    end if;

    Read_Len := Natural'Min (Natural (RX_Size), MDNS_Buffer'Length);
    RX_Ptr := Read_Reg16 (Sn_RX_RD, S2_REG_RD);  -- Sn_RX_RD offset
                                                0x0028
    Read_Buf (RX_Ptr, S2_RX_RD, MDNS_Buffer (1 .. Read_Len));
    Write_Reg16 (Sn_RX_RD, S2_REG_WR,
                RX_Ptr + Uint16 (Read_Len));
    Write_Reg (Sn_CR, S2_REG_WR, CR_RECV);

    if mDNS_Parse_Query then
        Query_ID := Uint16 (MDNS_Buffer (9)) * 256 +
                      Uint16 (MDNS_Buffer (10));
        mDNS_Send_Response (Query_ID, Hostname);
        USART_Driver.Send_Line ("mDNS: respondido a query");
    end if;
end mDNS_Loop;

procedure Encode_DNS_Name (Hostname      :      String;

```

```

                                Result      : out Uint8_Array;
                                Result_Len  : out Natural) is
    Off      : Natural := Result'First;
    Label_Start : Natural := Hostname'First;
    Len      : Natural;
begin
    for I in Hostname'First .. Hostname'Last + 1 loop
        if I = Hostname'Last + 1 or else Hostname (I) = '.' then
            Len := I - Label_Start;
            Result (Off) := Uint8 (Len);
            Off := Off + 1;
            for J in Label_Start .. I - 1 loop
                Result (Off) := Character'Pos (Hostname (J));
                Off := Off + 1;
            end loop;
            Label_Start := I + 1;
        end if;
    end loop;
    Result (Off) := 0;
    Result_Len := Off - Result'First + 1;
end Encode_DNS_Name;

```

-- DHCP

```

procedure Socket3_Open_DHCP is
    Broadcast : constant Uint8_Array (1 .. 4) := (255, 255, 255, 255);
    Bcast_MAC : constant Uint8_Array (1 .. 6) :=
        (16#FF#, 16#FF#, 16#FF#, 16#FF#, 16#FF#, 16#FF#);
    Status     : Uint8;
begin
    Write_Reg (Sn_CR, S3_REG_WR, CR_CLOSE);
    delay until Clock + Milliseconds (10);

    Write_Reg (Sn_MR, S3_REG_WR, MR_UDP);    -- 0x02
    Write_Reg16 (Sn_PORT, S3_REG_WR, 68);
    Write_Reg (Sn_CR, S3_REG_WR, CR_OPEN);
    delay until Clock + Milliseconds (20);

    Status := Read_Reg (Sn_SR, S3_REG_RD);

```

```
Print_Hex ("DHCP socket status: ", Status);

Write_Buf (Sn_DHAR, S3_REG_WR, Bcast_MAC);
Write_Buf (Sn_DIPR, S3_REG_WR, Broadcast);
Write_Reg16 (Sn_DPORT, S3_REG_WR, 67);
end Socket3_Open_DHCP;

procedure Socket3_Close is
begin
    Write_Reg (Sn_CR, S3_REG_WR, CR_CLOSE);
end Socket3_Close;

procedure DHCP_Send (Data : Uint8_Array; Len : Natural) is
    TX_Ptr : Uint16;
    TX_Free : Uint16;
    Timeout : Natural := 0;
    IR : Uint8;
begin
    TX_Free := Read_Reg16 (Sn_TX_FSR, S3_REG_RD);
    Print_Hex ("DHCP TX_FSR_H: ", Uint8 (TX_Free / 256));
    Print_Hex ("DHCP TX_FSR_L: ", Uint8 (TX_Free mod 256));
    -- Debe ser 0x0800 (2048 bytes libres)

    TX_Ptr := Read_Reg16 (Sn_TX_WR, S3_REG_RD);
    Write_Buf (TX_Ptr and 16#07FF#, S3_TX_WR, Data (1 .. Len));
    Write_Reg16 (Sn_TX_WR, S3_REG_WR, TX_Ptr + Uint16 (Len));
    Write_Reg (Sn_CR, S3_REG_WR, CR_SEND);

    loop
        IR := Read_Reg (16#0002#, S3_REG_RD);
        exit when (IR and 16#10#) /= 0;
        exit when (IR and 16#08#) /= 0;
        exit when Timeout > 3000;
        Timeout := Timeout + 1;
        delay until Clock + Milliseconds (1);
    end loop;

    Print_Hex ("DHCP Sn_IR: ", IR);
    if (IR and 16#10#) /= 0 then
        USART_Driver.Send_Line ("DHCP: SEND_OK");
    elsif (IR and 16#08#) /= 0 then
        USART_Driver.Send_Line ("DHCP: SEND_TIMEOUT");
    end if;
end DHCP_Send;
```

```

else
    USART_Driver.Send_Line ("DHCP: SEND sin respuesta");
end if;

Write_Reg (16#0002#, S3_REG_WR, 16#FF#);
end DHCP_Send;
function DHCP_Recv (Timeout_MS : Natural) return Natural is
    Deadline : constant Time := Clock + Milliseconds (Timeout_MS);
    RX_Size   : Uint16;
    RX_Ptr     : Uint16;
    Read_Len   : Natural;
begin
    loop
        exit when Clock >= Deadline;
        RX_Size := Read_Reg16 (Sn_RX_RSR, S3_REG_RD);
        if RX_Size > 0 then
            Read_Len := Natural'Min (Natural (RX_Size),
                                     DHCP_Buffer'Length);
            RX_Ptr := Read_Reg16 (Sn_RX_RD, S3_REG_RD);
            Read_Buf (RX_Ptr, S3_RX_RD, DHCP_Buffer (1 .. Read_Len));
            Write_Reg16 (Sn_RX_RD, S3_REG_WR, RX_Ptr + Uint16
                        (Read_Len));
            Write_Reg (Sn_CR, S3_REG_WR, CR_RECV);
            return Read_Len;
        end if;
        delay until Clock + Milliseconds (5);
    end loop;
    return 0;
end DHCP_Recv;

function DHCP_Request (Timeout_MS : Natural := 10_000) return
    DHCP_Result is

    Zero_IP : constant Uint8_Array (1 .. 4) := (0, 0, 0, 0);
    Empty    : constant DHCP_Result :=
        (Success => False,
         IP      => Zero_IP,
         Subnet  => Zero_IP,
         Gateway => Zero_IP,
         DNS     => Zero_IP,
         Lease   => 0);

    My_MAC : Uint8_Array (1 .. 6);

```



```

XID      : constant Uint32 := 16#DEADBEEF#;

procedure Build_DHCP (Msg_Type : Uint8; Server_IP : Uint8_Array;
                     Req_IP   : Uint8_Array; Len : out Natural) is
  Off : Natural := 1;

  procedure Put8 (V : Uint8) is
  begin
    DHCP_Buffer (Off) := V; Off := Off + 1;
  end Put8;
  procedure Put16 (V : Uint16) is
  begin
    Put8 (Uint8 (V / 256)); Put8 (Uint8 (V mod 256));
  end Put16;
  procedure Put32 (V : Uint32) is
  begin
    Put8 (Uint8 ((V / 16#1000000#) mod 256));
    Put8 (Uint8 ((V /      16#10000#) mod 256));
    Put8 (Uint8 ((V /      16#100#) mod 256));
    Put8 (Uint8 ( V                mod 256));
  end Put32;

begin
  DHCP_Buffer := (others => 0);

  Put8 (1);           -- op      = BOOTREQUEST
  Put8 (1);           -- htype   = Ethernet
  Put8 (6);           -- hlen    = 6
  Put8 (0);           -- hops    = 0
  Put32 (XID);        -- xid     Off=9 tras esto
  Put16 (0);          -- secs
  Put16 (16#8000#);    -- flags   = BROADCAST Off=13
  Off := Off + 16;     -- ciaddr+yiaddr+siaddr+giaddr = 16 bytes a 0
  for I in My_MAC'Range loop
    Put8 (My_MAC (I));
  end loop;           -- chaddr MAC (Off=35)
  Off := Off + 10;     -- padding MAC a 0
  Off := Off + 64;     -- sname a 0
  Off := Off + 128;    -- file a 0
                     -- Off = 237 aquí

  Put8 (99);
  Put8 (130);

```

```

Put8 (83);
Put8 (99);          -- Off = 241

-- 53: DHCP Message Type
Put8 (53); Put8 (1); Put8 (Msg_Type);

-- 61: Client Identifier
Put8 (61); Put8 (7); Put8 (1);
for I in My_MAC'Range loop Put8 (My_MAC (I)); end loop;

-- 55: Parameter Request List
Put8 (55); Put8 (4);
Put8 (1);  -- Subnet Mask
Put8 (3);  -- Router
Put8 (6);  -- DNS
Put8 (51); -- Lease Time

if Msg_Type = 3 then
  Put8 (54); Put8 (4);
  for I in Server_IP'Range loop Put8 (Server_IP (I)); end loop;
  Put8 (50); Put8 (4);
  for I in Req_IP'Range loop Put8 (Req_IP (I)); end loop;
end if;

Put8 (255);  -- END
Len := Off - 1;
end Build_DHCP;

procedure Parse_DHCP (Recv_Len  : Natural;
                      Msg_Type  : out Uint8;
                      Offered   : out Uint8_Array;
                      Server_IP : out Uint8_Array;
                      Subnet    : out Uint8_Array;
                      Gateway   : out Uint8_Array;
                      DNS       : out Uint8_Array;
                      Lease     : out Uint32) is

  Base : constant Natural := 9;
  Off  : Natural;
  Opt  : Uint8;
  Olen : Uint8;
  XID_OK : Boolean;

```

```
begin
    Msg_Type := 0;
    Offered   := (others => 0);
    Server_IP := (others => 0);
    Subnet    := (others => 0);
    Gateway   := (others => 0);
    DNS       := (others => 0);
    Lease     := 0;

    if Recv_Len < Base + 240 then return; end if;

    XID_OK :=
        DHCP_Buffer (Base + 4) = Uint8 ((XID / 16#1000000#) mod
            256) and then
        DHCP_Buffer (Base + 5) = Uint8 ((XID /      16#10000#) mod
            256) and then
        DHCP_Buffer (Base + 6) = Uint8 ((XID /      16#100#) mod
            256) and then
        DHCP_Buffer (Base + 7) = Uint8 ( XID                      mod
            256);

    if not XID_OK then return; end if;

    for I in 1 .. 4 loop
        Offered (I) := DHCP_Buffer (Base + 15 + I);
    end loop;

    -- Saltar cabecera BOOTP (236 bytes) + magic cookie (4) = 240
    Off := Base + 240;

    loop
        exit when Off > Recv_Len;
        Opt := DHCP_Buffer (Off); Off := Off + 1;

        exit when Opt = 255;
        if Opt = 0 then
            null;
        else
            exit when Off > Recv_Len;
            Olen := DHCP_Buffer (Off); Off := Off + 1;
        end if;
    end loop;
```

```

case Opt is
when 53 =>
    Msg_Type := DHCP_Buffer (Off);
when 54 =>
    for I in 1 .. 4 loop
        Server_IP (I) := DHCP_Buffer (Off + I - 1);
    end loop;
when 1 =>
    for I in 1 .. 4 loop
        Subnet (I) := DHCP_Buffer (Off + I - 1);
    end loop;
when 3 =>
    for I in 1 .. 4 loop
        Gateway (I) := DHCP_Buffer (Off + I - 1);
    end loop;
when 6 =>
    for I in 1 .. 4 loop
        DNS (I) := DHCP_Buffer (Off + I - 1);
    end loop;
when 51 =>
    Lease :=
        Uint32 (DHCP_Buffer (Off))      * 16#1000000# +
        Uint32 (DHCP_Buffer (Off + 1)) *   16#10000# +
        Uint32 (DHCP_Buffer (Off + 2)) *   16#100# +
        Uint32 (DHCP_Buffer (Off + 3));
when others => null;
end case;

Off := Off + Natural (Olen);
end if;
end loop;
end Parse_DHCP;

```

```

Pkt_Len    : Natural;
Recv_Len   : Natural;
Msg_Type    : Uint8;
Offered     : Uint8_Array (1 .. 4) := (others => 0);
Server_IP   : Uint8_Array (1 .. 4) := (others => 0);
Subnet      : Uint8_Array (1 .. 4) := (others => 0);
Gateway     : Uint8_Array (1 .. 4) := (others => 0);
DNS         : Uint8_Array (1 .. 4) := (others => 0);
Lease       : Uint32 := 0;
Zero_IP4    : constant Uint8_Array (1 .. 4) := (others => 0);

```

```
begin
    Get_MAC (My_MAC);
    Socket3_Open_DHCP;

    USART_Driver.Send_Line ("DHCP: enviando DISCOVER...");

    Build_DHCP (1, Zero_IP4, Zero_IP4, Pkt_Len);
    DHCP_Send (DHCP_Buffer, Pkt_Len);

    Recv_Len := DHCP_Recv (Timeout_MS / 2);
    if Recv_Len = 0 then
        USART_Driver.Send_Line ("DHCP: timeout esperando OFFER");
        Socket3_Close;
        return Empty;
    end if;

    Parse_DHCP (Recv_Len, Msg_Type, Offered, Server_IP,
                Subnet, Gateway, DNS, Lease);

    if Msg_Type /= 2 then
        USART_Driver.Send_Line ("DHCP: no se recibio OFFER");
        Socket3_Close;
        return Empty;
    end if;

    USART_Driver.Send_Line ("DHCP: OFFER recibida, enviando
        REQUEST...");

    --request
    Build_DHCP (3, Server_IP, Offered, Pkt_Len);
    DHCP_Send (DHCP_Buffer, Pkt_Len);

    Recv_Len := DHCP_Recv (Timeout_MS / 2);
    if Recv_Len = 0 then
        USART_Driver.Send_Line ("DHCP: timeout esperando ACK");
        Socket3_Close;
        return Empty;
    end if;

    Parse_DHCP (Recv_Len, Msg_Type, Offered, Server_IP,
```

```
        Subnet, Gateway, DNS, Lease);

Socket3_Close;

if Msg_Type = 5 then
    USART_Driver.Send_Line ("DHCP: ACK recibido configuracion
        obtenida");
    return (Success => True,
        IP      => Offered,
        Subnet  => Subnet,
        Gateway => Gateway,
        DNS     => DNS,
        Lease   => Lease);
elsif Msg_Type = 6 then
    USART_Driver.Send_Line ("DHCP: NAK recibido rechazado por
        servidor");
    return Empty;
else
    USART_Driver.Send_Line ("DHCP: respuesta inesperada");
    return Empty;
end if;

end DHCP_Request;

end W5500;
```

Listing 19: w5500.adb — Cuerpo del controlador Ethernet

Paquete SSD1306 (Pantalla OLED)

```
with stm32f446; use stm32f446;

package SSD1306 is

    type Display_Model is (SSD1306_128x64, SSD1306_128x32,
        SSD1306_96x16);
    Model : constant Display_Model := SSD1306_128x64;
    -- cambiar_modelo

    WIDTH  : constant Uint8 :=
        (case Model is
            when SSD1306_128x64 => 128,
            when SSD1306_128x32 => 128,
            when SSD1306_96x16  => 96);

    HEIGHT : constant Uint8 :=
        (case Model is
            when SSD1306_128x64 => 64,
            when SSD1306_128x32 => 32,
            when SSD1306_96x16  => 16);

    PAGES  : constant Uint8 :=
        (case Model is
            when SSD1306_128x64 => 8,
            when SSD1306_128x32 => 4,
            when SSD1306_96x16  => 2);

    BUFFER_SIZE : constant Natural :=
        (case Model is
            when SSD1306_128x64 => 1024,
            when SSD1306_128x32 => 512,
            when SSD1306_96x16  => 192);

    SSD1306_ADDR : constant Uint8 := 16#3C#;

    procedure Init;
    procedure Clear_Display;
    procedure Display_On;
    procedure Display_Off;
```

```

procedure Set_Contrast   (Value      : Uint8);
procedure Set_Inverse    (Inverse    : Boolean);
procedure Draw_Pixel     (X : Uint8; Y : Uint8; b : Boolean);
procedure Update_Display;
procedure Put_Char       (X : Uint8; Y : Uint8; C : Character);
procedure Put_String     (X : Uint8; Y : Uint8; S : String);
procedure Draw_Line      (X0, Y0, X1, Y1 : Uint8; b : Boolean);
procedure Draw_Rect      (X, Y, W, H : Uint8; b : Boolean);
procedure Fill_Rect      (X, Y, W, H : Uint8; b : Boolean);

```

```

type Display_Buffer is array (0 .. BUFFER_SIZE - 1) of Uint8;
Frame_Buffer : Display_Buffer := (others => 0);

```

```

procedure Write_Command   (Cmd : Uint8);
procedure Write_Data      (Data : Uint8);
procedure Write_Data_Buffer (Data : Display_Buffer; Len :
    Natural);

```

```

-- Comandos

```

```

SSD1306_SETCONTRAST           : constant Uint8 := 16#81#;
SSD1306_DISPLAYALLON_RESUME  : constant Uint8 := 16#A4#;
SSD1306_DISPLAYALLON         : constant Uint8 := 16#A5#;
SSD1306_NORMALDISPLAY        : constant Uint8 := 16#A6#;
SSD1306_INVERTDISPLAY        : constant Uint8 := 16#A7#;
SSD1306_DISPLAYOFF           : constant Uint8 := 16#AE#;
SSD1306_DISPLAYON            : constant Uint8 := 16#AF#;
SSD1306_SETDISPLAYOFFSET     : constant Uint8 := 16#D3#;
SSD1306_SETCOMPINS           : constant Uint8 := 16#DA#;
SSD1306_SETVCOMDETECT        : constant Uint8 := 16#DB#;
SSD1306_SETDISPLAYCLOCKDIV   : constant Uint8 := 16#D5#;
SSD1306_SETPRECHARGE         : constant Uint8 := 16#D9#;
SSD1306_SETMULTIPLEX         : constant Uint8 := 16#A8#;
SSD1306_SETLOWCOLUMN         : constant Uint8 := 16#00#;
SSD1306_SETHIGHCOLUMN        : constant Uint8 := 16#10#;
SSD1306_SETSTARTLINE         : constant Uint8 := 16#40#;
SSD1306_MEMORYMODE           : constant Uint8 := 16#20#;
SSD1306_COLUMNADDR           : constant Uint8 := 16#21#;
SSD1306_PAGEADDR             : constant Uint8 := 16#22#;
SSD1306_COMSCANINC           : constant Uint8 := 16#C0#;
SSD1306_COMSCANDEC           : constant Uint8 := 16#C8#;
SSD1306_SEGREMAP              : constant Uint8 := 16#A0#;
SSD1306_CHARGEPUMP           : constant Uint8 := 16#8D#;

```



```

SSD1306_DEACTIVATE_SCROLL    : constant Uint8 := 16#2E#;

-- Valores que cambian según modelo
MULTIPLEX_VAL : constant Uint8 :=
  (case Model is
    when SSD1306_128x64 => 16#3F#,
    when SSD1306_128x32 => 16#1F#,
    when SSD1306_96x16  => 16#0F#);

COMPINS_VAL : constant Uint8 :=
  (case Model is
    when SSD1306_128x64 => 16#12#,
    when SSD1306_128x32 => 16#02#,
    when SSD1306_96x16  => 16#02#);

CONTRAST_VAL : constant Uint8 :=
  (case Model is
    when SSD1306_128x64 => 16#CF#,
    when SSD1306_128x32 => 16#8F#,
    when SSD1306_96x16  => 16#AF#);

PRECHARGE_VAL : constant Uint8 :=
  (case Model is
    when SSD1306_128x64 => 16#F1#,
    when SSD1306_128x32 => 16#F1#,
    when SSD1306_96x16  => 16#22#);

PAGE_MAX : constant Uint8 :=
  (case Model is
    when SSD1306_128x64 => 7,
    when SSD1306_128x32 => 3,
    when SSD1306_96x16  => 1);

COL_MAX : constant Uint8 :=
  (case Model is
    when SSD1306_128x64 => 127,
    when SSD1306_128x32 => 127,
    when SSD1306_96x16  => 95);

-- Fuente 5x7
type Font_Col is array (0 .. 4) of Uint8;
type Font_Type is array (Character range '!' .. '~') of Font_Col;
Font : constant Font_Type := (
  '!' => (16#00#, 16#00#, 16#4F#, 16#00#, 16#00#),

```

' " ' => (16#00#, 16#03#, 16#00#, 16#03#, 16#00#),
 ' # ' => (16#14#, 16#3E#, 16#14#, 16#3E#, 16#14#),
 ' \$ ' => (16#24#, 16#2A#, 16#7F#, 16#2A#, 16#12#),
 ' % ' => (16#63#, 16#13#, 16#08#, 16#64#, 16#63#),
 ' & ' => (16#36#, 16#49#, 16#55#, 16#22#, 16#50#),
 ' ' ' => (16#00#, 16#00#, 16#07#, 16#00#, 16#00#),
 ' (' => (16#00#, 16#1C#, 16#22#, 16#41#, 16#00#),
 ') ' => (16#00#, 16#41#, 16#22#, 16#1C#, 16#00#),
 ' * ' => (16#0A#, 16#04#, 16#1F#, 16#04#, 16#0A#),
 ' + ' => (16#04#, 16#04#, 16#1F#, 16#04#, 16#04#),
 ' , ' => (16#50#, 16#30#, 16#00#, 16#00#, 16#00#),
 ' - ' => (16#08#, 16#08#, 16#08#, 16#08#, 16#08#),
 ' . ' => (16#60#, 16#60#, 16#00#, 16#00#, 16#00#),
 ' / ' => (16#00#, 16#60#, 16#1C#, 16#03#, 16#00#),
 ' 0 ' => (16#3E#, 16#41#, 16#49#, 16#41#, 16#3E#),
 ' 1 ' => (16#00#, 16#02#, 16#7F#, 16#00#, 16#00#),
 ' 2 ' => (16#46#, 16#61#, 16#51#, 16#49#, 16#46#),
 ' 3 ' => (16#21#, 16#49#, 16#4D#, 16#4B#, 16#31#),
 ' 4 ' => (16#18#, 16#14#, 16#12#, 16#7F#, 16#10#),
 ' 5 ' => (16#4F#, 16#49#, 16#49#, 16#49#, 16#31#),
 ' 6 ' => (16#3E#, 16#51#, 16#49#, 16#49#, 16#32#),
 ' 7 ' => (16#01#, 16#01#, 16#71#, 16#0D#, 16#03#),
 ' 8 ' => (16#36#, 16#49#, 16#49#, 16#49#, 16#36#),
 ' 9 ' => (16#26#, 16#49#, 16#49#, 16#49#, 16#3E#),
 ' : ' => (16#00#, 16#33#, 16#33#, 16#00#, 16#00#),
 ' ; ' => (16#00#, 16#53#, 16#33#, 16#00#, 16#00#),
 ' < ' => (16#00#, 16#08#, 16#14#, 16#22#, 16#41#),
 ' = ' => (16#14#, 16#14#, 16#14#, 16#14#, 16#14#),
 ' > ' => (16#41#, 16#22#, 16#14#, 16#08#, 16#00#),
 ' ? ' => (16#06#, 16#01#, 16#51#, 16#09#, 16#06#),
 ' @ ' => (16#3E#, 16#41#, 16#49#, 16#15#, 16#1E#),
 ' A ' => (16#78#, 16#16#, 16#11#, 16#16#, 16#78#),
 ' B ' => (16#7F#, 16#49#, 16#49#, 16#49#, 16#36#),
 ' C ' => (16#3E#, 16#41#, 16#41#, 16#41#, 16#22#),
 ' D ' => (16#7F#, 16#41#, 16#41#, 16#41#, 16#3E#),
 ' E ' => (16#7F#, 16#49#, 16#49#, 16#49#, 16#49#),
 ' F ' => (16#7F#, 16#09#, 16#09#, 16#09#, 16#09#),
 ' G ' => (16#3E#, 16#41#, 16#41#, 16#49#, 16#7B#),
 ' H ' => (16#7F#, 16#08#, 16#08#, 16#08#, 16#7F#),
 ' I ' => (16#00#, 16#00#, 16#7F#, 16#00#, 16#00#),
 ' J ' => (16#38#, 16#40#, 16#40#, 16#41#, 16#3F#),
 ' K ' => (16#7F#, 16#08#, 16#08#, 16#14#, 16#63#),
 ' L ' => (16#7F#, 16#40#, 16#40#, 16#40#, 16#40#),
 ' M ' => (16#7F#, 16#06#, 16#18#, 16#06#, 16#7F#),

'N' => (16#7F#, 16#06#, 16#18#, 16#60#, 16#7F#),
 'O' => (16#3E#, 16#41#, 16#41#, 16#41#, 16#3E#),
 'P' => (16#7F#, 16#09#, 16#09#, 16#09#, 16#06#),
 'Q' => (16#3E#, 16#41#, 16#51#, 16#21#, 16#5E#),
 'R' => (16#7F#, 16#09#, 16#19#, 16#29#, 16#46#),
 'S' => (16#26#, 16#49#, 16#49#, 16#49#, 16#32#),
 'T' => (16#01#, 16#01#, 16#7F#, 16#01#, 16#01#),
 'U' => (16#3F#, 16#40#, 16#40#, 16#40#, 16#7F#),
 'V' => (16#0F#, 16#30#, 16#40#, 16#30#, 16#0F#),
 'W' => (16#1F#, 16#60#, 16#1C#, 16#60#, 16#1F#),
 'X' => (16#63#, 16#14#, 16#08#, 16#14#, 16#63#),
 'Y' => (16#03#, 16#04#, 16#78#, 16#04#, 16#03#),
 'Z' => (16#61#, 16#51#, 16#49#, 16#45#, 16#43#),
 '[' => (16#00#, 16#7F#, 16#41#, 16#00#, 16#00#),
 '\' => (16#03#, 16#1C#, 16#60#, 16#00#, 16#00#),
 ']' => (16#00#, 16#41#, 16#7F#, 16#00#, 16#00#),
 '^' => (16#0C#, 16#02#, 16#01#, 16#02#, 16#0C#),
 '_' => (16#40#, 16#40#, 16#40#, 16#40#, 16#40#),
 '`' => (16#00#, 16#01#, 16#02#, 16#04#, 16#00#),
 'a' => (16#20#, 16#54#, 16#54#, 16#54#, 16#78#),
 'b' => (16#7F#, 16#48#, 16#44#, 16#44#, 16#38#),
 'c' => (16#38#, 16#44#, 16#44#, 16#44#, 16#44#),
 'd' => (16#38#, 16#44#, 16#44#, 16#48#, 16#7F#),
 'e' => (16#38#, 16#54#, 16#54#, 16#54#, 16#18#),
 'f' => (16#08#, 16#7E#, 16#09#, 16#09#, 16#00#),
 'g' => (16#0C#, 16#52#, 16#52#, 16#54#, 16#3E#),
 'h' => (16#7F#, 16#08#, 16#04#, 16#04#, 16#78#),
 'i' => (16#00#, 16#00#, 16#7D#, 16#00#, 16#00#),
 'j' => (16#00#, 16#40#, 16#3D#, 16#00#, 16#00#),
 'k' => (16#7F#, 16#10#, 16#28#, 16#44#, 16#00#),
 'l' => (16#00#, 16#00#, 16#3F#, 16#40#, 16#00#),
 'm' => (16#7C#, 16#04#, 16#18#, 16#04#, 16#78#),
 'n' => (16#7C#, 16#08#, 16#04#, 16#04#, 16#78#),
 'o' => (16#38#, 16#44#, 16#44#, 16#44#, 16#38#),
 'p' => (16#7F#, 16#12#, 16#11#, 16#11#, 16#0E#),
 'q' => (16#0E#, 16#11#, 16#11#, 16#12#, 16#7F#),
 'r' => (16#00#, 16#7C#, 16#08#, 16#04#, 16#04#),
 's' => (16#48#, 16#54#, 16#54#, 16#54#, 16#24#),
 't' => (16#04#, 16#3E#, 16#44#, 16#44#, 16#00#),
 'u' => (16#3C#, 16#40#, 16#40#, 16#20#, 16#7C#),
 'v' => (16#1C#, 16#20#, 16#40#, 16#20#, 16#1C#),
 'w' => (16#1C#, 16#60#, 16#18#, 16#60#, 16#1C#),
 'x' => (16#44#, 16#28#, 16#10#, 16#28#, 16#44#),
 'y' => (16#46#, 16#28#, 16#10#, 16#08#, 16#06#),

```

        'z' => (16#44#, 16#64#, 16#54#, 16#4C#, 16#44#),
        '{' => (16#00#, 16#08#, 16#77#, 16#41#, 16#00#),
        '|' => (16#00#, 16#00#, 16#7F#, 16#00#, 16#00#),
        '}' => (16#00#, 16#41#, 16#77#, 16#08#, 16#00#),
        '~' => (16#10#, 16#08#, 16#18#, 16#10#, 16#08#)
    );

end SSD1306;

```

Listing 20: ssd1306.ads — Especificación del driver OLED

```

with Ada.Text_IO;

with I2C_Driver; use I2C_Driver;
with Ada.Real_Time; use Ada.Real_Time;
with USART_Driver; use USART_Driver;

package body SSD1306 is

    CONTROL_BYTE_CMD   : constant Uint8 := 16#00#;
    CONTROL_BYTE_DATA  : constant Uint8 := 16#40#;

    Bit_Masks : constant array (0 .. 7) of Uint8 :=
        (2#0000_0001#, 2#0000_0010#, 2#0000_0100#, 2#0000_1000#,
         2#0001_0000#, 2#0010_0000#, 2#0100_0000#, 2#1000_0000#);

    procedure Write_Command (Cmd : Uint8) is
        Cmd_Buffer : Uint8_Array (1 .. 2) := (CONTROL_BYTE_CMD, Cmd);
        Success    : Boolean;
    begin
        Success := Bus.Write_Buffer (SSD1306_ADDR, Cmd_Buffer);
    end Write_Command;

    procedure Write_Data (Data : Uint8) is
        Data_Buffer : Uint8_Array (1 .. 2) := (CONTROL_BYTE_DATA,
        Data);
        Success     : Boolean;
    begin
        Success := Bus.Write_Buffer (SSD1306_ADDR, Data_Buffer);
    end Write_Data;

```

```
procedure Write_Data_Buffer (Data : Display_Buffer; Len :  
    Natural) is  
    Tx_Buffer : Uint8_Array (1 .. Len + 1);  
    Success    : Boolean;  
begin  
    Tx_Buffer (1) := CONTROL_BYTE_DATA;  
    for I in 1 .. Len loop  
        Tx_Buffer (I + 1) := Data (I - 1);  
    end loop;  
    Success := Bus.Write_Buffer (SSD1306_ADDR, Tx_Buffer);  
end Write_Data_Buffer;
```

```
procedure Init is  
begin  
    delay 0.010;  
    Send_Line ("Inicializando SSD1306...");  
  
    Write_Command (SSD1306_DISPLAYOFF);  
    Write_Command (SSD1306_SETDISPLAYCLOCKDIV);  
    Write_Command (16#80#);  
    Write_Command (SSD1306_SETMULTIPLEX);  
    Write_Command (MULTIPLEX_VAL);  
    Write_Command (SSD1306_SETDISPLAYOFFSET);  
    Write_Command (16#00#);  
    Write_Command (SSD1306_SETSTARTLINE or 16#00#);  
    Write_Command (SSD1306_CHARGE_PUMP);  
    Write_Command (16#14#);  
    Write_Command (SSD1306_MEMORYMODE);  
    Write_Command (16#00#);  
    Write_Command (SSD1306_SEGREMAP or 16#01#);  
    Write_Command (SSD1306_COMSCANDEC);  
    Write_Command (SSD1306_SETCOMPINS);  
    Write_Command (COMPINS_VAL);  
    Write_Command (SSD1306_SETCONTRAST);  
    Write_Command (CONTRAST_VAL);  
    Write_Command (SSD1306_SETPRECHARGE);  
    Write_Command (PRECHARGE_VAL);  
    Write_Command (SSD1306_SETVCOMDETECT);  
    Write_Command (16#40#);  
    Write_Command (SSD1306_DISPLAYALLON_RESUME);  
    Write_Command (SSD1306_NORMALDISPLAY);  
    Write_Command (SSD1306_DEACTIVATE_SCROLL);
```

```
    Clear_Display;

    Write_Command (SSD1306_DISPLAYON);
    Send_Line ("Init completado");
    delay 0.010;
end Init;

procedure Clear_Display is
begin
    Frame_Buffer := (others => 0);
    Update_Display;
end Clear_Display;

procedure Display_On is
begin
    Write_Command (SSD1306_DISPLAYON);
end Display_On;

procedure Display_Off is
begin
    Write_Command (SSD1306_DISPLAYOFF);
end Display_Off;

procedure Set_Contrast (Value : Uint8) is
begin
    Write_Command (SSD1306_SETCONTRAST);
    Write_Command (Value);
end Set_Contrast;

procedure Set_Inverse (Inverse : Boolean) is
begin
    if Inverse then
        Write_Command (SSD1306_INVERTDISPLAY);
    else
        Write_Command (SSD1306_NORMALDISPLAY);
    end if;
end Set_Inverse;

procedure Draw_Pixel (X : Uint8; Y : Uint8; b : Boolean) is
    Page : Uint8;
    Bit : Uint8;
    Index : Integer;
begin
```

```
    if X >= WIDTH or else Y >= HEIGHT then
        return;
    end if;
    Page := Y / 8;
    Bit := Y mod 8;
    Index := Integer (Page) * Integer (WIDTH) + Integer (X);
    if Index > Frame_Buffer'Last then
        return;
    end if;
    if b then
        Frame_Buffer (Index) := Frame_Buffer (Index) or Bit_Masks
            (Integer (Bit));
    else
        Frame_Buffer (Index) := Frame_Buffer (Index) and not
            Bit_Masks (Integer (Bit));
    end if;
end Draw_Pixel;

procedure Update_Display is
begin
    Write_Command (SSD1306_COLUMNADDR);
    Write_Command (0);
    Write_Command (COL_MAX);
    Write_Command (SSD1306_PAGEADDR);
    Write_Command (0);
    Write_Command (PAGE_MAX);
    Write_Data_Buffer (Frame_Buffer, BUFFER_SIZE);
end Update_Display;

procedure Put_Char (X : Uint8; Y : Uint8; C : Character) is
    Page : constant Integer := Integer (Y) / 8;
    Index : Integer;
begin
    if C = ' ' then
        for Col in 0 .. 5 loop
            Index := Page * Integer (WIDTH) + Integer (X) + Col;
            if Index <= Frame_Buffer'Last then
                Frame_Buffer (Index) := 0;
            end if;
        end loop;
        return;
    end if;
```

```
if C not in Font'Range then
    return;
end if;

for Col in 0 .. 4 loop
    Index := Page * Integer (WIDTH) + Integer (X) + Col;
    if Index <= Frame_Buffer'Last then
        Frame_Buffer (Index) := Font (C)(Col);
    end if;
end loop;

Index := Page * Integer (WIDTH) + Integer (X) + 5;
if Index <= Frame_Buffer'Last then
    Frame_Buffer (Index) := 0;
end if;
end Put_Char;

procedure Put_String (X : Uint8; Y : Uint8; S : String) is
    Pos_X : Uint8 := X;
begin
    for I in S'Range loop
        exit when Pos_X >= WIDTH;
        Put_Char (Pos_X, Y, S (I));
        Pos_X := Pos_X + 6;
    end loop;
end Put_String;

procedure Draw_Line (X0, Y0, X1, Y1 : Uint8; b : Boolean) is
    DX : Integer := Integer (X1) - Integer (X0);
    DY : Integer := Integer (Y1) - Integer (Y0);
    X : Integer := Integer (X0);
    Y : Integer := Integer (Y0);
    SX : Integer := 1;
    SY : Integer := 1;
    Err : Integer;
    E2 : Integer;
begin
    if DX < 0 then DX := -DX; SX := -1; end if;
    if DY < 0 then DY := -DY; SY := -1; end if;
    Err := DX - DY;
    loop
```



```
    Draw_Pixel (Uint8 (X), Uint8 (Y), b);
    exit when X = Integer (X1) and Y = Integer (Y1);
    E2 := 2 * Err;
    if E2 > -DY then Err := Err - DY; X := X + SX; end if;
    if E2 < DX then Err := Err + DX; Y := Y + SY; end if;
  end loop;
end Draw_Line;

procedure Draw_Rect (X, Y, W, H : Uint8; b : Boolean) is
  X_End : Uint8 := X + W - 1;
  Y_End : Uint8 := Y + H - 1;
begin
  Draw_Line (X,      Y,      X_End, Y,      b);
  Draw_Line (X_End, Y,      X_End, Y_End, b);
  Draw_Line (X_End, Y_End, X,      Y_End, b);
  Draw_Line (X,      Y_End, X,      Y,      b);
end Draw_Rect;

procedure Fill_Rect (X, Y, W, H : Uint8; b : Boolean) is
begin
  for J in 0 .. H - 1 loop
    for I in 0 .. W - 1 loop
      Draw_Pixel (X + I, Y + J, b);
    end loop;
  end loop;
end Fill_Rect;

end SSD1306;
```

Listing 21: ssd1306.adb — Cuerpo del driver OLED

Package Timers

```
with stm32f446; use stm32f446;
with System;

package Timer is

    type Timer_Id is
        (TIM_2, TIM_3, TIM_4, TIM_5,
         TIM_6, TIM_7,
         TIM_1, TIM_8,
         TIM_9, TIM_10, TIM_11,
         TIM_12, TIM_13, TIM_14);

    type Timer_Channel is (CH1, CH2, CH3, CH4);

    type Count_Mode is (Up, Down, Center_Aligned_1, Center_Aligned_2,
                        Center_Aligned_3);

    type OC_Mode is (Frozen, Active, Inactive, Toggle,
                    Force_Inactive, Force_Active,
                    PWM_Mode_1, PWM_Mode_2);

    type OC_Polarity is (Active_High, Active_Low);

    procedure Config_Timer (Tim      : Timer_Id;
                           Prescaler : Uint16;
                           Period    : Uint32;
                           Mode      : Count_Mode := Up);

    procedure Start (Tim : Timer_Id);

    procedure Stop (Tim : Timer_Id);

    procedure Generate_Update (Tim : Timer_Id);

    function  Get_Counter (Tim : Timer_Id) return Uint32;

    procedure Set_Counter (Tim : Timer_Id; Value : Uint32);

    procedure Config_PWM (Tim      : Timer_Id;
                         Channel    : Timer_Channel;
                         Duty       : Uint32;
                         Polarity   : OC_Polarity := Active_High);
```

```

procedure Set_Compare (Tim      : Timer_Id;
                      Channel : Timer_Channel;
                      Value   : Uint32);

function Get_Compare (Tim      : Timer_Id;
                    Channel : Timer_Channel) return Uint32;

procedure Config_Capture (Tim      : Timer_Id;
                        Channel : Timer_Channel;
                        Polarity : OC_Polarity := Active_High);

function Get_Capture (Tim      : Timer_Id;
                    Channel : Timer_Channel) return Uint32;

procedure Enable_Update_Interrupt (Tim : Timer_Id);

procedure Disable_Update_Interrupt (Tim : Timer_Id);

function Update_Flag (Tim : Timer_Id) return Boolean;

procedure Clear_Update_Flag (Tim : Timer_Id);

RCC_APB1ENR : Uint32 with
    Volatile,
    Address => System'To_Address (RCC + APB1ENR_A);

RCC_APB2ENR : Uint32 with
    Volatile,
    Address => System'To_Address (RCC + APB2ENR_A);

end Timer;

```

Listing 22: timers.ads — Especificación del driver TIMERS

```

with stm32f446; use stm32f446;
with System;
with System.Storage_Elements; use System.Storage_Elements;
with System.Address_To_Access_Conversions;

package body Timer is

    package Conv is new System.Address_To_Access_Conversions (Uint32);

    function Bit (N : Natural) return Uint32 is
    begin

```

```

    if N > 31 then
        raise Constraint_Error;
    end if;
    return Uint32 (2 ** N);
end Bit;

function Tim_Base (Tim : Timer_Id) return Storage_Offset is
begin
    case Tim is
        when TIM_1  => return Storage_Offset (TIM1_Base);
        when TIM_2  => return Storage_Offset (TIM2_Base);
        when TIM_3  => return Storage_Offset (TIM3_Base);
        when TIM_4  => return Storage_Offset (TIM4_Base);
        when TIM_5  => return Storage_Offset (TIM5_Base);
        when TIM_6  => return Storage_Offset (TIM6_Base);
        when TIM_7  => return Storage_Offset (TIM7_Base);
        when TIM_8  => return Storage_Offset (TIM8_Base);
        when TIM_9  => return Storage_Offset (TIM9_Base);
        when TIM_10 => return Storage_Offset (TIM10_Base);
        when TIM_11 => return Storage_Offset (TIM11_Base);
        when TIM_12 => return Storage_Offset (TIM12_Base);
        when TIM_13 => return Storage_Offset (TIM13_Base);
        when TIM_14 => return Storage_Offset (TIM14_Base);
    end case;
end Tim_Base;

function Reg (Base : Storage_Offset;
              Offset : Storage_Offset) return access Uint32 is
begin
    return Conv.To_Pointer (System.To_Address (Base + Offset));
end Reg;

function CR1 (Tim : Timer_Id) return access Uint32 is
begin return Reg (Tim_Base (Tim), TIM_CR1_Offset); end CR1;

function DIER (Tim : Timer_Id) return access Uint32 is
begin return Reg (Tim_Base (Tim), TIM_DIER_Offset); end DIER;

function SR (Tim : Timer_Id) return access Uint32 is
begin return Reg (Tim_Base (Tim), TIM_SR_Offset); end SR;

function EGR (Tim : Timer_Id) return access Uint32 is
begin return Reg (Tim_Base (Tim), TIM_EGR_Offset); end EGR;

```

```

function CCMR1 (Tim : Timer_Id) return access Uint32 is
begin return Reg (Tim_Base (Tim), TIM_CCMR1_Offset); end CCMR1;

function CCMR2 (Tim : Timer_Id) return access Uint32 is
begin return Reg (Tim_Base (Tim), TIM_CCMR2_Offset); end CCMR2;

function CCER (Tim : Timer_Id) return access Uint32 is
begin return Reg (Tim_Base (Tim), TIM_CCER_Offset); end CCER;

function CNT (Tim : Timer_Id) return access Uint32 is
begin return Reg (Tim_Base (Tim), TIM_CNT_Offset); end CNT;

function PSC (Tim : Timer_Id) return access Uint32 is
begin return Reg (Tim_Base (Tim), TIM_PSC_Offset); end PSC;

function ARR (Tim : Timer_Id) return access Uint32 is
begin return Reg (Tim_Base (Tim), TIM_ARR_Offset); end ARR;

function CCR (Tim : Timer_Id;
               Ch : Timer_Channel) return access Uint32 is
begin
    case Ch is
        when CH1 => return Reg (Tim_Base (Tim), TIM_CCR1_Offset);
        when CH2 => return Reg (Tim_Base (Tim), TIM_CCR2_Offset);
        when CH3 => return Reg (Tim_Base (Tim), TIM_CCR3_Offset);
        when CH4 => return Reg (Tim_Base (Tim), TIM_CCR4_Offset);
    end case;
end CCR;

function BDTR (Tim : Timer_Id) return access Uint32 is
begin return Reg (Tim_Base (Tim), TIM_BDTR_Offset); end BDTR;

procedure Enable_Clock (Tim : Timer_Id) is
begin
    case Tim is
        when TIM_2  => RCC_APB1ENR := RCC_APB1ENR or Bit (TIM2EN);
        when TIM_3  => RCC_APB1ENR := RCC_APB1ENR or Bit (TIM3EN);
        when TIM_4  => RCC_APB1ENR := RCC_APB1ENR or Bit (TIM4EN);
        when TIM_5  => RCC_APB1ENR := RCC_APB1ENR or Bit (TIM5EN);
        when TIM_6  => RCC_APB1ENR := RCC_APB1ENR or Bit (TIM6EN);
        when TIM_7  => RCC_APB1ENR := RCC_APB1ENR or Bit (TIM7EN);
        when TIM_12 => RCC_APB1ENR := RCC_APB1ENR or Bit (TIM12EN);
        when TIM_13 => RCC_APB1ENR := RCC_APB1ENR or Bit (TIM13EN);
    end case;
end Enable_Clock;

```

```

        when TIM_14 => RCC_APB1ENR := RCC_APB1ENR or Bit (TIM14EN);
        when TIM_1  => RCC_APB2ENR := RCC_APB2ENR or Bit (TIM1EN);
        when TIM_8  => RCC_APB2ENR := RCC_APB2ENR or Bit (TIM8EN);
        when TIM_9  => RCC_APB2ENR := RCC_APB2ENR or Bit (TIM9EN);
        when TIM_10 => RCC_APB2ENR := RCC_APB2ENR or Bit (TIM10EN);
        when TIM_11 => RCC_APB2ENR := RCC_APB2ENR or Bit (TIM11EN);
    end case;
end Enable_Clock;

procedure Config_Timer (Tim : Timer_Id;
                        Prescaler : Uint16;
                        Period : Uint32;
                        Mode : Count_Mode := Up) is
    Cr1_Val : Uint32;
begin
    Enable_Clock (Tim);
    CR1 (Tim).all := 0;
    PSC (Tim).all := Uint32 (Prescaler);
    ARR (Tim).all := Period;

    case Mode is
        when Up => Cr1_Val := 0;
        when Down => Cr1_Val := Bit (CR1_DIR);
        when Center_Aligned_1 => Cr1_Val := Bit (CR1_CMS1);
        when Center_Aligned_2 => Cr1_Val := Bit (CR1_CMS2);
        when Center_Aligned_3 => Cr1_Val := Bit (CR1_CMS1) or Bit
            (CR1_CMS2);
    end case;

    Cr1_Val := Cr1_Val or Bit (CR1_ARPE);
    CR1 (Tim).all := Cr1_Val;
    Generate_Update (Tim);

    if Tim = TIM_1 or else Tim = TIM_8 then
        BDTR (Tim).all := BDTR (Tim).all or Bit (BDTR_MOE);
    end if;
end Config_Timer;

procedure Start (Tim : Timer_Id) is
begin
    CR1 (Tim).all := CR1 (Tim).all or Bit (CR1_CEN);
end Start;

procedure Stop (Tim : Timer_Id) is

```

```
begin
    CR1 (Tim).all := CR1 (Tim).all and not Bit (CR1_CEN);
end Stop;

procedure Generate_Update (Tim : Timer_Id) is
begin
    EGR (Tim).all := Bit (EGR_UG);
end Generate_Update;

function Get_Counter (Tim : Timer_Id) return Uint32 is
begin
    return CNT (Tim).all;
end Get_Counter;

procedure Set_Counter (Tim : Timer_Id; Value : Uint32) is
begin
    CNT (Tim).all := Value;
end Set_Counter;

function OC_Mode_Bits (M : OC_Mode) return Uint32 is
begin
    case M is
        when Frozen => return 2#000#;
        when Active => return 2#001#;
        when Inactive => return 2#010#;
        when Toggle => return 2#011#;
        when Force_Inactive => return 2#100#;
        when Force_Active => return 2#101#;
        when PWM_Mode_1 => return 2#110#;
        when PWM_Mode_2 => return 2#111#;
    end case;
end OC_Mode_Bits;

procedure Config_PWM (Tim : Timer_Id;
                      Channel : Timer_Channel;
                      Duty : Uint32;
                      Polarity : OC_Polarity := Active_High) is
    OCxM : constant Uint32 := OC_Mode_Bits (PWM_Mode_1);
    Shift : Natural;
    Mask : Uint32;
    Val : Uint32;
begin
    CCR (Tim, Channel).all := Duty;
```

```

case Channel is
    when CH1 => Shift := 0;
    when CH2 => Shift := 8;
    when CH3 => Shift := 0;
    when CH4 => Shift := 8;
end case;

Val := (OCxM * Uint32 (2 ** 4)) or Bit (3);
Mask := Uint32 (16#FF#) * Uint32 (2 ** Shift);
Val := Val * Uint32 (2 ** Shift);

if Channel = CH1 or else Channel = CH2 then
    CCMR1 (Tim).all := (CCMR1 (Tim).all and not Mask) or Val;
else
    CCMR2 (Tim).all := (CCMR2 (Tim).all and not Mask) or Val;
end if;

case Channel is
    when CH1 =>
        CCER (Tim).all := CCER (Tim).all or Bit (CCER_CC1E);
        if Polarity = Active_Low then
            CCER (Tim).all := CCER (Tim).all or Bit (CCER_CC1P);
        else
            CCER (Tim).all := CCER (Tim).all and not Bit
                (CCER_CC1P);
        end if;
    when CH2 =>
        CCER (Tim).all := CCER (Tim).all or Bit (CCER_CC2E);
        if Polarity = Active_Low then
            CCER (Tim).all := CCER (Tim).all or Bit (CCER_CC2P);
        else
            CCER (Tim).all := CCER (Tim).all and not Bit
                (CCER_CC2P);
        end if;
    when CH3 =>
        CCER (Tim).all := CCER (Tim).all or Bit (CCER_CC3E);
        if Polarity = Active_Low then
            CCER (Tim).all := CCER (Tim).all or Bit (CCER_CC3P);
        else
            CCER (Tim).all := CCER (Tim).all and not Bit
                (CCER_CC3P);
        end if;
    when CH4 =>
        CCER (Tim).all := CCER (Tim).all or Bit (CCER_CC4E);

```



```

        if Polarity = Active_Low then
            CCER (Tim).all := CCER (Tim).all or Bit (CCER_CC4P);
        else
            CCER (Tim).all := CCER (Tim).all and not Bit
                (CCER_CC4P);
        end if;
    end case;
end Config_PWM;

procedure Set_Compare (Tim : Timer_Id;
                       Channel : Timer_Channel;
                       Value : Uint32) is
begin
    CCR (Tim, Channel).all := Value;
end Set_Compare;

function Get_Compare (Tim : Timer_Id;
                      Channel : Timer_Channel) return Uint32 is
begin
    return CCR (Tim, Channel).all;
end Get_Compare;

procedure Config_Capture (Tim : Timer_Id;
                          Channel : Timer_Channel;
                          Polarity : OC_Polarity := Active_High)
    is
    IC_Val : constant Uint32 := 16#01#;
    Shift : Natural;
    Mask : Uint32;
begin
    case Channel is
        when CH1 => Shift := 0;
        when CH2 => Shift := 8;
        when CH3 => Shift := 0;
        when CH4 => Shift := 8;
    end case;

    Mask := Uint32 (16#FF#) * Uint32 (2 ** Shift);

    if Channel = CH1 or else Channel = CH2 then
        CCMR1 (Tim).all :=
            (CCMR1 (Tim).all and not Mask) or (IC_Val * Uint32 (2 **
                Shift));
    else

```

```

    CCMR2 (Tim).all :=
        (CCMR2 (Tim).all and not Mask) or (IC_Val * Uint32 (2 **
            Shift));
end if;

case Channel is
when CH1 =>
    CCER (Tim).all := CCER (Tim).all or Bit (CCER_CC1E);
    if Polarity = Active_Low then
        CCER (Tim).all := CCER (Tim).all or Bit (CCER_CC1P);
    else
        CCER (Tim).all := CCER (Tim).all and not Bit
            (CCER_CC1P);
    end if;
when CH2 =>
    CCER (Tim).all := CCER (Tim).all or Bit (CCER_CC2E);
    if Polarity = Active_Low then
        CCER (Tim).all := CCER (Tim).all or Bit (CCER_CC2P);
    else
        CCER (Tim).all := CCER (Tim).all and not Bit
            (CCER_CC2P);
    end if;
when CH3 =>
    CCER (Tim).all := CCER (Tim).all or Bit (CCER_CC3E);
    if Polarity = Active_Low then
        CCER (Tim).all := CCER (Tim).all or Bit (CCER_CC3P);
    else
        CCER (Tim).all := CCER (Tim).all and not Bit
            (CCER_CC3P);
    end if;
when CH4 =>
    CCER (Tim).all := CCER (Tim).all or Bit (CCER_CC4E);
    if Polarity = Active_Low then
        CCER (Tim).all := CCER (Tim).all or Bit (CCER_CC4P);
    else
        CCER (Tim).all := CCER (Tim).all and not Bit
            (CCER_CC4P);
    end if;
end case;
end Config_Capture;

function Get_Capture (Tim : Timer_Id;
    Channel : Timer_Channel) return Uint32 is
begin

```

```
        return CCR (Tim, Channel).all;
    end Get_Capture;

    procedure Enable_Update_Interrupt (Tim : Timer_Id) is
    begin
        DIER (Tim).all := DIER (Tim).all or Bit (DIER_UIE);
    end Enable_Update_Interrupt;

    procedure Disable_Update_Interrupt (Tim : Timer_Id) is
    begin
        DIER (Tim).all := DIER (Tim).all and not Bit (DIER_UIE);
    end Disable_Update_Interrupt;

    function Update_Flag (Tim : Timer_Id) return Boolean is
    begin
        return (SR (Tim).all and Bit (SR_UIF)) /= 0;
    end Update_Flag;

    procedure Clear_Update_Flag (Tim : Timer_Id) is
    begin
        SR (Tim).all := SR (Tim).all and not Bit (SR_UIF);
    end Clear_Update_Flag;

end Timer;
```

Listing 23: ssd1306.adb — Cuerpo del driver TIMERS

Paquete Tasks

```
with Ada.Real_Time; use Ada.Real_Time;

package Tasks is

    task Blink with
        Storage_Size => 4096 is
        entry Stop;
    end Blink;

    task Heartbeat with
        Storage_Size => 4096;

end Tasks;
```

Listing 24: tasks.ads — Especificación del paquete Tasks

```
with Ada.Real_Time; use Ada.Real_Time;

package body Tasks is

    task body Blink is
        Period : constant Time_Span := Milliseconds (500);
        Next : Time := Clock;
        Running : Boolean := True;
    begin
        while Running loop
            select
                accept Stop do
                    Running := False;
                end Stop;
            or
                delay until Next;
                Next := Next + Period;
            end select;
        end loop;
    end Blink;

    task body Heartbeat is
        Period : constant Time_Span := Milliseconds (1000);
        Next : Time := Clock;
    begin
        loop
            Next := Next + Period;
```

```
        delay until Next;  
    end loop;  
end Heartbeat;  
  
end Tasks;
```

Listing 25: tasks.adb — Cuerpo del paquete task

Programas de prueba

```
with System;

procedure Main is

    -- Definiciones de direcciones de memoria
    RCC_Base      : constant := 16#4002_3800#;
    GPIOA_Base    : constant := 16#4002_0000#;

    -- Desplazamientos (offsets)
    RCC_AHB1ENR_Offset : constant := 16#30#;
    GPIOx_MODER_Offset  : constant := 16#00#;
    GPIOx_ODR_Offset    : constant := 16#14#;

    -- Tipos
    type Uint32 is mod 2**32;
    for Uint32'Size use 32;

    -- Registros mapeados en memoria
    RCC_AHB1ENR : Uint32 with
        Volatile,
        Address => System'To_Address (RCC_Base + RCC_AHB1ENR_Offset);

    GPIOA_MODER : Uint32 with
        Volatile,
        Address => System'To_Address (GPIOA_Base +
            GPIOx_MODER_Offset);

    GPIOA_ODR : Uint32 with
        Volatile,
        Address => System'To_Address (GPIOA_Base + GPIOx_ODR_Offset);

    -- Constantes
    IOPAEN_Bit      : constant := 0;
    MODER5_Offset   : constant := 10;  -- 5 * 2
    PIN5_High       : constant := 2**5;

begin

    -- Habilitar reloj para GPIOA
    RCC_AHB1ENR := RCC_AHB1ENR or (2**IOPAEN_Bit);

    -- Configurar PA5 como salida
    GPIOA_MODER := GPIOA_MODER and not (3 * 2**MODER5_Offset);
    GPIOA_MODER := GPIOA_MODER or (1 * 2**MODER5_Offset);
```

```
-- Encender LED
GPIOA_ODR := GPIOA_ODR or PIN5_High;

-- Bucle infinito
loop
    null;
end loop;

end Main;
```

Listing 26: test_basic.adb — Programa de prueba del runtime

```
with GPIO;      use GPIO;
with stm32f446; use stm32f446;
with Ada.Real_Time; use Ada.Real_Time;

procedure Main is

    LED : constant GPIO_Point := (Port => PORT_A, Pin => 9);
        --Corresponde a D8 en nucleo
    In_Pin : constant GPIO_Point := (Port => PORT_A, Pin =>
        8); --Corresponde a D7 en nucleo

begin

    Config_Output (LED);
    Config_Input(In_Pin , Pull_Down); -- para que no se quede floating
loop
    if Read(In_Pin) then
        Set (LED);
        delay(1.0);
        Clear (LED);
        delay(1.0);
        Toggle (LED);
        delay(1.0);
        Clear (LED);
    end if;
end loop;

end Main;
```

Listing 27: test_gpio.adb — Programa de prueba de GPIO


```
with stm32f446; use stm32f446;
with GPIO;      use GPIO;
with ADC;       use ADC;
with USART;
with USART_Driver; use USART_Driver;
with Ada.Real_Time; use Ada.Real_Time;

procedure Main is

  -- Pines ADC
  PA0 : constant GPIO_Point := (Port => PORT_A, Pin => 0);
  PA1 : constant GPIO_Point := (Port => PORT_A, Pin => 1);

  -- Resultados
  Raw_CH0 : Uint32;
  Raw_CH1 : Uint32;
  mV_CH0  : Natural;
  mV_CH1  : Natural;
  Temp_Raw : Integer;
  Temp_Int : Integer;
  Temp_Dec : Integer;
  Vref_mV  : Natural;

  function To_String (V : Integer) return String is
    S      : String (1 .. 12) := (others => ' ');
    Idx    : Integer := 12;
    Val    : Integer := V;
    Neg    : Boolean := Val < 0;
  begin
    if Val = 0 then
      return "0";
    end if;
    if Neg then Val := -Val; end if;
    while Val > 0 loop
      S (Idx) := Character'Val (Character'Pos ('0') + Val mod 10);
      Idx := Idx - 1;
      Val := Val / 10;
    end loop;
    if Neg then
```

```

        S (Idx) := '-';
        Idx := Idx - 1;
    end if;
    return S (Idx + 1 .. 12);
end To_String;

function To_String (V : Uint32) return String is
begin
    return To_String (Integer (V));
end To_String;

begin

    Initialize (115200);
    Send_Line ("=====");
    Send_Line ("  STM32F446 ADC Test");
    Send_Line ("  Nucleo-F446RE");
    Send_Line ("=====");
    delay(0.1);

    Config_Analog (PA0);
    Config_Analog (PA1);

    Init (ADC_1, Res_12bit, Right, Single);

    Enable_Temp_Sensor;
    Enable_Vrefint;

    Send_Line ("=====");
    Send_Line ("  STM32F446 ADC Test");
    Send_Line ("  Nucleo-F446RE");
    Send_Line ("=====");
    delay(0.1);

loop

```

```
Raw_CH0 := Read      (ADC_1, 0);
mV_CH0  := Read_mV   (ADC_1, 0);
    Send_Line ("--- Lectura ADC ---");
    Send_Line ("CH0 (PA0) raw : " & To_String (Raw_CH0));
    Send_Line ("CH0 (PA0) mV  : " & To_String (mV_CH0));

Raw_CH1 := Read      (ADC_1, 1);
mV_CH1  := Read_mV   (ADC_1, 1);
    Send_Line ("CH1 (PA1) raw : " & To_String (Raw_CH1));
    Send_Line ("CH1 (PA1) mV  : " & To_String (mV_CH1));

Temp_Raw := Read_Temperature;
Temp_Int := Temp_Raw / 100;
Temp_Dec := abs (Temp_Raw mod 100);
    Send_Line ("Temp raw      : " & To_String (Temp_Raw));
    Send_Line ("Temp          : " & To_String (Temp_Int)
               & "."
               & To_String (Temp_Dec)
               & " C");

Vref_mV := Read_Vrefint_mV;
    Send_Line ("Vrefint      : " & To_String (Vref_mV) & "
               mV");
    Send_Line ("-----");

delay(5.0);

end loop;

end Main;
```

Listing 28: test_adc.adb — Programa de prueba de ADC

```
with System;
with USART;
with USART_Driver; use USART_Driver;
with stm32f446;      use stm32f446;
with Ada.Real_Time; use Ada.Real_Time;
with SPI;
with SPI_driver; use SPI_driver;
-- Pin 11 =PA7=MOSI y 12=PA6=MISO para Nucleo

procedure Main is
  TX : Uint8 := 16#55#;
  RX : Uint8;
  Next_Time : Time := Clock;

begin

  USART_Driver.Initialize (115200);
  USART_Driver.Send_Line ("USART INICIALIZADO");
  SPI_Driver.Initialize;
  USART_Driver.Send_Line ("SPI INICIALIZADO");
  loop
    RX := SPI_Driver.Transfer_8 (TX);
    if RX = TX then
      USART_Driver.Send_Line ("SPI OK");
    else
      USART_Driver.Send_Line ("SPI ERROR");
    end if;
    Next_Time := Next_Time + Milliseconds (500);
    delay until Next_Time;
  end loop;
end Main;
```

Listing 29: test_spi.adb — Programa de prueba de SPI

```
with Ada.Text_IO;           use Ada.Text_IO;
with Ada.Real_Time;        use Ada.Real_Time;
with stm32f446;            use stm32f446;
with I2C;
with I2C_Driver; use I2C_Driver;
with SSD1306;              use SSD1306;
with USART;
with USART_Driver; use USART_Driver;

procedure Main is

  procedure Wait (ms : Natural) is
  begin
    delay Duration (ms) / 1000.0;
  end Wait;

  procedure Test_Text is
  begin
    Clear_Display;

    Put_String (0, 0, "HOLA MUNDO!");
    Put_String (0, 8, "SSD1306 OLED");
    Put_String (0, 16, "Ada + I2C");
    Put_String (0, 24, "1234567890");
    Update_Display;
    Wait (2000);
  end Test_Text;

  procedure Test_Pixels is
  begin
    Clear_Display;

    for I in 0 .. 31 loop
      Draw_Pixel (0, Uint8 (I), True);
      Draw_Pixel (127, Uint8 (I), True);
    end loop;
    for I in 0 .. 127 loop
      Draw_Pixel (Uint8 (I), 0, True);
      Draw_Pixel (Uint8 (I), 31, True);
    end loop;
    Update_Display;
    Wait (2000);
```

```
end Test_Pixels;

procedure Test_Lines is
begin
    Clear_Display;
    -- Bordes
    Draw_Line (0, 0, 127, 0, True);
    Draw_Line (0, 31, 127, 31, True);
    Draw_Line (0, 0, 0, 31, True);
    Draw_Line (127, 0, 127, 31, True);
    -- Diagonales
    Draw_Line (0, 0, 127, 31, True);
    Draw_Line (0, 31, 127, 0, True);
    -- Cruz central
    Draw_Line (0, 16, 127, 16, True);
    Draw_Line (64, 0, 64, 31, True);
    Update_Display;
    Wait (2000);
end Test_Lines;

procedure Test_Rectangles is
begin
    Clear_Display;

    Draw_Rect (0, 0, 128, 32, True);
    Draw_Rect (10, 4, 108, 24, True);
    Draw_Rect (20, 8, 88, 16, True);
    Update_Display;
    Wait (2000);
end Test_Rectangles;

procedure Test_Filled_Rectangles is
begin
    Clear_Display;
    Put_String (0, 0, "Barras:");
    Fill_Rect (0, 8, 100, 6, True);
    Fill_Rect (0, 16, 75, 6, True);
    Fill_Rect (0, 24, 50, 6, True);
    Update_Display;
    Wait (2000);
end Test_Filled_Rectangles;
```

```
procedure Test_Effects is
begin

    for C in 0 .. 255 loop
        Set_Contrast (Uint8 (C));
        Wait (3);
    end loop;
    for C in reverse 0 .. 255 loop
        Set_Contrast (Uint8 (C));
        Wait (3);
    end loop;

    Set_Inverse (True); Wait (500);
    Set_Inverse (False); Wait (500);

    Set_Contrast (CONTRAST_VAL);
end Test_Effects;


procedure Test_Animation is
    X_Pos : Integer := 0;
    Y_Pos : Integer := 0;
    Step  : Integer := 0;
begin
    for Frame in 1 .. 50 loop
        Clear_Display;
        for DX in 0 .. 5 loop
            for DY in 0 .. 5 loop
                if X_Pos + DX <= 127 and Y_Pos + DY <= 31 then
                    Draw_Pixel (Uint8 (X_Pos + DX),
                                Uint8 (Y_Pos + DY), True);
                end if;
            end loop;
        end loop;

        Put_String (0, 24, "Frame:");
        declare
            Frame_Str : String := Integer'Image (Frame);
        begin
            Put_String (44, 24,
                        Frame_Str (Frame_Str'First + 1 .. Frame_Str'Last));
        end;
    end loop;
end;
```

```
Update_Display;

Step := Step + 1;
case Step mod 4 is
    when 0 => X_Pos := X_Pos + 5;
    when 1 => Y_Pos := Y_Pos + 5;
    when 2 => X_Pos := X_Pos - 5;
    when 3 => Y_Pos := Y_Pos - 5;
    when others => null;
end case;

if X_Pos < 0 then X_Pos := 0; Step := 0; end if;
if X_Pos > 122 then X_Pos := 122; Step := 2; end if;
if Y_Pos < 0 then Y_Pos := 0; Step := 1; end if;
if Y_Pos > 26 then Y_Pos := 26; Step := 3; end if;

Wait (100);
end loop;
end Test_Animation;

procedure Test_Full_Demo is
begin

    Clear_Display;
    Put_String (10, 8, "SSD1306 DEMO");
    Put_String (10, 18, "Ada + I2C");
    Update_Display;
    Wait (2000);

    Clear_Display;
    for Line in 0 .. 3 loop
        declare
            Y_Pos : Uint8 := Uint8 (Line * 8);
            Msg : String := "Linea " & Integer'Image (Line + 1);
        begin
            Put_String (0, Y_Pos, Msg);
        end;
    end loop;
    Update_Display;
    Wait (2000);
```



```
    Clear_Display;
    Draw_Rect (0, 0, 60, 32, True);
    Fill_Rect (65, 4, 60, 24, True);
    Draw_Line (0, 0, 127, 31, True);
    Update_Display;
    Wait (2000);

    Clear_Display;
    Put_String (0, 0, "!@#$$%^&*()_+");
    Put_String (0, 8, "{ } [ ] : ; < > ? , . /");
    Put_String (0, 16, "` ~ - = | \ \");
    Update_Display;
    Wait (2000);

end Test_Full_Demo;

begin
    PU.Initialize (115200);
    PU.Send_Line ("USART INICIALIZADO");
    Wait (100);
    PU.Send_Line ("I2C_CONFIG:");
    Bus.Mostrar_Config;
    --Bus.Test_Hardware;
    --Bus.Scan_I2C_Bus;
    --Bus.Test_Minimo;
    Bus.Initialize;
    PU.Send_Line ("I2C INICIALIZADO");

    declare
        SR2 : constant Uint32 := I2C_SR2;
        CR1 : constant Uint32 := I2C_CR1;
    begin
        if (SR2 and Uint32 (2)) /= 0 then
            PU.Send_Line ("DIAG: bus BUSY!");
        else
            PU.Send_Line ("DIAG: bus libre OK");
        end if;
        if (CR1 and Uint32 (2**I2C_CR1_PE)) /= 0 then
            PU.Send_Line ("DIAG: PE=1 OK");
        else
            PU.Send_Line ("DIAG: PE=0 PROBLEMA");
        end if;
    end;
```

```
        end if;
    end;

    Wait (100);
    SSD1306.Init;
    PU.Send_Line ("SSD1306 INICIALIZADO");
    Wait (1500);

    Test_Text;          PU.Send_Line ("Test_Text OK");
    Test_Pixels;        PU.Send_Line ("Test_Pixels OK");
    Test_Lines;         PU.Send_Line ("Test_Lines OK");
    Test_Rectangles;    PU.Send_Line ("Test_Rectangles OK");
    Test_Filled_Rectangles; PU.Send_Line ("Test_Filled OK");
    Test_Effects;       PU.Send_Line ("Test_Effects OK");
    Test_Animation;     PU.Send_Line ("Test_Animation OK");
    Test_Full_Demo;     PU.Send_Line ("Test_Full_Demo OK");

    Clear_Display;
    Put_String (20, 12, "PRUEBAS OK");
    Update_Display;

exception
    when others => null;
end Main;
```

Listing 30: test_ssd1306.adb — Programa de prueba de SSD1306

```
-- 763
with System;
with stm32f446; use stm32f446;
with USART;
with USART_Driver; use USART_Driver;
with Ada.Real_Time; use Ada.Real_Time;

procedure Main is

    -- LED en PA5
    LED_Port : constant := 16#4002_0000#;  -- GPIOA
    LED_Pin   : constant := 5;
    LED_Mask  : constant := 2**LED_Pin;

    GPIOA_BSRR : Uint32 with
        Volatile,
        Address => System'To_Address (LED_Port + 16#18#);

    procedure LED_On is
    begin
        GPIOA_BSRR := LED_Mask;
    end LED_On;

    procedure LED_Off is
    begin
        GPIOA_BSRR := LED_Mask * 2**16;
    end LED_Off;

    procedure LED_Toggle is
    begin
        GPIOA_ODR : Uint32 with
            Volatile,
            Address => System'To_Address (LED_Port + 16#14#);
        GPIOA_ODR := GPIOA_ODR xor LED_Mask;
    end LED_Toggle;

    Received : Uint8;
    Counter : Integer := 0;

begin
    -- Configurar LED como salida
    declare
        GPIOA_MODER : Uint32 with
```

```
    Volatile,
    Address => System'To_Address (LED_Port + 16#00#);
begin
    GPIOA_MODER := GPIOA_MODER and not (3 * 2**(LED_Pin * 2));
    GPIOA_MODER := GPIOA_MODER or (1 * 2**(LED_Pin * 2));    -- 01 =
    Sal
end;

Initialize (115200);

delay(0.1);

Send_Line ("");

Send_Line ("");
Send_Line ("USART2 a 115200 baudios");
Send_Line ("");
Send_Line ("Comandos disponibles:");
Send_Line (" '1' - LED");
Send_Line (" '0' - of LED");
Send_Line (" 't' - Toggle");
Send_Line (" 'c' - Mostrar contador");
Send_Line (" 'h' help");
Send_Line ("");
Send_String ("Listo> ");

loop

    if      Data_Available then
        Received :=      Read_Char;
        Send_Char (Received);

        case Received is
            when Character'Pos ('1') =>
                LED_On;
                Send_Line (" LED encendido");
                Send_String ("Listo> ");

            when Character'Pos ('0') =>
                LED_Off;
```

```
        Send_Line (" LED apagado");
        Send_String ("Listo> ");

when Character'Pos ('t') =>
    LED_Toggle;
    Send_Line (" LED toggle");
    Send_String ("Listo> ");

when Character'Pos ('c') =>
    Counter := Counter + 1;
    Send_String (" Contador: ");

declare
    Num : Integer := Counter;
    Temp : String (1 .. 10);
    Len : Integer := 0;
begin
    if Num = 0 then
        Send_String ("0");
    else
        while Num > 0 loop
            Len := Len + 1;
            Temp (Len) := Character'Val (Character'Pos
                ('0') + (Num mod 10));
            Num := Num / 10;
        end loop;
        for I in reverse 1 .. Len loop
            Send_Char (Character'Pos (Temp (I)));
        end loop;
    end if;
end;

    Send_Line ("");
    Send_String ("Listo> ");

when Character'Pos ('h') | Character'Pos ('?') =>
    Send_Line ("");
    Send_Line ("Comandos:");
    Send_Line (" 1 - Encender LED");
    Send_Line (" 0 - Apagar LED");
    Send_Line (" t - Toggle LED");
    Send_Line (" c - Mostrar contador");
    Send_Line (" h/? - Mostrar ayuda");
    Send_String ("Listo> ");
```

```
when 13 | 10 =>  -- Para hacer salto de línea le ponemos
                un null

                null;

when others =>
    Send_Line (" comando no reconocido (usa 'h' para
               ayuda)");
    Send_String ("Listo> ");
end case;
end if;

-- Indicar que vive
LED_On;

delay(0.01);
LED_Off;

delay(1.0);
end loop;

end Main;
```

Listing 31: test_usart.adb — Programa de prueba de USART

```
with Tasks;  
with Ada.Real_Time; use Ada.Real_Time;  
  
procedure Main is  
begin  
    delay until Clock + Milliseconds (3000);  
    Tasks.Blink.Stop;  
end Main;
```

Listing 32: test_tasks.adb — Programa de prueba de las tareas

```

with USART_Driver;   use USART_Driver;
with STM32F446;      use STM32F446;
with Ada.Real_Time;  use Ada.Real_Time;
with SPI_Driver;     use SPI_Driver;
with W5500;          use W5500;
with System;

procedure Main is

    My_MAC : Uint8_Array (1 .. 6) := (16#DE#, 16#AD#, 16#BE#, 16#EF#,
        16#FE#, 16#ED#);
    My_IP   : Uint8_Array (1 .. 4) := (192, 168, 1, 180);
    My_SN   : Uint8_Array (1 .. 4) := (255, 255, 255, 0);
    My_GW   : Uint8_Array (1 .. 4) := (192, 168, 1, 1);
    Google  : Uint8_Array (1 .. 4) := (8, 8, 8, 8);
    DNS_IP  : Uint8_Array (1 .. 4);
    DNS_Server : constant Uint8_Array (1 .. 4) := (192, 168, 1, 1);

    Server_IP   : Uint8_Array (1 .. 4) := (192, 168, 1, 1);
    Server_Port : constant Uint16 := 80;
    Local_Port  : constant Uint16 := 5000;

    Version      : Uint8;
    Status       : Uint8;
    Timeout      : Natural;
    Next_Time    : Time := Clock;

    -- Reset hardware W5500
    W5500_RST : constant Natural := 7;

    GPIOC_MODER   : Uint32 with Volatile, Address =>
        System'To_Address (GPIOC + 16#00#);
    GPIOC_OSPEEDR : Uint32 with Volatile, Address =>
        System'To_Address (GPIOC + 16#08#);
    GPIOC_PUPDR   : Uint32 with Volatile, Address =>
        System'To_Address (GPIOC + 16#0C#);
    GPIOC_ODR     : Uint32 with Volatile, Address =>
        System'To_Address (GPIOC + 16#14#);
    RCC_AHB1ENR   : Uint32 with Volatile, Address =>
        System'To_Address (RCC   + 16#30#);

    procedure W5500_HW_Reset is
    begin
        RCC_AHB1ENR := RCC_AHB1ENR or Uint32 (2**GPIOCEN);

```



```

GPIOC_MODER := (GPIOC_MODER
    and not (Uint32 (2**(2*W5500_RST)))
    and not (Uint32 (2**((2*W5500_RST)+1))))
    or Uint32 (2**(2*W5500_RST));
GPIOC_OSPEEDR := GPIOC_OSPEEDR
    and not (Uint32 (2**(2*W5500_RST)))
    and not (Uint32 (2**((2*W5500_RST)+1)));
GPIOC_PUPDR := GPIOC_PUPDR
    and not (Uint32 (2**(2*W5500_RST)))
    and not (Uint32 (2**((2*W5500_RST)+1)));
GPIOC_ODR := GPIOC_ODR and not (Uint32 (2**W5500_RST));
Next_Time := Clock + Milliseconds (2);
delay until Next_Time;
GPIOC_ODR := GPIOC_ODR or Uint32 (2**W5500_RST);
Next_Time := Clock + Milliseconds (200);
delay until Next_Time;
end W5500_HW_Reset;

procedure Print_IP (Label : String; IP : Uint8_Array) is
begin
    USART_Driver.Send_Line (Label
        & Integer'Image (Integer (IP (IP'First)))      & "."
        & Integer'Image (Integer (IP (IP'First + 1))) & "."
        & Integer'Image (Integer (IP (IP'First + 2))) & "."
        & Integer'Image (Integer (IP (IP'First + 3))));
end Print_IP;

procedure Check_Link is
    Phy : Uint8;
begin
    Phy := W5500.Read_PHYCFGR;
    W5500.Print_Hex ("PHYCFGR: ", Phy);
    if (Phy and 16#01#) /= 0 then
        USART_Driver.Send_Line ("Link Ethernet OK");
    else
        USART_Driver.Send_Line ("Sin link - verifica cable");
    end if;
end Check_Link;

procedure Verify_Configuration is
    MAC_R : Uint8_Array (1 .. 6);
    IP_R   : Uint8_Array (1 .. 4);

```

```
begin
    W5500.Get_MAC (MAC_R);
    USART_Driver.Send_Line ("--- Verificacion ---");
    for I in MAC_R'Range loop
        Print_Hex ("MAC[" & Integer'Image (I) & "]: ", MAC_R (I));
    end loop;
    W5500.Get_IP (IP_R);
    Print_IP ("IP leida: ", IP_R);
end Verify_Configuration;

begin

    USART_Driver.Initialize (115200);
    USART_Driver.Send_Line ("=== W5500 Test ===");
    SPI_Driver.Initialize;
    USART_Driver.Send_Line ("SPI OK");

    USART_Driver.Send_Line ("Reseteando W5500...");
    W5500_HW_Reset;
    USART_Driver.Send_Line ("Reset OK");

    Version := W5500.Read_Version;
    Print_Hex ("W5500 VERSION: ", Version);
    if Version /= 16#04# then
        USART_Driver.Send_Line ("ERROR: W5500 no responde");
        loop null; end loop;
    end if;
    USART_Driver.Send_Line ("W5500 OK");

    Check_Link;

    W5500.Set_Gateway (My_GW);
    W5500.Set_Subnet (My_SN);
    W5500.Set_MAC (My_MAC);
    W5500.Set_IP (My_IP);
    USART_Driver.Send_Line ("Red configurada");
    Verify_Configuration;

    if W5500.Ping (Google, 3000) then
        USART_Driver.Send_Line ("Gateway alcanzable");
    else
```

```
    USART_Driver.Send_Line ("Gateway no responde");
end if;

W5500.Socket0_Open_TCP (Local_Port);
USART_Driver.Send_Line ("Socket0 OPEN");

Timeout := 0;
loop
    Status := W5500.Socket0_Status;
    exit when Status = SOCK_INIT;
    Timeout := Timeout + 1;
    exit when Timeout > 1000;
    Next_Time := Clock + Milliseconds (1);
    delay until Next_Time;
end loop;
Print_Hex ("Socket0 status: ", Status);

if Status /= SOCK_INIT then
    USART_Driver.Send_Line ("ERROR: socket no llego a INIT");
    loop null; end loop;
end if;

W5500.Socket0_Connect (Server_IP, Server_Port);
USART_Driver.Send_Line ("CONNECT enviado");

Timeout := 0;
loop
    Status := W5500.Socket0_Status;
    exit when Status = SOCK_ESTABLISHED;
    exit when Status = SOCK_CLOSED;
    Timeout := Timeout + 1;
    exit when Timeout > 3000;
    Next_Time := Clock + Milliseconds (1);
    delay until Next_Time;
end loop;
Print_Hex ("Socket0 final: ", Status);

if Status = SOCK_ESTABLISHED then
    USART_Driver.Send_Line ("TCP CONNECTED");
else
    USART_Driver.Send_Line ("Conexion fallida");
end if;
```

```

W5500.Socket0_Close;
USART_Driver.Send_Line ("Socket cerrado");

if Resolve_DNS ("uhu.es", DNS_Server, DNS_IP) then
    -- 150.214.167.13
    if Ping (DNS_IP) then
        Print_IP ("IP de url: ", DNS_IP);
        USART_Driver.Send_Line (" Dns funcional y destino
                                alcanzable");
    end if;
end if;

mDNS_Announce("ejemplo.local");

loop
    HTTP_Server_mDNS (80, "ejemplo.local");
end loop;

loop
    Next_Time := Clock + Milliseconds (1000);
    delay until Next_Time;
end loop;
end Main; with USART_Driver; use USART_Driver;
with STM32F446; use STM32F446;
with Ada.Real_Time; use Ada.Real_Time;
with SPI_Driver; use SPI_Driver;
with W5500; use W5500;
with System;

procedure Main is

    My_MAC : Uint8_Array (1 .. 6) := (16#DE#, 16#AD#, 16#BE#, 16#EF#,
    16#FE#, 16#ED#);
    My_IP : Uint8_Array (1 .. 4) := (192, 168, 1, 180);
    My_SN : Uint8_Array (1 .. 4) := (255, 255, 255, 0);
    My_GW : Uint8_Array (1 .. 4) := (192, 168, 1, 1);
    Google : Uint8_Array (1 .. 4) := (8, 8, 8, 8);
    DNS_IP : Uint8_Array (1 .. 4);
    DNS_Server : constant Uint8_Array (1 .. 4) := (192, 168, 1, 1);

```

```

DHCP :   DHCP_Result ;

Server_IP   :   Uint8_Array (1 .. 4) := (192, 168, 1, 1);
Server_Port :   constant Uint16 := 80;
Local_Port  :   constant Uint16 := 5000;

Version     :   Uint8;
Status      :   Uint8;
Timeout     :   Natural;
Next_Time   :   Time := Clock;

-- Reset hardware W5500
W5500_RST :   constant Natural := 7;

GPIOC_MODER    :   Uint32 with Volatile, Address =>
    System'To_Address (GPIOC + 16#00#);
GPIOC_OSPEEDR  :   Uint32 with Volatile, Address =>
    System'To_Address (GPIOC + 16#08#);
GPIOC_PUPDR    :   Uint32 with Volatile, Address =>
    System'To_Address (GPIOC + 16#0C#);
GPIOC_ODR      :   Uint32 with Volatile, Address =>
    System'To_Address (GPIOC + 16#14#);
RCC_AHB1ENR    :   Uint32 with Volatile, Address =>
    System'To_Address (RCC   + 16#30#);

procedure W5500_HW_Reset is
begin
    RCC_AHB1ENR := RCC_AHB1ENR or Uint32 (2**GPIOCEN);
    GPIOC_MODER := (GPIOC_MODER
        and not (Uint32 (2**((2*W5500_RST))))
        and not (Uint32 (2**((2*W5500_RST)+1)))))
        or Uint32 (2**((2*W5500_RST)));
    GPIOC_OSPEEDR := GPIOC_OSPEEDR
        and not (Uint32 (2**((2*W5500_RST))))
        and not (Uint32 (2**((2*W5500_RST)+1))));
    GPIOC_PUPDR := GPIOC_PUPDR
        and not (Uint32 (2**((2*W5500_RST))))
        and not (Uint32 (2**((2*W5500_RST)+1))));
    GPIOC_ODR := GPIOC_ODR and not (Uint32 (2**W5500_RST));
    Next_Time := Clock + Milliseconds (2);
    delay until Next_Time;
    GPIOC_ODR := GPIOC_ODR or Uint32 (2**W5500_RST);
    Next_Time := Clock + Milliseconds (200);
    delay until Next_Time;

```

```
end W5500_HW_Reset;

procedure Print_IP (Label : String; IP : Uint8_Array) is
begin
    USART_Driver.Send_Line (Label
        & Integer'Image (Integer (IP (IP'First)))      & "."
        & Integer'Image (Integer (IP (IP'First + 1))) & "."
        & Integer'Image (Integer (IP (IP'First + 2))) & "."
        & Integer'Image (Integer (IP (IP'First + 3))));
end Print_IP;

procedure Check_Link is
    Phy : Uint8;
begin
    Phy := W5500.Read_PHYCFGR;
    W5500.Print_Hex ("PHYCFGR: ", Phy);
    if (Phy and 16#01#) /= 0 then
        USART_Driver.Send_Line ("Link Ethernet OK");
    else
        USART_Driver.Send_Line ("Sin link - verifica cable");
    end if;
end Check_Link;

procedure Verify_Configuration is
    MAC_R : Uint8_Array (1 .. 6);
    IP_R   : Uint8_Array (1 .. 4);
begin
    W5500.Get_MAC (MAC_R);
    USART_Driver.Send_Line ("--- Verificacion ---");
    for I in MAC_R'Range loop
        Print_Hex ("MAC[" & Integer'Image (I) & "]: ", MAC_R (I));
    end loop;
    W5500.Get_IP (IP_R);
    Print_IP ("IP leida: ", IP_R);
end Verify_Configuration;

begin

    USART_Driver.Initialize (115200);
    USART_Driver.Send_Line ("=== W5500 Test ===");
    SPI_Driver.Initialize;
    USART_Driver.Send_Line ("SPI OK");
```

```
USART_Driver.Send_Line ("Reseteando W5500...");
W5500_HW_Reset;
USART_Driver.Send_Line ("Reset OK");

Version := W5500.Read_Version;
Print_Hex ("W5500 VERSION: ", Version);
if Version /= 16#04# then
    USART_Driver.Send_Line ("ERROR: W5500 no responde");
    loop null; end loop;
end if;
USART_Driver.Send_Line ("W5500 OK");

Check_Link;

W5500.Set_MAC      (My_MAC);
DHCP:=W5500.DHCP_Request;
if DHCP.Success then
    w5500.Set_IP      (DHCP.IP);
    w5500.Set_Subnet  (DHCP.Subnet);
    w5500.Set_Gateway (DHCP.Gateway);
else

W5500.Set_Gateway (My_GW);
W5500.Set_Subnet  (My_SN);
W5500.Set_IP      (My_IP);

end if;
USART_Driver.Send_Line ("Red configurada");
Verify_Configuration;

if W5500.Ping (Google, 3000) then
    USART_Driver.Send_Line ("Gateway alcanzable");
else
    USART_Driver.Send_Line ("Gateway no responde");
end if;

W5500.Socket0_Open_TCP (Local_Port);
USART_Driver.Send_Line ("Socket0 OPEN");
```

```
Timeout := 0;
loop
    Status := W5500.Socket0_Status;
    exit when Status = SOCK_INIT;
    Timeout := Timeout + 1;
    exit when Timeout > 1000;
    Next_Time := Clock + Milliseconds (1);
    delay until Next_Time;
end loop;
Print_Hex ("Socket0 status: ", Status);

if Status /= SOCK_INIT then
    USART_Driver.Send_Line ("ERROR: socket no llego a INIT");
    loop null; end loop;
end if;

W5500.Socket0_Connect (Server_IP, Server_Port);
USART_Driver.Send_Line ("CONNECT enviado");

Timeout := 0;
loop
    Status := W5500.Socket0_Status;
    exit when Status = SOCK_ESTABLISHED;
    exit when Status = SOCK_CLOSED;
    Timeout := Timeout + 1;
    exit when Timeout > 3000;
    Next_Time := Clock + Milliseconds (1);
    delay until Next_Time;
end loop;
Print_Hex ("Socket0 final: ", Status);

if Status = SOCK_ESTABLISHED then
    USART_Driver.Send_Line ("TCP CONNECTED");
else
    USART_Driver.Send_Line ("Conexion fallida");
end if;

W5500.Socket0_Close;
USART_Driver.Send_Line ("Socket cerrado");
```



```
if Resolve_DNS ("uhu.es", DNS_Server, DNS_IP) then
    -- 150.214.167.13
    if Ping (DNS_IP) then
        Print_IP ("IP de url: ", DNS_IP);
        USART_Driver.Send_Line (" Dns funcional y destino
                                alcanzable");
    end if;
end if;

mDNS_Announce("ejemplo.local");

loop
    HTTP_Server_mDNS (80, "ejemplo.local");
end loop;

loop
    Next_Time := Clock + Milliseconds (1000);
    delay until Next_Time;
end loop;
end Main;
```

Listing 33: test_w5500.adb — Programa de prueba de W5500

Programas de demostración final

```
with Ada.Real_Time; use Ada.Real_Time;

package TaskPant is
  task Pintar_Task with
    Priority => 10,          -- prioridad estática obligatoria en
                           Ravenscar
    Storage_Size => 1024; -- tamaño de pila explícito

end TaskPant;
```

Listing 34: taskpant.adb —Especificación del programa de integración de la pantalla

```
with Ada.Text_IO;      use Ada.Text_IO;
with Ada.Real_Time;    use Ada.Real_Time;
with stm32f446;        use stm32f446;
with I2C;
with I2C_Driver; use I2C_Driver;
with SSD1306;          use SSD1306;
```

```
package body TaskPant is

  task body Pintar_Task is
    procedure Wait (ms : Natural) is
    begin
      delay Duration (ms) / 1000.0;
    end Wait;

    procedure Test_Text is
    begin
      Clear_Display;

      Put_String (0, 0, "HOLA MUNDO!");
      Put_String (0, 8, "SSD1306 OLED");
      Put_String (0, 16, "Ada + I2C");
      Put_String (0, 24, "1234567890");
      Update_Display;
      Wait (2000);
    end Test_Text;
```

```
procedure Test_Pixels is
begin
    Clear_Display;
    -- Bordes para 128x32
    for I in 0 .. 31 loop
        Draw_Pixel (0,    Uint8 (I), True);
        Draw_Pixel (127, Uint8 (I), True);
    end loop;
    for I in 0 .. 127 loop
        Draw_Pixel (Uint8 (I), 0,    True);
        Draw_Pixel (Uint8 (I), 31, True);
    end loop;
    Update_Display;
    Wait (2000);
end Test_Pixels;
```

```
procedure Test_Lines is
begin
    Clear_Display;
    -- Bordes
    Draw_Line (0,    0,    127, 0,    True);
    Draw_Line (0,    31, 127, 31, True);
    Draw_Line (0,    0,    0,    31, True);
    Draw_Line (127, 0,    127, 31, True);
    -- Diagonales
    Draw_Line (0, 0,    127, 31, True);
    Draw_Line (0, 31, 127, 0,    True);
    -- Cruz central
    Draw_Line (0,    16, 127, 16, True);
    Draw_Line (64,    0,    64, 31, True);
    Update_Display;
    Wait (2000);
end Test_Lines;
```

```
procedure Test_Rectangles is
begin
    Clear_Display;

    Draw_Rect (0,    0,    128, 32, True);
    Draw_Rect (10,    4,    108, 24, True);
    Draw_Rect (20,    8,    88,    16, True);
```

```
        Update_Display;
        Wait (2000);
end Test_Rectangles;

procedure Test_Filled_Rectangles is
begin
    Clear_Display;
    Put_String (0, 0, "Barras:");
    Fill_Rect (0, 8, 100, 6, True);
    Fill_Rect (0, 16, 75, 6, True);
    Fill_Rect (0, 24, 50, 6, True);
    Update_Display;
    Wait (2000);
end Test_Filled_Rectangles;

procedure Test_Effects is
begin

    for C in 0 .. 255 loop
        Set_Contrast (Uint8 (C));
        Wait (3);
    end loop;
    for C in reverse 0 .. 255 loop
        Set_Contrast (Uint8 (C));
        Wait (3);
    end loop;
    -- Inversión
    Set_Inverse (True); Wait (500);
    Set_Inverse (False); Wait (500);
    -- Restaurar contraste
    Set_Contrast (CONTRAST_VAL);
end Test_Effects;

procedure Test_Animation is
    X_Pos : Integer := 0;
    Y_Pos : Integer := 0;
    Step  : Integer := 0;
begin
    for Frame in 1 .. 50 loop
        Clear_Display;
        for DX in 0 .. 5 loop
```

```
    for DY in 0 .. 5 loop
        if X_Pos + DX <= 127 and Y_Pos + DY <= 31 then
            Draw_Pixel (Uint8 (X_Pos + DX),
                        Uint8 (Y_Pos + DY), True);
        end if;
    end loop;
end loop;

Put_String (0, 24, "Frame:");
declare
    Frame_Str : String := Integer'Image (Frame);
begin
    Put_String (44, 24,
                Frame_Str (Frame_Str'First + 1 .. Frame_Str'Last));
end;
Update_Display;

Step := Step + 1;
case Step mod 4 is
    when 0 => X_Pos := X_Pos + 5;
    when 1 => Y_Pos := Y_Pos + 5;
    when 2 => X_Pos := X_Pos - 5;
    when 3 => Y_Pos := Y_Pos - 5;
    when others => null;
end case;

if X_Pos < 0 then X_Pos := 0; Step := 0; end if;
if X_Pos > 122 then X_Pos := 122; Step := 2; end if;
if Y_Pos < 0 then Y_Pos := 0; Step := 1; end if;
if Y_Pos > 26 then Y_Pos := 26; Step := 3; end if;

Wait (100);
end loop;
end Test_Animation;

procedure Test_Full_Demo is
begin

    Clear_Display;
    Put_String (10, 8, "SSD1306 DEMO");
    Put_String (10, 18, "Ada + I2C");
    Update_Display;
```

```

    Wait (2000);

    Clear_Display;
    for Line in 0 .. 3 loop
        declare
            Y_Pos : Uint8 := Uint8 (Line * 8);
            Msg    : String := "Linea " & Integer'Image (Line + 1);
        begin
            Put_String (0, Y_Pos, Msg);
        end;
    end loop;
    Update_Display;
    Wait (2000);

    Clear_Display;
    Draw_Rect (0, 0, 60, 32, True);
    Fill_Rect (65, 4, 60, 24, True);
    Draw_Line (0, 0, 127, 31, True);
    Update_Display;
    Wait (2000);

    Clear_Display;
    Put_String (0, 0,  "!@#$$%^&*()_+");
    Put_String (0, 8,  "{ } [ ] : ; < > ? , . / " );
    Put_String (0, 16, " '~ - = | \ \ " );
    Update_Display;
    Wait (2000);

end Test_Full_Demo;

begin

    Wait (100);

    Bus.Mostrar_Config;
    -- Bus.Test_Hardware;
    -- Bus.Scan_I2C_Bus;
    -- Bus.Test_Minimo;
    Bus.Initialize;

```

```
declare
    SR2 : constant Uint32 := I2C_SR2;
    CR1 : constant Uint32 := I2C_CR1;
begin

    Wait (100);
    SSD1306.Init;

    Wait (1500);

    Test_Text;
    Test_Pixels;
    Test_Lines;
    Test_Rectangles;
    Test_Filled_Rectangles;
    Test_Effects;
    Test_Animation;
    Test_Full_Demo;

    Clear_Display;
    Put_String (20, 12, "PRUEBAS OK");
    Update_Display;

exception
    when others => null;
end;
end Pintar_Task;
end TaskPant;
```

Listing 35: taskpant.ads — Cuerpo del programa de integración de la pantalla

```
with Ada.Real_Time; use Ada.Real_Time;

package Tasks is
  task Blink_Task with
    Priority => 10,          -- prioridad estática obligatoria en
                           Ravenscar
    Storage_Size => 1024; -- tamaño de pila explícito

end Tasks;
```

Listing 36: tasks.adb —Especificación del programa de integración de Heartbeat

```
with Ada.Real_Time; use Ada.Real_Time;
with GPIO; use GPIO;
package body Tasks is

  task body Blink_Task is
    Next : Time := Clock;
    LED : constant GPIO_Point := (Port => PORT_A, Pin => 9);
        --Corresponde a D8 en nucleo
  begin
    Config_Output (LED);
    loop
      Toggle (LED);
      Next := Next + Milliseconds (500);
      delay until Next;
    end loop;
  end Blink_Task;

end Tasks;
```

Listing 37: tasks.ads — Cuerpo del programa de Heartbeat


```

with USART_Driver; use USART_Driver;
with STM32F446; use STM32F446;
with Ada.Real_Time; use Ada.Real_Time;
with SPI_Driver; use SPI_Driver;
with W5500; use W5500;
with GPIO; use GPIO;
with ADC; use ADC;
with USART;
with System;
with tasks;
with TaskPant;
procedure Main is
  PA0 : constant GPIO_Point := (Port => PORT_A, Pin => 0);
  PA1 : constant GPIO_Point := (Port => PORT_A, Pin => 1);

  -- Resultados
  Raw_CH0 : Uint32;
  Raw_CH1 : Uint32;
  mV_CH0 : Natural;
  mV_CH1 : Natural;
  Temp_Raw : Integer;
  Temp_Int : Integer;
  Temp_Dec : Integer;
  Vref_mV : Natural;
  My_MAC : Uint8_Array (1 .. 6) := (16#DE#, 16#AD#, 16#BE#, 16#EF#,
    16#FE#, 16#ED#);
  My_IP : Uint8_Array (1 .. 4) := (192, 168, 1, 180);
  My_SN : Uint8_Array (1 .. 4) := (255, 255, 255, 0);
  My_GW : Uint8_Array (1 .. 4) := (192, 168, 1, 1);
  Google : Uint8_Array (1 .. 4) := (8, 8, 8, 8);
  DNS_IP : Uint8_Array (1 .. 4);
  DNS_Server : constant Uint8_Array (1 .. 4) := (192, 168, 1, 1);

  Server_IP : Uint8_Array (1 .. 4) := (192, 168, 1, 1);
  Server_Port : constant Uint16 := 80;
  Local_Port : constant Uint16 := 5000;

  Version : Uint8;
  Status : Uint8;
  Timeout : Natural;
  Next_Time : Time := Clock;

  -- Reset hardware W5500
  W5500_RST : constant Natural := 7;

```

```

GPIOC_MODER      : Uint32 with Volatile, Address =>
    System'To_Address (GPIOC + 16#00#);
GPIOC_OSPEEDR    : Uint32 with Volatile, Address =>
    System'To_Address (GPIOC + 16#08#);
GPIOC_PUPDR      : Uint32 with Volatile, Address =>
    System'To_Address (GPIOC + 16#0C#);
GPIOC_ODR        : Uint32 with Volatile, Address =>
    System'To_Address (GPIOC + 16#14#);
RCC_AHB1ENR      : Uint32 with Volatile, Address =>
    System'To_Address (RCC    + 16#30#);

function To_String (V : Integer) return String is
    S      : String (1 .. 12) := (others => ' ');
    Idx    : Integer := 12;
    Val    : Integer := V;
    Neg    : Boolean := Val < 0;
begin
    if Val = 0 then
        return "0";
    end if;
    if Neg then Val := -Val; end if;
    while Val > 0 loop
        S (Idx) := Character'Val (Character'Pos ('0') + Val mod 10);
        Idx := Idx - 1;
        Val := Val / 10;
    end loop;
    if Neg then
        S (Idx) := '-';
        Idx := Idx - 1;
    end if;
    return S (Idx + 1 .. 12);
end To_String;

function To_String (V : Uint32) return String is
begin
    return To_String (Integer (V));
end To_String;

procedure W5500_HW_Reset is
begin
    RCC_AHB1ENR := RCC_AHB1ENR or Uint32 (2**GPIOCEN);
    GPIOC_MODER := (GPIOC_MODER

```

```

        and not (Uint32 (2**((2*W5500_RST)+1))))
    or Uint32 (2**((2*W5500_RST)+1)))));
GPIOC_OSPEEDR := GPIOC_OSPEEDR
    and not (Uint32 (2**((2*W5500_RST)+1)))));
GPIOC_PUPDR := GPIOC_PUPDR
    and not (Uint32 (2**((2*W5500_RST)+1)))));
GPIOC_ODR := GPIOC_ODR and not (Uint32 (2**W5500_RST));
Next_Time := Clock + Milliseconds (2);
delay until Next_Time;
GPIOC_ODR := GPIOC_ODR or Uint32 (2**W5500_RST);
Next_Time := Clock + Milliseconds (200);
delay until Next_Time;
end W5500_HW_Reset;

procedure Print_IP (Label : String; IP : Uint8_Array) is
begin
    USART_Driver.Send_Line (Label
        & Integer'Image (Integer (IP (IP'First))) & "."
        & Integer'Image (Integer (IP (IP'First + 1))) & "."
        & Integer'Image (Integer (IP (IP'First + 2))) & "."
        & Integer'Image (Integer (IP (IP'First + 3))));
end Print_IP;

procedure Check_Link is
    Phy : Uint8;
begin
    Phy := W5500.Read_PHYCFGR;
    W5500.Print_Hex ("PHYCFGR: ", Phy);
    if (Phy and 16#01#) /= 0 then
        USART_Driver.Send_Line ("Link Ethernet OK");
    else
        USART_Driver.Send_Line ("Sin link - verifica cable");
    end if;
end Check_Link;

procedure Verify_Configuration is
    MAC_R : Uint8_Array (1 .. 6);
    IP_R   : Uint8_Array (1 .. 4);
begin

```

```
W5500.Get_MAC (MAC_R);
USART_Driver.Send_Line ("--- Verificacion ---");
for I in MAC_R'Range loop
    Print_Hex ("MAC[" & Integer'Image (I) & "]: ", MAC_R (I));
end loop;
W5500.Get_IP (IP_R);
Print_IP ("IP leida: ", IP_R);
end Verify_Configuration;

begin

USART_Driver.Initialize (115200);
USART_Driver.Send_Line ("=== W5500 Test ===");
SPI_Driver.Initialize;
USART_Driver.Send_Line ("SPI OK");

Config_Analog (PA0);
Config_Analog (PA1);
Init (ADC_1, Res_12bit, Right, Single);
Enable_Temp_Sensor;
Enable_Vrefint;

USART_Driver.Send_Line ("Reseteando W5500...");
W5500_HW_Reset;
USART_Driver.Send_Line ("Reset OK");

Version := W5500.Read_Version;
Print_Hex ("W5500 VERSION: ", Version);
if Version /= 16#04# then
    USART_Driver.Send_Line ("ERROR: W5500 no responde");
    loop null; end loop;
end if;
USART_Driver.Send_Line ("W5500 OK");

Check_Link;

W5500.Set_Gateway (My_GW);
W5500.Set_Subnet (My_SN);
W5500.Set_MAC (My_MAC);
W5500.Set_IP (My_IP);
USART_Driver.Send_Line ("Red configurada");
```

```
Verify_Configuration;

if W5500.Ping (Google, 3000) then
    USART_Driver.Send_Line ("Gateway alcanzable");
else
    USART_Driver.Send_Line ("Gateway no responde");
end if;

W5500.Socket0_Open_TCP (Local_Port);
USART_Driver.Send_Line ("Socket0 OPEN");

Timeout := 0;
loop
    Status := W5500.Socket0_Status;
    exit when Status = SOCK_INIT;
    Timeout := Timeout + 1;
    exit when Timeout > 1000;
    Next_Time := Clock + Milliseconds (1);
    delay until Next_Time;
end loop;
Print_Hex ("Socket0 status: ", Status);

if Status /= SOCK_INIT then
    USART_Driver.Send_Line ("ERROR: socket no llego a INIT");
    loop null; end loop;
end if;

W5500.Socket0_Connect (Server_IP, Server_Port);
USART_Driver.Send_Line ("CONNECT enviado");

Timeout := 0;
loop
    Status := W5500.Socket0_Status;
    exit when Status = SOCK_ESTABLISHED;
    exit when Status = SOCK_CLOSED;
    Timeout := Timeout + 1;
    exit when Timeout > 3000;
    Next_Time := Clock + Milliseconds (1);
    delay until Next_Time;
end loop;
Print_Hex ("Socket0 final: ", Status);

if Status = SOCK_ESTABLISHED then
```

```

        USART_Driver.Send_Line ("TCP CONNECTED");
    else
        USART_Driver.Send_Line ("Conexion fallida");
    end if;

W5500.Socket0_Close;
USART_Driver.Send_Line ("Socket cerrado");

if Resolve_DNS ("uhu.es", DNS_Server, DNS_IP) then
    -- 150.214.167.13
    if Ping (DNS_IP) then
        Print_IP ("IP de url: ", DNS_IP);
        USART_Driver.Send_Line (" Dns funcional y destino
                                alcanzable");
    end if;
end if;

mDNS_Announce("ejemplo.local");

loop
    Temp_Raw := ADC.Read_Temperature;
    Temp_Int := Temp_Raw / 100;
    Temp_Dec := abs (Temp_Raw mod 100);
    USART_Driver.Send_Line ("<h1>"&To_String (Temp_Raw)&"</h1>");
    HTTP_Server_mDNSe (80, "ejemplo.local", "<h1>Temperatura de
    ref:"&To_String (Temp_Int)&". "&To_String
    (Temp_Dec)&"</h1>");
    -- HTTP_Server_mDNS (80, "ejemplo.local");

end loop;

loop
    Next_Time := Clock + Milliseconds (1000);
    delay until Next_Time;
end loop;
end Main;

```

Listing 38: main.adb —Programa principal y tarea del servidor